Subject: Reinforcement Learning Revisit

Date: from August 11, 2026 to October 15, 2026

Contents

1 Lecture 1: Basic Concepts **1**

1.1 Basic Concepts . . . 1

1.2 Markov Decision Process . . . 3

1.3 Questions and Answers . . . 3

2 Lecture 2: Bellman Equation **4**

2.1 State Value . . . 4

2.2 Bellman Equation . . . 5

2.3 Action Value . . . 7

3 Lecture 3: Bellman Optimality Equation, Value Iteration and Policy Iteration **8**

3.1 Optimal Policy and Optimal State Value . . . 8

3.2 Bellman Optimality Equation . . . 8

3.3 Solving an Optimal Policy from the BOE . . . 9

3.4 Optimality Analysis . . . 10

3.5 Value Iteration . . . 12

3.6 Policy Iteration . . . 13

3.7 Truncated Policy Iteration . . . 15

4 Lecture 4: Monte Carlo Learning **16**

4.1 MC Basic . . . 16

4.2 MC Exploring Starts . . . 17

4.3 MC ϵ-Greedy . . . 18

5 Lecture 5: Stochastic Approximation and Stochastic Gradient Descent **20**

5.1 Robbins-Monro Algorithm (RM Algorithm) . . . 20

5.2 Dvoretzky's Convergence Theorem . . . 22

5.3 Stochastic Gradient Decent . . . 22

6 Lecture 6: Temporal-Difference Learning **24**

6.1 Basic TD: TD Learning of State Values . . . 24

6.2 Sarsa: TD Learning of Action Values . . . 26

6.3 Expected Sarsa: TD Learning of Action Values . . . 27

6.4 n-Step Sarsa: TD Learning of Action Values . . . 28

6.5 Q-Learning: TD Learning of Optimal Action Values . . . 29

6.6 Comparison and Conclusion . . . 32

7 Lecture 7: Value Function Approximation **33**

7.1 Function Approximation . . . 33

7.2 Function Approximation based TD Learning . . . 34

7.3 Theoretical Analysis for TD-Linear . . . 36

7.4 Function Approximation based Sarsa . . . 38

7.5 Deep Q-Learning . . . 38

8 Policy Gradient **41**
8.1 Policy Gradient . . . 41
8.2 Metrics to Define Optimal Policies . . . 41
8.3 Gradients of the Metrics . . . 42
8.4 REINFORCE Algorithm . . . 43

9 Actor-Critic Methods **45**
9.1 Q-Value based Actor-Critic (QAC) Algorithm . . . 45
9.2 Advantage Actor-Critic (A2C) . . . 45
9.3 Off-policy Actor-Critic . . . 47
9.4 Deterministic Actor-Critic (DPG) . . . 49

A Proof of Lecture 2 **50**

B Proof of Lecture 3 **51**

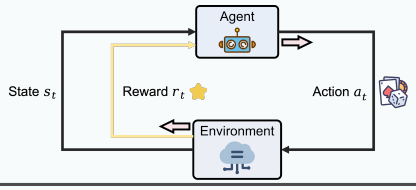
C Proof of Lecture 5 **54**

1 Lecture 1: Basic Concepts

1.1 Basic Concepts

Introduction

强化学习研究的是：一个智能体 (**agent**) 如何在不确定环境 (**environment**) 中通过交互 (**action**) 和奖励反馈 (**reward**) 学习一套能最大化长期期望回报的策略 (**policy**). 其是监督学习和非监督学习外第三种基本 ML 方法.



Example: Grid World

一个直观且容易理解的强化学习环境是网格世界 (**Grid World**). 在网格世界中，可以想象有一个机器人 (agent) 每次只能移动一格，它的目的是在初始位置 (图1.1中绿色) 以"最好"的方式到达终点 (图1.1中黄色)，中途会遇到一些不能进入障碍物 (图1.1中红色). 其每次移动有五种选择 (如图1.2 所示): 上下左右和原地不动.

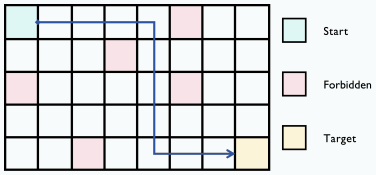


Figure 1.1: Grid world example

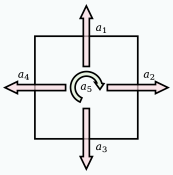


Figure 1.2: Action example

虽然上述例子足够简单，但问题依然可以很复杂，例如 ① 目标是什么？找到一条从起点到达终点的最佳路径还是对任意位置都需要找到最佳路径？② "最好的方式"如何定义？最短的路径、不要撞墙、还是不要撞到障碍物？③ 障碍物是绝对不能进入还是可以但是进入了会有惩罚？惩罚多少？如何施加？④ 当 agent 不知晓周围环境时如何规划路径?

State, Action and Policy

State. 状态 (**state**) 是 agent 当前情况的反映，其需要包含预测未来状态所需要的全部相关信息，例如位置、当前时间、剩余生命值等. LLM 场景中最自然的 token-level state 是 prompt + 当前已生成 tokens. State 一般记为 s_t.

State Space. 状态空间 (**state space**) 是所有可能状态的集合，一般记为 $\mathcal{S} = \{s_t\}_{t=1}^{n}$. 状态空间可以有限离散，例如棋盘上的位置；也可以无限和连续，例如机器人的状态可能包含连续的位置、速度和关节角度. LLM 场景下状态空间近似包含所有可能的 token 组合: $\mathcal{S} = \bigcup_{t=0}^{L} \mathcal{V}^t$，其中 $\mathcal{V}$ 为 vocabulary，L 为最大上下文长度. (离散巨大)

Action. 动作 (**action**) 是 agent 在一定下状态下可以执行的行为，其依赖于状态，例如前进方向、速度. LLM 场景下 token-level action 是下一个 token；agentic LLM 中 action 也可能是调用某工具. Action 一般记为 a_t.

Action Space of State. 一定状态下所有可执行动作的集合，与状态 (state) 相关，一般记为 $\mathcal{A}(s_t) = \{a_i\}_{i=1}^{k}$.

State Transition. 某一状态采取行动后转变成另一状态，例如可以表示为 $s_1 \xrightarrow{a_2} s_2$. 其定义了 agent 与 environment 交互的前后 state 的变化，可以用表格 (tabular) 的形式展现，例如下面是一个确定性的 state transition:

State	a_1 (UPWARD)	a_2 (RIGHTWARD)	a_3 (DOWNWARD)	a_4 (LEFTWARD)	a_5 (STILL)
s_1	s_1	s_2	s_4	s_1	s_1
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
s_9	s_6	s_9	s_9	s_8	s_9

State Transition Probability. 由于环境是不确定的，因此采取某 action 后 state 的变化可能并不是确定性而是使用概率分布进行描述. 例如 $P(s_2|s_1, a_2) = 0.9$ 表示在 state s_1 时采取 action a_2 变成 state s_2 的概率为 90%.

Policy. 指导 agent 在某状态时应采取什么行动，通常记为 π. Policy 的本质是一个从状态到动作概率分布的映射: $\pi : \mathcal{S} \to \Delta(\mathcal{A})$，其中 $\Delta(\mathcal{A})$ 表示动作空间上的概率分布，满足 $\pi(a \mid s) \geqslant 0, \sum_{a \in \mathcal{A}(s)} \pi(a \mid s) = 1$. 例如在 s_1 时可以采用 3 种 action: $\pi(a_1|s_1) = 0, \pi(a_2|s_1) = 1, \pi(a_3|s_1) = 0$. 这是一个确定性 policy，满足 $\pi(a \mid s) = \begin{cases} 1, & a = \mu(s), \\ 0, & a \neq \mu(s). \end{cases}$

Policy 也可以使用表格 (tabular) 形式展现，例如下面包括了 deterministic policy 和 stochastic policy:

State	a_1 (UPWARD)	a_2 (RIGHTWARD)	a_3 (DOWNWARD)	a_4 (LEFTWARD)	a_5 (STILL)
s_1	0.2	0.1	0	0.5	0.2
⋮	⋮	⋮	⋮	⋮	⋮
s_9	**1**	0	0	0	0

Note 1.1. *LLM* 本身就是一个由 θ 参数化的随机 *policy:* $\pi_\theta(a_t \mid s_t) = \pi_\theta(y_{t+1} \mid x, y_{1:t})$，它由 *logits* 经 *softmax* 得到：$\pi_\theta(a \mid s) = \frac{\exp(z_\theta(s,a))}{\sum_{a'} \exp(z_\theta(s,a'))}$. 一整段回答的概率是 $\pi_\theta(y_{1:T} \mid x) = \prod_{t=0}^{T-1} \pi_\theta(y_{t+1} \mid x, y_{1:t})$.

Reward

Reward. 奖励 (**reward**) 是环境在某一步给出的标量反馈，一般记为 $r_t \in \mathbb{R}$. ① Reward 可以依赖于当前状态 s、动作 a、下一状态 s' 或它们的组合[a]. ② Reward 的相对大小反映了对模型当前 action / policy 的鼓励与否，一般可以简单认为 $r > 0$ 为鼓励，反之不鼓励. ③ Reward 是引导 agent 变得越来越好的重要凭据，其设计需要一定的专家知识，例如 LLM 的 RLHF 中由人类专家对 LLM 生成答案进行打分. ④ Reward 也可以使用表格形式展现，

State	a_1 (UPWARD)	a_2 (RIGHTWARD)	a_3 (DOWNWARD)	a_4 (LEFTWARD)	a_5 (STILL)
s_1	r_{boundary}	0	0	r_{boundary}	0
⋮	⋮	⋮	⋮	⋮	⋮
s_9	$r_{\text{forbidden}}$	r_{boundary}	r_{boundary}	0	r_{target}

Rewards in LLM

常见的 LLM reward 包括：① Reward model 打分 (RLHF). 由于 RL 训练数据量巨大，因此一般通过人类评价一部分回答，用这些数据训练 reward model，再由 reward model 来近似人类对回答质量的判断.

② 可验证奖励 RLVR: $R(x,y) = \begin{cases} 1, & \text{答案通过测试,} \\ 0, & \text{否则.} \end{cases}$. 例如数学答案、代码单元测试、形式化证明验证.

③ Agentic LM 工具使用奖励. 例如综合任务是否完成、调用成本、等待时间和错误操作进行打分.

[a]也有教程认为 reward 不依赖下一状态 s'，例如 *Mathematical Foundations of Reinforcement Learning* from Shiyu Zhao.

Trajectory, Episode and Return

Trajectory. 一条轨迹 (**trajectory**) 是 policy 与 environment 共同作用产生的随机样本，标准轨迹写为 $\tau = (S_0, A_0, R_1, S_1, A_1, R_2, \ldots)$，其是一条 state - action - reward chain，例如：$s_1 \xrightarrow[r=0]{a_2} s_2 \xrightarrow[r=0]{a_3} s_5 \xrightarrow[r=0]{a_3} s_8 \xrightarrow[r=1]{a_2} s_9$.

Note 1.2. ① τ 中大写字母表示随机变量，因为每个 *state, action* 和 *reward* 都具有随机性. 一条确定性的轨迹 τ 的概率为: $P_\pi(\tau) = \rho_0(s_0) \prod_t \pi(a_t \mid s_t) p(s_{t+1}, r_{t+1} \mid s_t, a_t)$. ② *LLM* 中给定 *prompt* x，一条 *token-level trajectory* 写为：

$$s_0 = (x), a_0 = y_1, s_1 = (x, y_1), a_1 = y_2, \ldots, a_{T-1} = EOS.$$

采样出的一条 *trajectory* 并不足以反映 *policy* 的好坏，因为同一个 *LLM* 重新采样可能得到完全不同的结果. 因此实际训练中需要对多个 *prompt (X)*、多个 *sampled response (Y)* 估计平均表现：$J(\pi) = \mathbb{E}_{X \sim \rho_0, Y \sim \pi(\cdot|X)}[G_0]$.

Episode/Trial. 当任务会终止时，该有限长度的从初始状态开始到 terminal state 结束的完整轨迹被称为 **episode/-trial**: $S_0, A_0, R_1, \ldots, S_T, S_T \in \mathcal{S}_{\text{end}}$. 存在 terminal states 的任务被称为 episodic task，否则为 continuing task.

Return. 回报 (**return**) 是从时间 t 开始，未来 reward 的累计量，因此又称为 total rewards 或 cumulative rewards，一般记为 G_t. (定义 G_t 为未来而非之前 reward 累积是因为之前轨迹已确定，RL 更关注未来决策.)

Discounted Return. 折扣回报 (**discounted return**) 定义为 $G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}, \gamma \in [0, 1)$. ① 其目的是避免无限轨迹的 return 无界，因为若 reward 有界，$|R_t| \leqslant R_{\max}$ 则 $|G_t| \leqslant \frac{R_{\max}}{1-\gamma}$. ② LLM 回答通常是 episode，故一般使用 $\gamma = 1$. 使用 $\gamma < 1$ 反而隐式偏好更短的回答，并且若最终 reward 为负，反而会鼓励 agent 拖延坏结果(更接近 0).

1.2 Markov Decision Process

Markov Decision Process (MDP)

马尔可夫决策过程 (Markov decision processe, MDP) 是描述序列决策问题的数学模型，强化学习实际就是在 agent 与环境的不断交互中优化决策过程，因此可以将其纳入 MDP 框架中. 有限状态、有限动作、带折扣因子的 MDP 通常写成：$\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, \mathcal{R}, \rho_0, \gamma)$，其中 $\gamma \in [0, 1]$ 为折扣因子，ρ_0 为初始状态分布. MDP 框架主要包括：

1. 集合部分: ① state space $\mathcal{S}$ ② action space $\mathcal{A}(s)$，与状态相关 ③ reward space $\mathcal{R}(s, a)$，与状态、动作相关.
2. 模型部分: ① State transition probability: 在 state s 采取 action a 后 state 变为 s' 的概率记为 $p(s'|s, a)$，其需满足 $\sum_{s' \in \mathcal{S}} p(s' \mid s, a) = 1, \forall (s, a)$ ② Reward probability：在 state s 采取 action a 后获得 reward r 的概率记为 $p(r|s, a)$，其需要满足 $\sum_{r \in \mathcal{R}(s,a)} p(r \mid s, a) = 1, \forall (s, a)$.

MDP 的一个重要性质是马尔可夫性 (**Markov property**)，是随机过程的一种无记忆属性 (memoryless property)：给定当前状态之后未来的变化不再依赖更早的历史. 这里指下一步的 state, reward 与前面的 state, action 无关：

$$p(s_{t+1} \mid s_t, a_t, s_{t-1}, a_{t-1}, \ldots, s_0, a_0) = p(s_{t+1} \mid s_t, a_t), \quad p(r_{t+1} \mid s_t, a_t, s_{t-1}, a_{t-1}, \ldots, s_0, a_0) = p(r_{t+1} \mid s_t, a_t) \tag{1.1}$$

Note 1.3. ① (s, a)，$p(s'|s, a)$ 和 $p(r|s, a)$ 都是 *model* 的一部分，其可以是时变/非稳定的 *(time-variant / nonstationary)* 也可以是非时变/稳定的 *(time-invariant/stationary).* 时变指环境可以随时间变化. ② 若 *MDP* 中的 *decision/policy* 确定那么其等同马尔可夫过程 *(Markov process).* 不过注意虽然此时具有概率性的 *model* 已确定，但状态序列仍然可能是随机的. 此时 *MDP* 被诱导为一个由策略决定的 *Markov reward process (MRP).*

1.3 Questions and Answers

Q & A

Q1: 可以将所有的 reward 设置为负吗?

A1: 可以. 事实上对一个 action 鼓励/ 不鼓励是由 reward 的相对值而非绝对值决定.

Q2: Reward 是下一 state s' 的函数吗?

A2: 实际上 reward 可以仅依赖当前状态 ($r(s)$)，例如只要当前在危险区域就获得 -1；也可以依赖当前状态和动作 ($r(s, a)$)，例如工具调用需要付出成本；也可以依赖完整转移 ($r(s, a, s')$)，例如只有实际进入目标状态才获得奖励. 一种会让人误解的情况是仅关注当前 (s, a) 下的 expected additive return 时，期望 reward 可以写为：

$$\bar{r}(s, a) = \mathbb{E}[R_{t+1} \mid S_t = s, A_t = a] = \sum_{s'} p(s' \mid s, a) \sum_r r \, p(r \mid s, a, s') = \sum_{s'} p(s' \mid s, a) \bar{r}(s, a, s'). \tag{1.2}$$

此时为了计算期望价值使用了一个经过边缘化的 reward 表示但并不意味原始环境中的 reward 与下一状态无关.

2 Lecture 2: Bellman Equation

2.1 State Value

Return Calculating

回报 (**return**) 定义为未来 reward 的累积量，可以度量 policy 的好坏. 由于 reward 等都可能带有随机性 (写为大写字母 R)，因此 return 本质也是一个随机变量，一般使用带折扣因子 γ 的 discounted return 防止累和发散:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots . \tag{2.1}$$

Return 有多种计算方式，此处介绍两种且不考虑随机性，即 trajectory 已确定：① 根据定义 (discounted) 累和. ② 自助法 (**bootstrapping**)，即借助对未来的估计来估计现在：由于每个 state 的 return 都依赖于下一个 return: $g_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \cdots = r_{t+1} + \gamma g_{t+1}$，因此可以将所有 state 的 return 串联成矩阵形式统一求解.

Example 2.1. 根据定义，图 *2.1* 中所有 *discounted return* 的计算方式如式 *(2.2)* 所示:

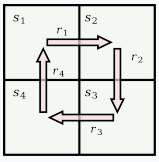


Figure 2.1: Illustrating example

$$\begin{aligned}
g_1 &= r_1 + \gamma r_2 + \gamma^2 r_3 + \cdots = r_1 + \gamma\left(r_2 + \gamma r_3 + \ldots\right) = r_1 + \gamma g_2,\\
g_2 &= r_2 + \gamma r_3 + \gamma^2 r_4 + \cdots = r_2 + \gamma\left(r_3 + \gamma r_4 + \ldots\right) = r_2 + \gamma g_3,\\
g_3 &= r_3 + \gamma r_4 + \gamma^2 r_1 + \cdots = r_3 + \gamma\left(r_4 + \gamma r_1 + \ldots\right) = r_3 + \gamma g_4,\\
g_4 &= r_4 + \gamma r_1 + \gamma^2 r_2 + \cdots = r_4 + \gamma\left(r_1 + \gamma r_2 + \ldots\right) = r_4 + \gamma g_1.
\end{aligned} \tag{2.2}$$

虽然四个变量相互关联构成循环看似不可解，但实际上写成如下线性矩阵-向量方程 *(linear matrix-vector equation)* 后只需求逆矩阵即可:

$$\underbrace{\begin{bmatrix} g_1 \\ g_2 \\ g_3 \\ g_4 \end{bmatrix}}_{g} = \begin{bmatrix} r_1 \\ r_2 \\ r_3 \\ r_4 \end{bmatrix} + \begin{bmatrix} \gamma g_2 \\ \gamma g_3 \\ \gamma g_4 \\ \gamma g_1 \end{bmatrix} = \underbrace{\begin{bmatrix} r_1 \\ r_2 \\ r_3 \\ r_4 \end{bmatrix}}_{r} + \gamma \underbrace{\begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \end{bmatrix}}_{P} \underbrace{\begin{bmatrix} g_1 \\ g_2 \\ g_3 \\ g_4 \end{bmatrix}}_{g} \quad \Rightarrow \quad g = r + \gamma P g \Rightarrow g = (I - \gamma P)^{-1} r. \tag{2.3}$$

State Value

由于 return G_t 是随机变量，因此一次 rollout / sampled trajectory 得到的 return 也只是一个样本. 因此为整体上比较 policy 的好坏，一个直接的想法就是在期望意义上进行比较.

Example 2.2. 图 *2.2* 的例子中，在初始点 s_1 此 *policy* 有 p_1 概率选择向右、有 p_2 概率向下 *($p_1 + p_2 = 1$)*，因此一个简单的想法就是求期望，如式 *(2.4)* 所示.

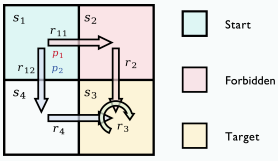


Figure 2.2: Illustrating example

$$\begin{aligned}
g_1 &= p_1\left(r_{11} + \gamma r_2 + \gamma^2 r_3 + \gamma^3 r_3 + \cdots\right) + p_2\left(r_{12} + \gamma r_4 + \gamma^2 r_3 + \gamma^3 r_3 + \cdots\right)\\
&\xlongequal{p_1+p_2=1} p_1(r_{11} + \gamma r_2) + p_2(r_{12} + \gamma r_4) + r_3(\gamma^2 + \gamma^3 + \cdots).
\end{aligned} \tag{2.4}$$

State Value. State value 用于评价 policy π 从 state s 出发持续执行的表现，衡量的依据是该 state 处持续执行 policy π 后 return 的期望值，其全称为 **state value function**:

$$v_\pi(s) \triangleq \mathbb{E}[G_t \mid S_t = s] \tag{2.5}$$

Note 2.1. ① *State value* 是 *state* s 的函数，并且依赖于 *policy* π. ② *State value* 并不能直接评价 *policy* 的全局表现，只能在固定 s 下进行比较. ③ 当无随机性即考虑确定性 *trajectory* 时，*return* 的累和 G_t 就与 *state value* 相同. ④ *State value* 的定义要求 *state* s 作为评价起点，不一定意味着真实任务必须从 s 启动.

Definition 2.1. 给定两 *policy* π_1, π_2，若对于 *state* s 有 $v_{\pi_1}(s) \leqslant v_{\pi_2}$ 则称 *policy* π_1 在 *state* s 处不劣于 π_1. 若对于任意 *state* s 都有 $v_{\pi_1}(s) \leqslant v_{\pi_2}$ 则称 *policy* π_2 全局优于 π_2.

2.2 Bellman Equation

Bellman Equation (Element-wise Form)

虽然每个 state 的 state value 中包含了未知的下一步 state value 而无法单独求解，但从 Example 2.1 中可以看出将所有 state 联系后可通过方程组求解. **Bellman Equation** (BE) 就是描述不同 state 的 state value 关系的工具.

BE 的核心思想是利用 return 的递归结构将当前状态价值拆解为 "下一步奖励+ $\gamma\times$ 下一状态价值"，从而将不同 state 联系起来. 对于一个 trajectory $S_t \xrightarrow{A_t} R_{t+1}, S_{t+1} \xrightarrow{A_{t+1}} R_{t+2}, S_{t+2}, \ldots$，discounted return 可以写为:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = R_{t+1} + \gamma(R_{t+2} + \gamma R_{t+3} + \cdots) = \underbrace{R_{t+1}}_{\text{immediate reward}} + \gamma \cdot \underbrace{G_{t+1}}_{\text{future reward}}. \tag{2.6}$$

代入 state value 定义式 (2.5) 后可以得到:

$$v_\pi(s) = \mathbb{E}[G_t|S_t = s] = \mathbb{E}[R_{t+1} + \gamma G_{t+1}|S_t = s] = \underbrace{\mathbb{E}[R_{t+1}|S_t = s]}_{①} + \underbrace{\gamma\mathbb{E}[G_{t+1}|S_t = s]}_{②}. \tag{2.7}$$

其中 ① 为 **mean of immediate reward**:

$$\mathbb{E}[R_{t+1} \mid S_t = s] = \sum_{a\in\mathcal{A}(s)} \pi(a \mid s)\mathbb{E}[R_{t+1} \mid S_t = s, A_t = a] = \sum_{a\in\mathcal{A}(s)} \pi(a \mid s) \sum_{r\in\mathcal{R}(s,a)} p(r \mid s, a) \cdot r. \tag{2.8}$$

② 为 **mean of future reward**:

$$\begin{aligned}\mathbb{E}[G_{t+1}|S_t = s] &\xrightarrow[\text{expectation}]{\text{law of total}} \sum_{s'\in\mathcal{S}} \mathbb{E}[G_{t+1}|S_t = s, S_{t+1} = s']p_\pi(s'|s) \xrightarrow[\text{process property}]{\text{Markov decision}} \sum_{s'\in\mathcal{S}} \mathbb{E}[G_{t+1}|S_{t+1} = s']p_\pi(s'|s) \\ &= \sum_{s'\in\mathcal{S}} v_\pi(s')p_\pi(s'|s) \xrightarrow[\text{expectation}]{\text{law of total}} \sum_{s'\in\mathcal{S}} v_\pi(s') \sum_{a\in\mathcal{A}(s)} p(s'|s, a)\pi(a|s).\end{aligned} \tag{2.9}$$

将式 (2.8) (2.9) 代回式 (2.7) 中可以得到 element-wise form **Bellman Equation**:

$$\begin{aligned}v_\pi(s) &= \mathbb{E}[R_{t+1}|S_t = s] + \gamma\mathbb{E}[G_{t+1}|S_t = s] = \underbrace{\sum_{a\in\mathcal{A}(s)} \pi(a|s) \sum_{r\in\mathcal{R}(s,a)} p(r|s, a)r}_{\text{mean of immediate rewards}} + \underbrace{\gamma \sum_{a\in\mathcal{A}(s)} \pi(a|s) \sum_{s'\in\mathcal{S}} p(s'|s, a)v_\pi(s')}_{\text{mean of future rewards}} \\ &= \sum_{a\in\mathcal{A}(s)} \pi(a|s) \left[\sum_{r\in\mathcal{R}(s,a)} p(r|s, a)r + \gamma \sum_{s'\in\mathcal{S}} p(s'|s, a)v_\pi(s') \right], \quad \forall s \in \mathcal{S}.\end{aligned} \tag{2.10}$$

Note 2.2. ① *Bellman Equation (2.10)* 实际是将 *state* s 的 *state value* 用所有 *state* 进行表示，因此描述了不同 *state* 的 *state value function* 间的联系. ② 虽然式 *(2.10)* 表面只有一个表达式，但通过遍历 $s \in \mathcal{S}$ 可以得到一族共 $|\mathcal{S}|$ 个方程. ③ 求解 *Bellman Equation* 的过程又被称为 *policy evalution*，因为实际就是计算所有 *state value*、对 *policy* 进行评估.

根据全概率公式 (law of total probability)，Bellman Equation (2.10) 也可以写为:

$$v_\pi(s) \xrightarrow[p(s'|s,a)=\sum_{r\in\mathcal{R}} p(s',r|s,a)]{p(r|s,a)=\sum_{s'\in\mathcal{S}} p(s',r|s,a)} \sum_{a\in\mathcal{A}(s)} \pi(a|s) \sum_{s'\in\mathcal{S}} \sum_{r\in\mathcal{R}} p(s', r|s, a)[r + \gamma v_\pi(s')]. \tag{2.11}$$

如果在某些问题中 reward 只与 s' 相关，那么上述 Bellman Equation 又可以进一步写为

$$v_\pi(s) = \sum_{a\in\mathcal{A}} \pi(a|s) \sum_{s'\in\mathcal{S}} p(s'|s, a) \left[r(s') + \gamma v_\pi(s')\right]. \tag{2.12}$$

Bellman Equation (Matrix-vector Form)

Bellman Equation (2.10) 实际包括 $|\mathcal{S}|$ 个方程，将它们组合起来就可以显式得到 matrix-vecotr form 的 BE.
首先为书写方便，对式 (2.10) 进行如下改写：

$$v_\pi(s) = \underbrace{\sum_{a\in\mathcal{A}(s)} \pi(a|s) \sum_{r\in\mathcal{R}(s,a)} p(r|s,a)r}_{\text{mean of immediate rewards}} + \underbrace{\gamma \sum_{a\in\mathcal{A}(s)} \pi(a|s) \sum_{s'\in\mathcal{S}} p(s'|s,a)v_\pi(s')}_{\text{mean of future rewards}} = r_\pi(s) + \gamma \sum_{s'} p_\pi(s'|s)v_\pi(s'), \quad (2.13)$$

其中 $r_\pi(s)$ 为 mean of immediate rewards，$p_\pi(s'|s)$ 为在 policy π 下 state 由 s 转变为 s' 的概率 (式 (2.9))：

$$r_\pi(s) \triangleq \sum_a \pi(a|s) \sum_r p(r|s,a)r, \quad p_\pi(s'|s) \triangleq \sum_a \pi(a|s)p(s'|s,a). \quad (2.14)$$

从而对于 n 个 state 的 state space $\mathcal{S} = \{s_1, s_2, \cdots, s_n\}$，$n$ 个 Bellman Equation 可写为：

$$v_\pi(s_i) = r_\pi(s_i) + \gamma \sum_{s_j} p_\pi(s_j \mid s_i)v_\pi(s_j), \quad i = 1, 2, \ldots, n \quad \Rightarrow \quad \boldsymbol{v}_\pi = \boldsymbol{r}_\pi + \gamma \boldsymbol{P}_\pi \boldsymbol{v}_\pi, \quad (2.15)$$

其中 $\boldsymbol{v}_\pi = [v_\pi(s_1), \ldots, v_\pi(s_n)]^T \in \mathbb{R}^n$, $\boldsymbol{r}_\pi = [r_\pi(s_1), \ldots, r_\pi(s_n)]^T \in \mathbb{R}^n$, $\boldsymbol{P}_\pi \in \mathbb{R}^{n\times n}$，$[\boldsymbol{P}_\pi]_{ij} = p_\pi(s_j|s_i)$ 为状态转移矩阵 (state transition matrix)，满足 ① $0 \leqslant [\boldsymbol{P}_\pi]_{ij} = p_\pi(s_j|s_i) \leqslant 1$ ② $\boldsymbol{P}_\pi \, \mathbf{1} = \mathbf{1}$，其中 $\mathbf{1} = [1, 1, \cdots, 1]^T$.
将式 (2.15) 写成矩阵形式可以得到：

$$\underbrace{\begin{bmatrix} v_\pi(s_1) \\ v_\pi(s_2) \\ \vdots \\ v_\pi(s_n) \end{bmatrix}}_{\boldsymbol{v}_\pi} = \underbrace{\begin{bmatrix} r_\pi(s_1) \\ r_\pi(s_2) \\ \vdots \\ r_\pi(s_n) \end{bmatrix}}_{\boldsymbol{r}_\pi} + \gamma \underbrace{\begin{bmatrix} p_\pi(s_1|s_1) & \cdots & p_\pi(s_{n-1}|s_1) & p_\pi(s_n|s_1) \\ p_\pi(s_1|s_2) & \cdots & p_\pi(s_{n-1}|s_2) & p_\pi(s_n|s_2) \\ \vdots & \ddots & \vdots & \vdots \\ p_\pi(s_1|s_n) & \cdots & p_\pi(s_{n-1}|s_4) & p_\pi(s_n|s_n) \end{bmatrix}}_{\boldsymbol{P}_\pi} \underbrace{\begin{bmatrix} v_\pi(s_1) \\ v_\pi(s_2) \\ \vdots \\ v_\pi(s_n) \end{bmatrix}}_{\boldsymbol{v}_\pi}. \quad (2.16)$$

Solve the Bellman Equation

① 求解 Bellman Equation (2.15) (2.16) 最直接的方式就是直接求其闭式解 (closed-form solution):

$$\boldsymbol{v}_\pi = (\boldsymbol{I} - \gamma \boldsymbol{P}_\pi)^{-1} \boldsymbol{r}_\pi. \quad (2.17)$$

矩阵 $\boldsymbol{I} - \gamma \boldsymbol{P}_\pi$ 具有有如下性质：(1) 可逆[a]，因此保证了解的存在性. (2) $(\boldsymbol{I} - \gamma \boldsymbol{P}_\pi)^{-1} \geqslant_{\text{elem}} \boldsymbol{I}$. 因为 $(I - \gamma P_\pi)^{-1} = I + \gamma P_\pi + \gamma^2 P_\pi^2 + \cdots \geqslant_{\text{elem}} I$. (3) $(\boldsymbol{I} - \gamma \boldsymbol{P}_\pi)^{-1} \boldsymbol{r} \geqslant \boldsymbol{r} \geqslant 0, \forall \boldsymbol{r} \geqslant 0$. (直接利用性质 2 即可).

Note 2.3. 性质 *(2)* ① 保证 *state value* $\boldsymbol{v}_\pi$ 至少和 *reward* $\boldsymbol{r}_\pi$ 具有相同的趋势. 即如果 *reward* $\boldsymbol{r}_\pi$ 非负那么 *state value* $\boldsymbol{v}_\pi$ 也必然非负. ② 可以用于构造收敛性证明. 如值迭代 *(value iteration)* 或策略迭代 *(policy iteration)* 算法中常用 $\boldsymbol{v}_k \leqslant \boldsymbol{v}_{k+1}$ 来证明值函数单调收敛于最优 *state value*. 性质 *(3)* 进一步加强了 *state value function* 的单调性. 即如果 *reward* $\boldsymbol{r}_\pi$ 非负，那么其影响不会被折扣因子 γ 和转移矩阵 $\boldsymbol{P}_\pi$ 降到 *0*，而是至少保持原有的奖励大小.

Note 2.4. 虽然解析解形式直接，但数值上大规模矩阵求逆很困难，需要使用特殊的数值方法 *(*且仍然难以求解*)*.

② 求解 Bellman Equation (2.15) (2.16) 的另一种方式是通过迭代法找其迭代解 (iterative solution):

$$\boldsymbol{v}_{k+1} = \boldsymbol{r}_\pi + \gamma \boldsymbol{P}_\pi \boldsymbol{v}_k, \quad k = 0, 1, 2, \ldots. \quad (2.18)$$

可以证明[b] 上述迭代过程一定收敛(证明过程详见附录A):

$$\boldsymbol{v}_k \xrightarrow{k\to\infty} \boldsymbol{v}_\pi = (\boldsymbol{I} - \gamma \boldsymbol{P}_\pi)^{-1} \boldsymbol{r}_\pi. \quad (2.19)$$

[a] 证明需要使用 Gershgorin circle theorem，详见附录A
[b] 如果熟悉的话这其实就是一个不动点迭代的证明，使用 Banach 不动点定理

2.3 Action Value

Action Value

Action value 也是强化学习中的一个重要概念，其定义需要和 state value 相联系. Action value 是指在某 state 下采取相应的某 action 后能够获得的 average return，其依赖 state, action 和 policy：

$$q_\pi(s,a) \triangleq \mathbb{E}[G_t|S_t=s,A_t=a]. \tag{2.20}$$

State value 与 action value 的区别在于 state value 是指 agent 从一个 state 出发能得到的 average return，而 action value 是指 agent 从一个 state 出发并且做出一个 action 后能得到的 average return:

$$v_\pi(s)=\mathbb{E}[G_t \mid S_t=s], \quad q_\pi(s,a) \triangleq \mathbb{E}[G_t|S_t=s,A_t=a]. \tag{2.21}$$

实际上 action value 与 state value 像一个硬币的两面，可以相互推导：

$$\underbrace{\mathbb{E}[G_t|S_t=s]}_{v_\pi(s)}=\sum_a \underbrace{\mathbb{E}[G_t|S_t=s,A_t=a]}_{q_\pi(s,a)}\pi(a|s)=\sum_a \pi(a|s)\underbrace{\left[\sum_r p(r|s,a)r+\gamma\sum_{s'}p(s'|s,a)v_\pi(s')\right]}_{q_\pi(s,a)} \tag{2.22a}$$

$$\Rightarrow v_\pi(s)=\sum_a \pi(a|s)q_\pi(s,a).$$

$$q_\pi(s,a)=\sum_r p(r|s,a)r+\gamma\sum_{s'}p(s'|s,a)v_\pi(s'). \tag{2.22b}$$

Examples in LLM

在 token 级建模中，可以把语言生成写成 Markov Decision Process (MDP): ① State $s_t=(x,y_{<t})$: prompt + 已经生成的 token ② Action $a_t=y_t$: 下一个 token ③ Policy $\pi_\theta(a_t \mid s_t)$: 语言模型的 next-token distribution ④ Reward: reward model 分数、规则奖励、工具执行结果或人工反馈. 于是

$$V^\pi(x,y_{<t})=\mathbb{E}_\pi[\text{从此前缀继续生成的最终奖励}], \quad Q^\pi(x,y_{<t},y_t)=\mathbb{E}_\pi[\text{选择该token 后的最终奖励}].$$

3 Lecture 3: Bellman Optimality Equation, Value Iteration and Policy Iteration

3.1 Optimal Policy and Optimal State Value

Optimal Policy and Optimal State Value

State s 处的 state value 定义为该 state 处未来 (discounted) return 的期望值: $v_\pi(s) = \mathbb{E}[G_t \mid S_t = s]$. 对于不同的 policy 可以通过比较它们在某个 state 或全局的 state value 来比较它们的好坏.

Definition 3.1 (Better Policy)**.** 对于两 *policy* π_1, π_2，若他们的 *state value* 有如下全局关系则称 *policy* π_1 好于 π_2:

$$v_{\pi_1}(s) \geqslant v_{\pi_2}(s), \quad \forall s \in \mathcal{S}. \tag{3.1}$$

Definition 3.2 (Optimal Policy & Optimal State Value)**.** 若 *policy* π^* 的 *state value* 有:

$$v_{\pi^*}(s) \geqslant v_\pi(s), \quad \forall s \in \mathcal{S}, \forall \pi, \tag{3.2}$$

则称 *policy* π^* 为 *optimal policy.* 称 π^* 的 *state values* 为 *optimal state values*，并定义 $v^*(s) \triangleq \sup_\pi v_\pi(s)$. 在有限状态、有限动作且 $0 \leqslant \gamma < 1$ 的 *MDP* 中，存在一个确定性最优策略 π^* 使得 $v_{\pi^*}(s) = v^*(s), \forall s \in \mathcal{S}$.

3.2 Bellman Optimality Equation

Bellman Optimality Equation

定义了 optimal policy 和 optimal state value 后自然而然引出了下列问题:

1. **Existence & Uniqueness**: optimal policy 是否存在? 若存在是否唯一?
2. **Stochasticity**: optimal policy 是确定性的 (deterministic) 还是随机性的 (stochastic)?
3. **Algorithm**: 如何得到 optimal policy 和 optimal state values?

分析 optimal policy 和 optimal state value 的重要工具是 **Bellman Optimality Equation (BOE)**. 与之前 Bellman Equation (BE) 思路相似，通过求解 BOE 就可以得到 optimal policy 和 optimal state values. BE 形式如下:

$$v_\pi(s) = \underbrace{\sum_{a\in\mathcal{A}} \pi(a|s) \sum_{r\in\mathcal{R}} p(r|s,a)r}_{\text{mean of immediate rewards}} + \gamma \underbrace{\sum_{a\in\mathcal{A}} \pi(a|s) \sum_{s'\in\mathcal{S}} p(s'|s,a)v_\pi(s')}_{\text{mean of future rewards}} = \sum_{a\in\mathcal{A}} \pi(a|s) \left[\sum_{r\in\mathcal{R}} p(r|s,a)r + \gamma \sum_{s'\in\mathcal{S}} p(s'|s,a)v_\pi(s') \right], \quad \forall s \in \mathcal{S}. \tag{3.3}$$

Bellman Optimality Equation 不再像 Bellman Equation (3.3) 一样对给定 policy 的动作分布取期望，而是在每个状态上调整 policy 使得 one-step look-ahead value (即当前 state value 展开一步 $G_t = R_t + \gamma G_{t+1}$) 最大:

$$v^*(s) = \max_\pi \sum_{a\in\mathcal{A}} \pi(a|s) \left(\sum_{r\in\mathcal{R}} p(r|s,a)r + \gamma \sum_{s'\in\mathcal{S}} p(s'|s,a)v^*(s') \right) \triangleq \max_\pi \sum_{a\in\mathcal{A}} \pi(a|s)q(s,a), \quad \forall s \in \mathcal{S}. \tag{3.4}$$

式 (3.4) 是 BOE 的 elementwise form，与 BE 的 $\boldsymbol{v} = \boldsymbol{r}_\pi + \gamma \boldsymbol{P}_\pi \boldsymbol{v}$ 形式相似，其 matrix-vector form 如下:

$$\boldsymbol{v}^* = \max_\pi (\boldsymbol{r}_\pi + \gamma \boldsymbol{P}_\pi \boldsymbol{v}^*), \quad [\boldsymbol{r}_\pi]_s \triangleq \sum_{a\in\mathcal{A}} \pi(a|s) \sum_{s',r} p(s',r|s,a)r, \quad [\boldsymbol{P}_\pi]_{s,s'} \triangleq p(s'|s) \triangleq \sum_{a\in\mathcal{A}} \pi(a|s)p(s'|s,a). \tag{3.5}$$

Note 3.1. 当 *reward* 与下一 *state* s' 条件独立时有 $[\boldsymbol{r}_\pi]_s = \sum_a \pi(a|s) \sum_r p(r|s,a)r$.

根据定义 optimal policy 对所有 state 都有最优性，因此 max 为逐状态取最大值，矩阵形式中可以写进矩阵中:

$$\max_\pi \underbrace{\begin{bmatrix} v_\pi(s_1) \\ v_\pi(s_2) \\ \vdots \\ v_\pi(s_n) \end{bmatrix}}_{\max_\pi \boldsymbol{v}_\pi} = \begin{bmatrix} \max_\pi v_\pi(s_1) \\ \max_\pi v_\pi(s_2) \\ \vdots \\ \max_\pi v_\pi(s_n) \end{bmatrix}. \tag{3.6}$$

Note 3.2. *BOE (3.4)* 中 $p(r|s,a), p(s'|s,a)$ 作为模型部分均已知，$v(s)$, $v(s')$ 为需求解的未知量.

3.3 Solving an Optimal Policy from the BOE

Bellman Optimality Operator

在得到 BOE (3.4) (3.5) 后自然也产生了如下问题：① **Existence & Uniqueness**: BOE 是否有解？若有解是否唯一？ ② **Algorithm**: 如何求解 BOE？ ③ **Optimality**: BOE 的解与 optimal policy 有何关系？

上述问题的一种解决角度是将 Bellman Equation (3.3) 和 Bellman Optimality Equation (3.4) (3.5) 都通过算子的角度去理解. 定义一步期望奖励为 $\bar{r}(s,a) = \mathbb{E}[R_{t+1} \mid S_t = s, A_t = a] = \sum_{s',r} p(s', r \mid s, a)r$，那么式 (3.3) 可以写为：

$$v_\pi(s) = \sum_a \pi(a \mid s) \left[\bar{r}(s,a) + \gamma \sum_{s'} P(s' \mid s, a) v_\pi(s') \right] \tag{3.7}$$

也可以写成如下 Bellman Operator 形式：

$$\boldsymbol{v}_\pi = T^\pi \boldsymbol{v}_\pi, \quad (T^\pi v)(s) = \sum_a \pi(a \mid s) \left[\bar{r}(s,a) + \gamma \sum_{s'} P(s' \mid s, a) v(s') \right]. \tag{3.8}$$

Definition 3.3 (Bellman Optimality Operator)**.** *Bellman optimality operator* $T^* : \mathbb{R}^{|\mathcal{S}|} \longrightarrow \mathbb{R}^{|\mathcal{S}|}$ 定义为*:*

$$(T^* \boldsymbol{v})(s) \triangleq \max_{a \in \mathcal{A}(s)} \left[\bar{r}(s,a) + \gamma \sum_{s' \in \mathcal{S}} P(s' \mid s, a) v(s') \right] = \max_{a \in \mathcal{A}(s)} q_{\boldsymbol{v}}(s,a). \tag{3.9}$$

其中 $\bar{r}(s,a) \triangleq \mathbb{E}[R_{t+1} \mid S_t = s, A_t = a]$. 从而 Bellman optimality equation (BOE) 可以写为如下不动点方程：

$$\boldsymbol{v}^* = T^* \boldsymbol{v}^*, \quad v^*(s) = \max_{a \in \mathcal{A}(s)} \left[r(s,a) + \gamma \sum_{s' \in \mathcal{S}} P(s' \mid s, a) v^*(s') \right], \forall s. \tag{3.10}$$

Existence and Uniqueness of Solution in BOE

对于不动点方程 (3.10)，根据压缩映射原理 (Contraction Mapping Theorem) / 不动点定理 (Fixed Point Theorem)，如果 T^* 是压缩映射 (见附录 B)，即存在 $c \in [0,1)$ 使得

$$\|T^* x - T^* y\| \leqslant c\|x - y\|, \quad \forall x, y, \tag{3.11}$$

那么 T^* 存在唯一不动点 $x^* = T^* x^*$，并且从任意初值 x_0 出发，迭代 $x_{k+1} = T^* x_k$ 都收敛到 x^*.

Theorem 3.1 (Contraction property of $f(v)$)**.** *BOE* 中的映射 T^* 是一个压缩映射，$\forall\, \boldsymbol{v}, \boldsymbol{u} \in \mathbb{R}^{|\mathcal{S}|}$，满足

$$\|T^* \boldsymbol{u} - T^* \boldsymbol{v}\|_\infty \leqslant \gamma \|\boldsymbol{u} - \boldsymbol{v}\|_\infty, \quad \|\boldsymbol{v}\|_\infty \triangleq \max_{s \in \mathcal{S}} |v(s)|, \tag{3.12}$$

其中 $\gamma \in [0,1)$ 为 *discount factor*，$\|\cdot\|_\infty$ 为 *maximum norm (*取绝对值最大者*).*

由 Banach 压缩映射定理，可以立即得到 BOE 解的存在性、唯一性以及相应的迭代算法：

Theorem 3.2 (Existence, Uniqueness, and Algorithm)**.** 对于 *BOE* $\boldsymbol{v}^* = T^* \boldsymbol{v}^* = \max_\pi (r_\pi + \gamma P_\pi v)$，总存在唯一解，且该解可以被如下方式迭代逼近：

$$\boldsymbol{v}_{k+1} = T^* \boldsymbol{v}_k = \max_\pi (\boldsymbol{r}_\pi + \gamma P_\pi \boldsymbol{v}_k), \tag{3.13}$$

且产生的序列 $\{\boldsymbol{v}_k\}$ 指数收敛至不动点 $\boldsymbol{v}^*$，收敛速率由 γ 控制. 进一步地，其误差满足

$$\|\boldsymbol{v}_k - \boldsymbol{v}^*\|_\infty \leqslant \gamma^k \|\boldsymbol{v}_0 - \boldsymbol{v}^*\|_\infty. \tag{3.14}$$

Note 3.3. ① ***Uniqueness of optimal policy****: Optimal state value* v^* 是唯一确定的，但 *optimal policy* 并不一定，有可能出现两个 *policy* 均为最优. ② ***Stochasticity of optimal policy****:* 从 *optimal policy* 并不一定唯一就可以看出 *optimal policy* 既可以是随机的也可以是确定性的，但根据定理*3.4* 可以确定的是一定存在一个确定性的 *optimal policy.*

Greedy Policy

Theorem 1 中虽然给出了迭代算法 (式 (3.13)) 并且给出了收敛保证，但 $\max_\pi$ 尚未给出具体的选择方式.
假设环境模型已知，即奖励概率 $p(r \mid s,a)$、状态转移概率 $p(s' \mid s,a)$、折扣因子 γ 均已知，为了数值求解，我们先任意初始化一个 policy value funtion $\boldsymbol{v}_0$ (例如 $v_0(s)=0, \forall s$). 此时 action value q 也会得到初始化：

$$q_0(s,a) = \sum_r p(r|s,a)r + \gamma \sum_{s'} p(s'|s,a)v_0(s'). \tag{3.15}$$

由于 $v_\pi(s) = \sum_a \pi(a|s) q_\pi(s,a)$，在 $q_\pi(s,a)$ 均已知的情况求解此凸组合的最大值显然有：

$$v_0(s) = \sum_{a\in\mathcal{A}} \pi(a|s)\cdot q_0(s,a) \leqslant \underbrace{\left(\sum_{a\in\mathcal{A}} \pi(a \mid s)\right)}_{=1} \cdot q(s,a^*) = q(s,a^*), \quad q(s,a^*) = \max_{a\in\mathcal{A}(s)} q(s,a). \tag{3.16}$$

因此在当前的环境模型 (action value 分布) 下为最大化 state value，只需要只取最大的 action value $q(s,a)$ 即可：

$$\max_\pi v = \max_\pi \sum_{a\in\mathcal{A}} \pi(a|s) q(s,a) = \max_{a\in\mathcal{A}(s)} q(s,a). \tag{3.17}$$

此时的 policy π 只选择了最大的 action value 值，因此被称为 greedy policy:

$$\pi(a \mid s) = \begin{cases} 1, & a = a^* \\ 0, & a \neq a^* \end{cases}, \quad a^* = \arg\max_a q(s,a). \tag{3.18}$$

Note 3.4. *Greedy policy* 告诉我们：给定 v 后最好的 *one-step policy* 是关于 v 的 *greedy policy*，即只选择最大化 *action value* 的 *action*. 然而 *greedy policy* 并不能表明当前 *policy* 是全局最优的，因为当前 v 可能是一个错误估计. 但 *Theorem 1* 给出的迭代保证说明反复应用 T^*、按照 *greedy policy* 不断迭代最终必然收敛到唯一的最优解.

因此迭代公式 (3.13) 可写为：

$$v_{k+1}(s) = \max_{a\in\mathcal{A}(s)} \left[r(s,a) + \gamma \sum_{s'\in\mathcal{S}} P(s' \mid s,a) v_k(s') \right] = \max_{a\in\mathcal{A}(s)} q(s,a). \tag{3.19}$$

3.4 Optimality Analysis

Optimality Analysis

BOE 实际是固定了最优 policy 下的特殊 BE. Bellman Optimal Equation (3.4) (3.5) 求解的是 optimal state value $\boldsymbol{v}^*$，因此此时的 policy π^* 满足 Definition 3.2 中 optimal policy 的定义，并且是一个固定的 policy，其满足：

$$\pi^*(\cdot \mid s) \in \arg\max_{\pi(\cdot|s)} \sum_a \pi(a \mid s) \left[\bar{r}(s,a) + \gamma \sum_{s'} p(s' \mid s,a) v^*(s') \right]. \tag{3.20}$$

此时 Bellman Optimal Equation 在固定 policy π^* 下就可以转化为一个特殊的 Bellman Equation:

$$\boldsymbol{v}^* = \boldsymbol{r}_{\pi^*} + \gamma \boldsymbol{P}_{\pi^*} \boldsymbol{v}^* \tag{3.21}$$

关于 Bellman Optimal Equation 解的最优性，有如下结论 (证明见附录 B).

Theorem 3.3. 通过 *Bellman Optimal Equation* 求解得到的 v^* 是 *optimal state value*，π^* 是 *optimal policy*，即

$$v^* = v_{\pi^*} \geqslant v_\pi, \forall \pi. \tag{3.22}$$

Theorem 3.4 (Greedy Optimal Policy)**.** $\forall s \in \mathcal{S}$，*BOE* 的最优 *policy* (也称为 *deterministic greedy policy*)为：

$$\pi^*(a \mid s) = \begin{cases} 1 & \text{if } a = a^*(s) \\ 0 & \text{if } a \neq a^*(s) \end{cases} \tag{3.23a}$$

$$a^*(s) \triangleq \arg\max_a q^*(s,a), \quad q^*(s,a) \triangleq \sum_{r\in\mathcal{R}} p(r \mid s,a)r + \gamma \sum_{s'\in\mathcal{S}} p(s' \mid s,a)v^*(s'). \tag{3.23b}$$

Analysis for Bellman Optimal Equation

对于 Bellman Optimal Equation:

$$v(s) = \max_\pi \sum_{a\in\mathcal{A}} \pi(a|s) \left(\sum_{r\in\mathcal{R}} p(r|s,a)r + \gamma \sum_{s'\in\mathcal{S}} p(s'|s,a)v(s') \right) \tag{3.24}$$

黑色部分是未知并需求解的量，红色部分为已知量，可能对最终结果造成影响，其中

1. r: 预先设计的 reward.
2. $p(r|s,a), p(s'|s,a)$: 系统模型. 其中 $p(s' \mid s,a)$ 描述环境如何转移，$p(r \mid s,a)$ 描述环境给出多少即时奖励.
3. γ: discount factor.

由于系统的模型由外部决定，一旦确定后 agent 不能直接修改，因此主要分析 r 和 γ 的改变对 BOE 结果的影响.

Note 3.5. 策略 π 决定 *agent* 想做什么；系统模型 P 决定它做了以后世界实际如何演化. 例如机器人执行“向前走” *(forward)*，可能 $P(\text{向前移动} \mid s, \textit{forward}) = 0.9, P(\text{停留原地} \mid s, \textit{forward}) = 0.1$.

Discount factor determines long-sighted / short-sighted. Discounted return 的定义为

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots .$$

因此第 k 步之后得到的 reward R，在当前时刻只值 $\gamma^k R$，因此 γ 越小延迟奖励衰减得越快. 因此 ① 较大的 γ 会使 agent “远视” (long-sighted)，更加重视未来的 reward. ② 相反地，较小的 γ 会使 agent “短视” (short-sighted).

Meaningless Detour. Discount factor 的存在使得一些无意义的绕远路 (meaningless detour) 策略被抑制，因为这样的 reward 会被延后且“打折”. 例如 T 步后到达终点有 return $G_{\text{direct}} = \gamma^T R$，但绕路后为 $G_{\text{detour}} = \gamma^{T+d} R$.

Degeneration. 将 BOE 写成如下形式:

$$v^*(s) = \max_a q^*(s,a), q^*(s,a) = \bar{r}(s,a) + \gamma \sum_{s'} P(s' \mid s,a)v^*(s')$$

当 $\gamma = 0$ 时 BOE 退化成 $v^*(s) = \max_a \bar{r}(s,a)$. 此时未来状态的价值完全消失，因此 agent 只能最大化一步即时奖励. 这时若到达目标需要先经历若干步“即时收益低”甚至“即时惩罚”的动作 agent 就可能永远达不到目标.

Theorem 3.5 (Optimal Policy Invariance)**.** 考虑一马尔可夫决策过程 *(MDP)*，其中$\boldsymbol{v}^* \in \mathbb{R}^{|S|}$ 是满足

$$\boldsymbol{v}^* = \max_{\pi\in\Pi} (\boldsymbol{r}_\pi + \gamma \boldsymbol{P}_\pi \boldsymbol{v}^*)$$

的 *optimal state value*. 如果对 *reward* $\boldsymbol{r} \in \mathbb{R}$ 进行仿射变换 $\alpha\boldsymbol{r} + \beta$，其中 $\alpha, \beta \in \mathbb{R}$ 且 $\alpha > 0$，则相应的 *optimal state value* $\boldsymbol{v}'$ 也将是 $\boldsymbol{v}^*$ 的仿射变换:

$$\boldsymbol{v}' = \alpha \boldsymbol{v}^* + \frac{\beta}{1-\gamma}\mathbf{1},$$

其中 *discount factor* $\gamma \in (0,1)$，$\mathbf{1} = [1,\ldots,1]^T$. 因此从 $\boldsymbol{v}'$ 导出的 *optimal state value* 对于 *reward* 的仿射变换保持不变.

3.5 Value Iteration

Value Iteration

Theorem 1 给出的迭代算法式 (3.13) 实际被称为 Value Iteration，因为其通过不断迭代 state value $\boldsymbol{v}_k$ 逼近 $\boldsymbol{v}^*$:

$$\boldsymbol{v}_{k+1} = T^*\boldsymbol{v}_k = \max_{\pi}(\boldsymbol{r}_\pi + \gamma P_\pi \boldsymbol{v}_k). \tag{3.25}$$

实际上上述算法的实现过程可以拆解两步：① Policy Update (PU) ② Value Update (VU).

① **Policy Update (PU).** 在给定 $\boldsymbol{v}_k, \boldsymbol{r}_k, P_\pi$ 前提下首先计算式 (3.25) 右边部分，即估计新的 π_{k+1}:

$$\pi_{k+1} = \arg\max_{\pi}(\boldsymbol{r}_\pi + \gamma P_\pi \boldsymbol{v}_k), \quad \pi_{k+1}(s) = \arg\max_{\pi} \sum_a \pi(a|s) \underbrace{\left(\sum_r p(r|s,a)r + \gamma \sum_{s'} p(s'|s,a)v_k(s') \right)}_{q_k(s,a)}, s \in \mathcal{S}. \tag{3.26}$$

根据 greedy policy (3.18)，此时的 π_{k+1} 选择最大的 action value:

$$\pi_{k+1}(a \mid s) = \begin{cases} 1, & a = a_k^*(s) \\ 0, & a \neq a_k^*(s) \end{cases}, \quad a_k^*(s) = \arg\max_a q_k(s,a). \tag{3.27}$$

② **Value Update (VU).** 在得到 π_{k+1} 后再对 state value 进行更新:

$$\boldsymbol{v}_{k+1} = \boldsymbol{r}_{\pi_{k+1}} + \gamma P_{\pi_{k+1}} \boldsymbol{v}_k, \quad v_{k+1}(s) = \sum_a \pi_{k+1}(a|s) \underbrace{\left(\sum_r p(r|s,a)r + \gamma \sum_{s'} p(s'|s,a)v_k(s') \right)}_{q_k(s,a)} = \max_a q_k(s,a), s \in \mathcal{S}. \tag{3.28}$$

Note 3.6. ① v_k 是 *optimal state value* v^* 的中间估计，不一定是任何策略的精确状态价值. ② q_k 是基于 v_k 的一步前瞻估计，不一定是某个 *policy* 的精确 *action value.*

上述过程可以总结为：① 得到 state value 中间估计 $v_k(s) \to$ ② 计算 action value 前瞻估计 $q_k(s,a) = \sum_r p(r|s,a)r + \gamma \sum_{s'} p(s'|s,a)v_k(s') \to$ ③ greedy policy $a_k^*(s) = \arg\max_a q_k(s,a) \to$ ④ value update $v_{k+1}(s) = \max_a q_k(s,a)$.

$$v_k(s) \to q_k(s,a) \to \text{greedy policy } \pi_{k+1}(a \mid s) \to \text{new value } v_{k+1} = \max_a q_k(s,a)$$

Algorithm 1 Value Iteration Algorithm

Require: The probability model $p(r \mid s,a)$ and $p(s' \mid s,a)$ for all (s,a) are known. Initial guess v_0, tolerance $\theta > 0$.
Ensure: Approximate optimal value function v^* and a greedy policy π^*.

while $\|v_k - v_{k-1}\| \geqslant \theta$ **do**
  for each state $s \in \mathcal{S}$ **do**
    for each action $a \in \mathcal{A}(s)$ **do**
      $q_k(s,a) \leftarrow \sum_r p(r \mid s,a)r + \gamma \sum_{s'} p(s' \mid s,a)v_k(s')$ ▷ q-value
    end for
    $a_k^*(s) \leftarrow \arg\max_a q_k(s,a)$ ▷ Maximum action value
    Policy Update: ▷ greedy policy

$$\pi_{k+1}(a \mid s) \leftarrow \begin{cases} 1 & \text{if } a = a_k^*(s), \\ 0 & \text{otherwise} \end{cases}$$

    Value Update: $v_{k+1}(s) \leftarrow \max_a q_k(s,a)$
  end for
end while

3.6 Policy Iteration

Policy Iteration

Value Iteration 的原理是聚焦于 state value，反复应用 Bellman optimality operator 不断迭代 state value $\boldsymbol{v}_k$ 逼近 $\boldsymbol{v}^*$，其目的是不断获取更好的 state value 中间估计 (不一定是真实的 state value). 在 Definition 3.2 中最优性同时定义了 optimal policy，因此另一个视角是聚焦于 policy，通过循环更新一个真实的 policy 并将其迭代至最优.

Policy Iteration 需要基于真实的 state value 对 policy 进行迭代，因此分为两步：① Policy Evaluation (PE)，即基于当前 policy 根据 BE 计算 state value ② Policy Improvement (PI)，即根据当前 state value 迭代 policy.

① **Policy Evaluation (PE).** 基于当前 policy，根据 Bellman Equation 求解 policy:

$$\boldsymbol{v}_{\pi_k} = \boldsymbol{r}_{\pi_k} + \gamma \boldsymbol{P}_{\pi_k} \boldsymbol{v}_{\pi_k}, \quad v_{\pi_k}(s) = \sum_a \pi_k(a|s) \left(\sum_r p(r|s,a) r + \gamma \sum_{s'} p(s'|s,a) v_{\pi_k}(s') \right), s \in S, \tag{3.29}$$

其中 policy π_k 由上一步迭代得到，$\boldsymbol{r}_{\pi_k}, P_{\pi_k}$ 均为已知系统模型，$\boldsymbol{v}_{\pi_k}$ 为预求未知量.

② **Policy Improvement (PI).** 基于当前真实的 state value 使用 greedy policy 更新 policy:

$$\pi_{k+1} = \arg\max_{\pi} (\boldsymbol{r}_\pi + \gamma P_\pi \boldsymbol{v}_{\pi_k}), \quad \pi_{k+1}(s) = \arg\max_{\pi} \sum_a \pi(a|s) \underbrace{\left(\sum_r p(r|s,a) r + \gamma \sum_{s'} p(s'|s,a) v_{\pi_k}(s') \right)}_{q_{\pi_k}(s,a)}, s \in \mathcal{S},$$

$$\Rightarrow \pi_{k+1}(a \mid s) = \begin{cases} 1, & a = a_k^*(s) \\ 0, & a \neq a_k^*(s) \end{cases}, \quad a_k^*(s) = \arg\max_a q_{\pi_k}(s,a). \tag{3.30}$$

在上述 Policy Iteration 框架中，实际有一些重要问题并不明晰:

1. **Q 1:** 在 **Policy Evaluation (PE)** 中如何求解 $\boldsymbol{v}_{\pi_k} = \boldsymbol{r}_{\pi_k} + \gamma P_{\pi_k} \boldsymbol{v}_{\pi_k}$？使用迭代法还是显示求解?
   - **A 1:** 由于闭式解 $\boldsymbol{v}_{\pi_k} = (I - \gamma P_{\pi_k})^{-1} \boldsymbol{r}_{\pi_k}$ 中矩阵求逆数值上很困难，因此转而使用迭代法:
   $$\boldsymbol{v}_{\pi_k}^{(j+1)} = \boldsymbol{r}_{\pi_k} + \gamma P_{\pi_k} \boldsymbol{v}_{\pi_k}^{(j)}, \quad j = 0, 1, 2, \ldots \tag{3.31}$$
2. **Q 2:** 在 **Policy Improvement(PI)** 中 policy π_{k+1} 一定比 π_k 更好吗？为什么？
   - **A 2:** 详见附录B **Theorem ??** 及其证明.
3. **Q 3:** 算法最终一定收敛至 optimal policy π^* 吗？为什么？
   - **A 3:** 一定收敛至 optimal policy. 详见附录B **Theorem ??** 及其证明.

Policy Iteration 算法运行过程中会先后产生如下序列:

$$\pi_0 \xrightarrow{PE} v_{\pi_0} \xrightarrow{PI} \pi_1 \xrightarrow{PE} v_{\pi_1} \xrightarrow{PI} \pi_2 \xrightarrow{PE} v_{\pi_2} \xrightarrow{PI} \cdots$$

注意上述 policy value 都是真实的 policy value，并非像 value iteration 一样是中间估计.

Note 3.7. ① 在迭代过程中接近目标的 *state* 相较远离目标的更先达到 *optimal policy*，因为前者往往是后者的前提. ② 距离目标较近的 *state* 相较于远离目标的 *state* 的 *state value* 更大，因为较远的 *state* 在接近目标时获得的正奖励 *(positive reward)* 已经多次 *discount*.

Policy Iteration Algorithm

Algorithm 2 Policy Iteration Algorithm

Require: The probability model $p(r \mid s,a)$ and $p(s' \mid s,a)$ for all (s,a) are known. Initial guess π_0, tolerance θ_1, θ_2.
Ensure: **Aim:** Search for the optimal state value and an optimal policy.

while $||v_{\pi_k} - v_{\pi_{k-1}}|| \geqslant \theta_1$, for the k-th iteration **do** ▷ means not converged
  Step 1: Policy Evaluation(PE): ▷ to calculate the state value of π_k
  Initialization: an arbitrary initial guess $v_{\pi_k}^{(0)}$
  while $||v_{\pi_k}^{(j+1)} - v_{\pi_k}^{(j)}|| \geqslant \theta_2$ **do** ▷ means not converged
    for every state $s \in S$ **do** ▷ iteration method
      $$v_{\pi_k}^{(j+1)}(s) \leftarrow \sum_a \pi_k(a \mid s) \left[\sum_r p(r \mid s,a) r + \gamma \sum_{s'} p(s' \mid s,a) v_{\pi_k}^{(j)}(s') \right]$$
    end for
  end while
  Step 2: Policy Improvement(PI):
  for every state $s \in S$ **do**
    for every action $a \in A(s)$ **do**
      $q_{\pi_k}(s,a) \leftarrow \sum_r p(r \mid s,a) r + \gamma \sum_{s'} p(s' \mid s,a) v_{\pi_k}(s')$
    end for
    $a_k^*(s) \leftarrow \arg\max_a q_{\pi_k}(s,a)$
    Update policy:
    $$\pi_{k+1}(a \mid s) \leftarrow \begin{cases} 1 & \text{if } a = a_k^*(s), \\ 0 & \text{otherwise} \end{cases}$$
  end for
end while

Comparison of Policy Iteration and Value Iteration

Table 3.1: Comparison of policy iteration and value iteration

Policy Iteration	**Value Iteration**
PE: $v_{\pi_k} = r_{\pi_k} + \gamma P_{\pi_k} v_{\pi_k}$ **PI:** $\pi_{k+1} = \arg\max_\pi (r_\pi + \gamma P_\pi v_{\pi_k})$	**PU:** $\pi_{k+1} = \arg\max_\pi (r_\pi + \gamma P_\pi v_k)$ **VU:** $v_{k+1} = r_{\pi_{k+1}} + \gamma P_{\pi_{k+1}} v_k$
$\pi_0 \xrightarrow{PE} v_{\pi_0} \xrightarrow{PI} \pi_1 \xrightarrow{PE} v_{\pi_1} \xrightarrow{PI} \pi_2 \xrightarrow{PE} \cdots$	$v_0 \xrightarrow{PU} \pi_1' \xrightarrow{VU} v_1 \xrightarrow{PU} \pi_2' \xrightarrow{VU} v_2 \xrightarrow{PU} \cdots$

* 红色与蓝色部分为两种算法迭代的不同侧重点.
* PE: Policy evaluation; PI: Policy improvement; PU: Policy update; VU: Value update.

Table 3.2: Comparison of corresponding steps in policy iteration and value iteration

Stage	**Policy Iteration**	**Value Iteration**
1. Policy	π_0	|
2. Value[a]	$v_{\pi_0} = r_{\pi_0} + \gamma P_{\pi_0} v_{\pi_0}$	$v_0 := v_{\pi_0}$
3. Policy[b]	$\pi_1 \in \arg\max_\pi \left(r_\pi + \gamma P_\pi v_{\pi_0} \right)$	$\pi_1' \in \arg\max_\pi \left(r_\pi + \gamma P_\pi v_0 \right)$
4. Value[c]	$v_{\pi_1} = r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}$	$v_1 = r_{\pi_1'} + \gamma P_{\pi_1'} v_0$
5. Policy	$\pi_2 \in \arg\max_\pi \left(r_\pi + \gamma P_\pi v_{\pi_1} \right)$	$\pi_2' \in \arg\max_\pi \left(r_\pi + \gamma P_\pi v_1 \right)$
	$\vdots$	

[a] For comparison, we initialize value iteration with $v_0 = v_{\pi_0}$. In general, v_0 may be arbitrary.
[b] If $v_0 = v_{\pi_0}$ and the same tie-breaking rule is used, then $\pi_1' = \pi_1$.
[c] Policy iteration performs a complete policy evaluation, whereas value iteration performs only one Bellman backup.

3.7 Truncated Policy Iteration

Truncated Policy Iteration

通过 Table 3.2 可以看出，Policy Iteration 与 Value Iteration 在相同初始化的情况下第一次出现差异是在第 4 步，此时 Policy Iteration 理论上是要完全求解出 Bellman Equation，若使用迭代法求解即需要从 v_0 迭代 (无数次) 至收敛，但 Value Iteration 只需要从 v_0 迭代一步. Truncated Policy Iteration 则是二者的中间方案：计算 j 次:

$$\begin{aligned}
& & v_{\pi_1}^{(0)} &= v_0 \\
\text{value iteration} \leftarrow v_1 \leftarrow & & v_{\pi_1}^{(1)} &= r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}^{(0)} \\
& & v_{\pi_1}^{(2)} &= r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}^{(1)} \\
& & \vdots & \\
\textbf{truncated policy iteration} \leftarrow \bar{v}_1 \leftarrow & & v_{\pi_1}^{(j)} &= r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}^{(j-1)} \\
& & \vdots & \\
\text{policy iteration} \leftarrow v_{\pi_1} \leftarrow & & v_{\pi_1}^{(\infty)} &= r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}^{(\infty)}
\end{aligned} \tag{3.32}$$

Note 3.8. ① 上述比较都建立于假设 $v_{\pi_1}^{(0)} = v_0 = v_{\pi_0}$，否则两种算法不能比较. ② *Policy Iteration* 中实际迭代法解 *BOE* 时也不会进行无数次计算，也是一个截断的. ③ *Truncated Policy Iteration* 在计算 v_{π_k} 时次数 j 多于 *value iteration* 算法但少于 *policy iteration* 算法. 计算次数 j 控制每轮策略评价深度与外层改进次数之间的计算权衡. 最佳 m 取决于状态规模、稀疏性、γ 和求解器. 详细分析见附录*B* ***Theorem ??*** 及其证明.

Algorithm 3 Truncated Policy Iteration Algorithm

Require: The probability model $p(r \mid s,a)$ and $p(s' \mid s,a)$ for all (s,a) are known. Initial guess π_0.
Ensure: Search for the optimal state value and an optimal policy.

while the policy has not converged **do**
  Policy evaluation:
  Initialization: select the initial guess as $v_k^{(0)} = v_{k-1}$. The maximum iteration is set to j_{truncate}.
  while $j < j_{\text{truncate}}$ **do**
    for every state $s \in S$ **do**
      $v_k^{(j+1)}(s) \leftarrow \sum_a \pi_k(a \mid s) \left[\sum_r p(r \mid s,a) r + \gamma \sum_{s'} p(s' \mid s,a) v_k^{(j)}(s') \right]$
    end for
  end while
  Set $v_k \leftarrow v_k^{(j_{\text{truncate}})}$
  Policy improvement:
  for every state $s \in \mathcal{S}$ **do**
    for every action $a \in \mathcal{A}(s)$ **do**
      $q_k(s,a) \leftarrow \sum_r p(r \mid s,a) r + \gamma \sum_{s'} p(s' \mid s,a) v_k(s')$
    end for
    $a_k^*(s) \leftarrow \arg\max_a q_k(s,a)$
    Update policy:

$$\pi_{k+1}(a \mid s) \leftarrow \begin{cases} 1 & \text{if } a = a_k^*(s), \\ 0 & \text{otherwise} \end{cases}$$

  end for
end while

Small Summary

① 上述三种算法都包含两步：1. 更新 value，2. 更新 policy，这种思想被称为 **generalized policy iteration**[1]，是强化学习算法中的重要思想.

② Policy Iteration 与 Value Iteration 都是 model-based reinforcement learning (MBRL) 的方法，更准确地说是动态规划 (dynamic programming) 的方法. Policy Iteration 是下一节 Monte Carlo Learning (model-free) 的基础.

4 Lecture 4: Monte Carlo Learning

Introduction

Lecture 3 中的 Policy Iteration 和 Value Iteration 算法都是 model-based RL (MBRL) 方法，因为从式 (3.24) 可以看出他们需要已知环境模型 $p(r|s,a)$. 然而实际中很可能无法获取 p，因此一个简单的想法是通过采样进行估算.

Example 4.1. 考虑简单的掷硬币实验，投掷结果记为 X，若为正则 $X=+1$，反之 $X=0$，我们的目标是估计 $\mathbb{E}[X]$. ① *Model-based:* 当概率模型已知时，如 $p(X=1)=p(X=0)=0.5$，根据定义有 $\mathbb{E}[X]=\sum_x x\cdot p(x)=0.5$. ② *Model-free:* 根据大数定律 *(Law of Large Numbers)* $\mathbb{E}[X]\approx \bar{x}=\frac{1}{N}\sum_{j=1}^{N} x_j$，因此可以进行多次采样以逼近 $\mathbb{E}[X]$.

4.1 MC Basic

MC Basic – the Simplest MC-based RL Algorithm

MC Basic 是最简单的基于 Monte Carlo 估计的 RL 算法，其本质是将 Policy Iteration 的 policy evaluation $v_\pi(s)=\sum_a \pi(a|s)q_\pi(s,a)$ 中的 action value 的计算使用 Monte Carlo Estimation 替换成 model-free 的. 回顾：

1. **Policy Evaluation:** $\boldsymbol{v}_{\pi_k}=\boldsymbol{r}_{\pi_k}+\gamma \boldsymbol{P}_{\pi_k}\boldsymbol{v}_{\pi_k}$, $\quad v_{\pi_k}(s)=\sum_a \pi_k(a|s)\left(\sum_{r,s'} p(r,s'|s,a)r+\gamma\sum_{s'} p(s'|s,a)v_{\pi_k}(s')\right)$

2. **Policy Improvement:** $\pi_{k+1}(s)\in \arg\max_\pi \sum_a \pi(a|s)\underbrace{\left(\sum_{r,s'} p(r,s'|s,a)r+\gamma\sum_{s'} p(s'|s,a)v_{\pi_k}(s')\right)}_{q_{\pi_k}(s,a)}, s\in\mathcal{S}$

$$\Rightarrow \pi_{k+1}(a\mid s)=\begin{cases}1, & a=a_k^*(s)\\ 0, & a\neq a_k^*(s)\end{cases}, \quad a_k^*(s)=\arg\max_a q_{\pi_k}(s,a).$$

可以看到 Policy Evaluation 和 Policy Improvement 中都需要计算 action value $q_{\pi_k}(s,a)$. 在概率模型 $p(r,s'|s,a)$ 未知时，就需要使用 Monte Carlo estimation 进行估计. 回顾 action value 最原始的定义式：

$$q_{\pi_k}(s,a)=\mathbb{E}[G_t\mid S_t=s,A_t=a]=\mathbb{E}[R_{t+1}+\gamma R_{t+2}+\gamma^2 R_{t+3}+\ldots|S_t=s,A_t=a] \tag{4.1}$$

其中 G_t 为 discounted return. 从 (s,a) 开始根据 policy π_k 进行 N 次模拟 (trajectory)，它们的 return 记为 $g^{(i)}(s,a)$:

$$g^{(i)}(s,a)=r_{t+1}^{(i)}+\gamma r_{t+2}^{(i)}+\gamma^2 r_{t+3}^{(i)}+\cdots, \quad i=1,2,\ldots,N \tag{4.2}$$

那么 $\{g^{(i)}(s,a)\}_{i=1}^N$就是 G_t 的样本 (samples，在 RL 中称为经验，experience)。根据 Monte Carlo estimation 有：

$$q_{\pi_k}(s,a)=\mathbb{E}[G_t\mid S_t=s,A_t=a]\approx\frac{1}{N}\sum_{i=1}^{N} g^{(i)}(s,a). \tag{4.3}$$

Note 4.1. ① *MC Basic* 是建立在 *model-based* 算法 *Policy Iteration* 之上的 *model-free* 算法，区别在于对 *action value* q_{π_k} 的计算从基于概率模型计算变成了基于 *MC* 的估计. ② 由于 *Policy Iteration* 中 *Policy Improvement* $\pi_{new}(s)=\arg\max_a Q_\pi(s,a)$ 直接需要 *action value* Q 因此直接估计；若先估计 V 再通过 *MC* 计算 Q 误差可能增大. ③ *MC basic* 在理论上具有启发性，是别的算法的基石，但由于其效率较低，因此几乎没有应用. ④ *MC Basic* 与 *Policy Iteration* 一样仍然是收敛的，因为理论上若有足够多的 *experience* 就可以足够精确地逼近 *action value* q_{π_k}.

Note 4.2. ***Sparse Reward:*** 在使用 *MC estimation* 时 *trajectory* 的长度会对最终的估计产生影响，一个有趣的现象是 ***sparse reward***，是指 *reward* 的设计很稀疏，可能只在达到 *target* 时才有 *positive reward*，因此对于一个 *trajectory* 的最短步数有要求，因为如果步数不够就没有能到达 *target* 的可能，也就不能获得 *reward*，这对学习效率有很大影响，因此解决此问题的一个方案是设计 ***nonsparse rewards***，例如到达靠近 *target* 的区域也设置 *positive reward*.

MC Basic Algorithm

Algorithm 4 MC Basic Algorithm (a model-free variant of Policy Iteration)

Initialization: Initial guess π_0.
Aim: Search for an optimal policy.
while the value estimate has not converged, for the k-th iteration **do**
  for every state $s \in \mathcal{S}$ **do**
    for every action $a \in \mathcal{A}(s)$ **do**
      Collect sufficiently many trajectories starting from (s, a) following π_k.
      MC-based policy evaluation step:

$$q_{\pi_k}(s,a) \leftarrow \text{average return of all the trajectories starting from } (s,a)$$

    end for
    Policy improvement step:

$$a_k^*(s) \leftarrow \arg\max_a q_{\pi_k}(s,a)$$

$$\pi_{k+1}(a \mid s) = 1 \text{if } a = a_k^*(s), \text{and } \pi_{k+1}(a \mid s) = 0 \text{ otherwise}$$

  end for
end while

4.2 MC Exploring Starts

MC Exploring Starts – Use Data More Efficiently

在一条 trajectory 中，我们会连续访问许多 state-action pair，例如对于这样的一个 trajectory:

$$s_1 \xrightarrow[r=0]{a_2} s_2 \xrightarrow[r=0]{a_4} s_1 \xrightarrow[r=0]{a_2} s_2 \xrightarrow[r=1]{a_3} s_5 \xrightarrow[r=1]{a_1} \dots \tag{4.4}$$

MC Basic 算法使用的策略是 **Initial-visit method**，即原理上使用一整个 trajectory 计算出发点处 state-action pair (s_1, a_2)的 action value $q_\pi(s_1, a_2)$, 但其缺点是一个 trajectory 的数据没有被充分利用，因为一个完整的 trajectory 事实上可以依次拆分成如下多个 sub-trajectories:

$$\begin{aligned}
s_1 \xrightarrow{a_2} s_2 \xrightarrow{a_4} s_1 \xrightarrow{a_2} s_2 \xrightarrow{a_3} s_5 \xrightarrow{a_1} \dots &\text{[original trajectory, estimate } q_\pi(s_1,a_2)] \\
s_2 \xrightarrow{a_4} s_1 \xrightarrow{a_2} s_2 \xrightarrow{a_3} s_5 \xrightarrow{a_1} \dots &\text{[trajectory starting from } (s_2,a_4), \text{ estimate } q_\pi(s_2,a_4)] \\
s_1 \xrightarrow{a_2} s_2 \xrightarrow{a_3} s_5 \xrightarrow{a_1} \dots &\text{[trajectory starting from } (s_1,a_2), \text{ estimate } q_\pi(s_1,a_2)] \\
s_2 \xrightarrow{a_3} s_5 \xrightarrow{a_1} \dots &\text{[trajectory starting from } (s_2,a_3), \text{ estimate } q_\pi(s_2,a_3)] \\
s_5 \xrightarrow{a_1} \dots &\text{[trajectory starting from } (s_5,a_1), \text{ estimate } q_\pi(s_5,a_1)]
\end{aligned} \tag{4.5}$$

First vs. Every. 为了能够更充分利用一个 trajectory 的数据，更 data-efficient 的方法是计算其中所有出现过 visit (state-action pair) 的 action value. 同时，在一个 trajectory 中同一个 state-action pair 可能出现多次，这就产生了两种方式：① **First-visit Method:** 只使用第一次出现时计算得到的 $q_\pi(s_i, a_j)$ 作为返回值，第二次的不计算. 这样对于一个 state-action pair，一个 trajectory 只能得到一次采样. ② **Every-visit Method:** 对于重复出现的 state-action pair，每一次都计算 action value. 这样对于一个 state-action pair，一个 trajectory 可能得到多次采样.

Efficiency vs. Accuracy. 在 MC-based RL 中另一个重要的问题是什么时候更新 policy，这里仍然有两种方式：① **Average return:** 在 MC Basic 中，使用多个 trajectory 取平均的 average return 作为 action value 的估计值，这样效率较慢但是一次得到的 action value 较准确. ② **Single trajectory:** 只使用一次 trajectory 来估计 action value，这样虽然可能不准确，但更新效率更高. 这种思想与 truncated policy iteration 类似.

MC Exploring Starts Algorithm

MC Exploring Starts 中 *exploring* 指我们需要从一个 state-action pair 出发，访问到所有的 state-action pair 才能知道某一个 action 是否为最优，因此需要尽量 explore 所有的 pair. Starts 是指从一个 visit 出发有限次访问可能访问不到某些 pair，因此一般需要随机从一个 start 开始 explore，保证遍历所有的 state-action pair。算法如下：

Algorithm 5 MC Exploring Starts (a sample-efficient variant of MC Basic)

Initialization: Initial guess π_0.
Aim: Search for an optimal policy.
trajectory generation: Randomly select a starting state-action pair (s_0, a_0) and ensure that all pairs can be possibly selected. ▷ exploring starts requirement
for each trajectory **do**
  Following the current policy, generate a trajectory of length T: $s_0, a_0, r_1, \ldots, s_{T-1}, a_{T-1}, r_T$.
  Policy evaluation and policy improvement:
    Initialization: $g \leftarrow 0$
  for each step of the trajectory, $t = T-1, T-2, \ldots, 0$ **do** ▷ compute inversely to improve efficiency
    $g \leftarrow \gamma g + r_{t+1}$
    $\text{Returns}(s_t, a_t) \leftarrow \text{Returns}(s_t, a_t) + g$
    $\text{Num}(s_t, a_t) \leftarrow \text{Num}(s_t, a_t) + 1$
    Policy evaluation: $q(s_t, a_t) \leftarrow \text{Returns}(s_t, a_t)/\text{Num}(s_t, a_t)$
    Policy improvement: $\begin{cases} 1 & \text{if } a = \arg\max_a q(s_t, a), \\ 0 & \text{otherwise.} \end{cases}$
  end for
end for

Note 4.3. ① 形式上，如果起始分布满足 $P(S_0 = s, A_0 = a) > 0$ 对所有 *state-action pair* 都成立，那么经过足够多 *trajectory* 后，每个 (s, a) 都会被反复采样. ② *Exploring Starts* 是一种理论上很方便的探索假设，但在物理世界中通常难以实现，因为可能无法把机器人直接放到任意状态并强制执行任意动作. ③ 在计算一个 *trajectory* 中正向计算 g_t 需要预先知晓后续的 g_{t+1}: $g_t = \gamma g_{t+1} + r_{t+1}$，因此采用反向计算，先计算后续的 g_t 以提高计算效率. ④ 显然只有当一个 *state-action pair* 被充分访问多次时才能准确估计其 *action value*，这被称为 ***exploring starts requirement***.

4.3 MC ϵ-Greedy

MC ϵ-Greedy – MC without Exploring Starts

在实际中由于物理限制很难对 start 进行 explore，因此更常见的做法不是强制改变初始状态动作，而是在访问状态时给所有动作一定探索概率. 此时只需一个足够长的 trajectory 也可以访问每一个 state-action pair 很多次，满足 exploring starts requirement. 这样的 policy 被称为 **soft policy**，即在任意 state 任意 action 都有概率被执行.

MC ϵ-Greedy 的核心就是在 policy update 时将 policy 变成 soft policy 以去除 exploring starts requirement:

$$\pi(a \mid s) = \begin{cases} 1 - \frac{\varepsilon}{|\mathcal{A}(s)|}(|\mathcal{A}(s)| - 1), & \text{for the greedy action} \\ \frac{\varepsilon}{|\mathcal{A}(s)|}, & \text{for the other } |\mathcal{A}(s)| - 1 \text{ actions} \end{cases} \tag{4.6}$$

其中 $\epsilon \in [0, 1]$，$\mathcal{A}(s)$ 为 state s 对应的 action space.

Note 4.4. 可以看到 *soft policy / ϵ-greedy policy* 仍将最大的概率保留给了 *action value* 最大的 *greedy action*：

$$1 - \frac{\varepsilon}{|\mathcal{A}(s)|}(|\mathcal{A}(s)| - 1) = 1 - \varepsilon + \frac{\varepsilon}{|\mathcal{A}(s)|} \geqslant \frac{\varepsilon}{|\mathcal{A}(s)|}. \tag{4.7}$$

Exploitation vs. Exploration. 使用 ϵ-greedy policy 的原因是平衡 **exploitation** (始终选择最优) 与 **exploration** (探索). Exploitation 指只选择最优 action (即 greedy)， exploration 给别的 action 也分配机会，防止陷入局部最优.

Note 4.5. ① 当 $\epsilon = 0$ 时，就变成了 *greedy*，此时 *less exploration but more exploitation*. ② 当 $\epsilon = 1$ 时，所有的 *action* 选择概率相同，此时 *more exploration*. ③ 在实际使用中，可以设置较小的 ϵ，然后再逐渐减小.

Implementation. 在实际算法部署中可以通过 [0,1]上的均匀分布产生一随机数 x，若 $x \geqslant \epsilon$ 则选择 greedy action，否则对于 $\mathcal{A}(s)$ 中所有的 action 都有 $\frac{1}{|\mathcal{A}(s)|}$ 的概率选择到.

Consistency: 在 ϵ-Greedy 中，如果每一个 state 采取的 action 都有一定的概率，但是最大概率对应的 action 与 optimal action 一致，那么就称此 ϵ 下的算法是 consistent 的. 一般来说当 ϵ 较小时更容易保证 consistent.

MC ϵ-Greedy Algorithm

将 ϵ-greedy 的策略替换掉原先的 greedy 的策略，就可得到 MC ϵ-Greedy algorithm，同时注意此时 trajectory generation 时 start 的选择需要符合环境规定.

Algorithm 6 MC ϵ-Greedy (a variant of MC Exploring Starts)

Initialization: Initial guess π_0 and the value of $\epsilon \in [0, 1]$.
Aim: Search for an optimal policy.
trajectory generation: Randomly select an environment-permitted starting state-action pair (s_0, a_0). Following the current policy, generate an trajectory of length T: $s_0, a_0, r_1, \ldots, s_{T-1}, a_{T-1}, r_T$.
for each trajectory **do**
    Policy evaluation and policy improvement:
        Initialization: $g \leftarrow 0$
    for each step of the trajectory, $t = T-1, T-2, \ldots, 0$ **do** ▷ for efficiency
        $g \leftarrow \gamma g + r_{t+1}$
        $\text{Returns}(s_t, a_t) \leftarrow \text{Returns}(s_t, a_t) + g$
        $q(s_t, a_t) \leftarrow \text{average}(\text{Returns}(s_t, a_t))$
        Let $a^* = \arg\max_a q(s_t, a)$ and
        $$\pi(a \mid s_t) \leftarrow \begin{cases} 1 - \epsilon + \frac{\epsilon}{|\mathcal{A}(s_t)|} & \text{if } a = a^*, \\ \frac{\epsilon}{|\mathcal{A}(s_t)|} & \text{if } a \neq a^*. \end{cases}$$
    end for
end for

5 Lecture 5: Stochastic Approximation and Stochastic Gradient Descent

5.1 Robbins-Monro Algorithm (RM Algorithm)

Incremental update

许多算法中都涉及到收集样本并对某些值进行估计，常规的做法是使用非增量式计算 (**non-incremental**)，即等到 N 个样本采样完成后再进行下一步，但这往往会降低算法效率. 一种替代方式是使用增量式 (**incremental**) 更新，即立即使用新的采样数据迭代更新目标量，这样虽然在算法初期损失了部分精度但无需等待、提升了算法效率.

Example 5.1 (Mean Estimation)**.** 在求经验期望时，非增量式计算会采样 N 个样本 $\{x_i\}_{i=1}^N$ 并统一计算：

$$\mathbb{E}[X] \approx \bar{x} := \frac{1}{N}\sum_{i=1}^{N} x_i \tag{5.1}$$

但若规定：

$$w_{k+1} = \frac{1}{k}\sum_{i=1}^{k} x_i, \quad k = 1, 2, \ldots, N, \qquad w_k = \frac{1}{k-1}\sum_{i=1}^{k-1} x_i, \quad k = 2, 3, \ldots, N. \tag{5.2}$$

那么就能得到如下 *incremental* 更新格式:

$$w_{k+1} = \frac{1}{k}\sum_{i=1}^{k} x_i = \frac{1}{k}\left(\sum_{i=1}^{k-1} x_i + x_k\right) = \frac{1}{k}\left((k-1)w_k + x_k\right) = w_k - \frac{1}{k}(w_k - x_k) \tag{5.3}$$

一般地，上述 *mean estimation* 的 *incremental* 格式可以总结为：

$$w_{k+1} = w_k - \alpha_k(w_k - x_k), \tag{5.4}$$

其中 $\alpha_k > 0$，后续将说明当其满足一些条件时，仍然会保证收敛：$w_k \xrightarrow{k\to\infty} \mathbb{E}[X]$.

Robbins-Monro algorithm

Robbins-Monro (RM) 算法是随机近似算法 (Stochastic approximation, SA) 中的一种. SA 常用于方程求根问题和优化问题，相较于牛顿法、梯度下降等算法，SA 可以在函数表达式未知 (但可获得有噪输出) 的情况下使用迭代更新的方式求解问题. 上述 incremental 格式的 mean estimation 和随机梯度下降 (SGD) 都是 RM 算法的特殊情况.

Problem statement. 考虑一个方程的求零点问题 $f(w) = 0$，其中 $f(w)$ 为表达式未知的黑盒 (black box) (例如神经网络就可以抽象为 $f(w) = y$ 的形式，但是 g 的形式是未知的)，我们只能获得有噪输出

$$\tilde{f}(w, \eta) = f(w) + \eta.$$

其中 η 为观测噪声.

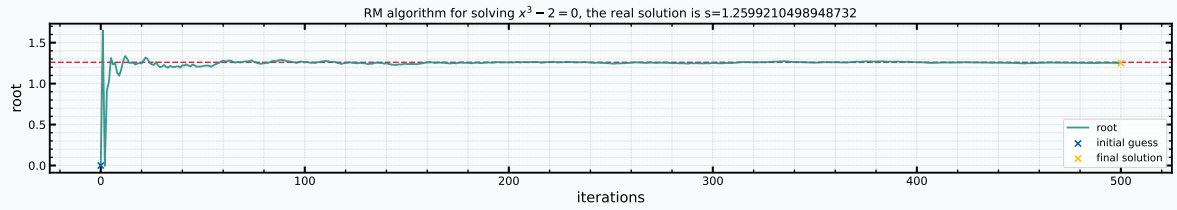
Note 5.1. 实际上无约束凸优化问题和方程的求根问题都是 $f(w) = 0$ 的形式：

$$\text{无约束凸优化问题：}\ f(w) = \nabla_w J(w) = 0 \qquad \text{求根问题：}\ f(w) = c \Rightarrow h(w) \triangleq f(w) - c = 0$$

Robbins-Monro (RM) algorithm. RM 算法通过如下迭代方式逼近 $f(w) = 0$ 的真实解：

$$w_{k+1} = w_k - a_k\tilde{f}(w_k, \eta_k), \quad k = 1, 2, 3, \ldots, \tag{5.5}$$

其中 w_k 为对解的第 k 次估计. 可以看出 RM 算法只依赖于输入和输出数据：① Input sequence: $\{w_k\}$ ② Noisy output sequence: $\{\tilde{f}(w_k, \eta_k)\}$. 一个简单的 求解 $f(x) = x^3 - 2 = 0$ 的根的数值案例如下图所示，初始猜测 $x_0 = 0$.



Robbins-Monro Algorithm – Convergence Analysis

Theorem 5.1 (Robbins-Monro theorem)**.** 对于 *Robbins-Monro* 算法，若 ① $\forall w, 0 < c_1 \leqslant \nabla_w f(w) \leqslant c_2$ *(*梯度假设*)* ② $\sum_{k=1}^{\infty} a_k = \infty, \sum_{k=1}^{\infty} a_k^2 < \infty$ *(*系数假设*)* ③ $\mathbb{E}[\eta_k|\mathcal{H}_k] = 0, \mathbb{E}[\eta_k^2|\mathcal{H}_k] < \infty$ *(*误差假设*)*，其中 $\mathcal{H}_k = \{w_k, w_{k-1}, \dots\}$. 设方程 $f(w) = 0$ 的根为 w^*，满足 $f(w^*) = 0$，那么 w_k 以概率*1* 收敛 *(with probability 1, w.p.1)* 至 w^*.

Note 5.2. 对于上述三个条件，有如下详细解读:

1. ① 梯度假设
   - *(a)* $\nabla_w f(w) > 0$，即原函数单调上升，保证了方程 $f(w) = 0$ 若有解则一定唯一. 这一条件一般可以满足，例如当处理优化问题时，$f(w)$ 本身即梯度，$\nabla f(w) > 0$ 表示原函数是凸的，即较常见的凸优化问题.
   - *(b)* $\nabla_w f(w) \leqslant c_2$ 说明梯度有界，不过不满足此条件算法有时也能够收敛.
2. ② 系数假设
   - *(a)* $\sum_{k=1}^{\infty} a_k^2 < \infty$ 表明 $a_k \xrightarrow{k\to\infty} 0$. 这可以保证 $w_{k+1} - w_k = -a_k \tilde{f}(w_k, \eta_k) \xrightarrow{k\to\infty} 0$，即收敛.
   - *(b)* $\sum_{k=1}^{\infty} a_k = \infty$ 表明 $a_k \xrightarrow{k\to\infty} 0$ 没有那么快. 将所有递推式 $w_{k+1} = w_k - a_k \tilde{f}(w_k, \eta_k)$ 两端累和可以得到

     $$w_1 - w_\infty = \sum_{k=1}^{\infty} a_k \tilde{f}(w_k, \eta_k)$$

     若 $\sum_{k=1}^{\infty} a_k < \infty$，由于 $\tilde{f}$ 有界，那么$|\sum_{k=1}^{\infty} a_k \tilde{f}(w_k, \eta_k)|$ 有界，这就要求初始猜测 w_1 不能距 w^* 过远，反之若收敛至零较慢则可放宽对初始猜测的要求.
   - *(c)* 取 $a_k = \frac{1}{k}$是可以满足要求的，因为有 $\sum_{k=1}^{n} \frac{1}{k} \xrightarrow{n\to\infty} \infty, \sum_{k=1}^{\infty} \frac{1}{k^2} < \infty$, 原因在于:

     $$\lim_{n\to\infty} \left(\sum_{k=1}^{n} \frac{1}{k} - \ln n \right) = \kappa, \ln n \xrightarrow{n\to\infty} \infty \Rightarrow \sum_{k=1}^{n} \frac{1}{k} \xrightarrow{n\to\infty} \infty, \quad \sum_{k=1}^{\infty} \frac{1}{k^2} = \frac{\pi^2}{6} < \infty (\textit{Basel problem}). \tag{5.6}$$

     其中 $\kappa \approx 0.577$ 为 *Euler-Mascheroni* 常数[a].
   - *(d)* 常见的做法并非令系数为 $\frac{1}{k}$，因为当 k 较大时，后面的数据在算法中起到的作用被无限缩小，这并不是我们想要的，因此常见的做法是开始时赋予较小的值再慢慢减小但不至于趋于 *0*
3. ③ – 测量误差假设. 这里并不要求 η 满足高斯性, 常见的做法是取 $\{\eta_k\}$ 来自一个 *i.i.d.* 的随机序列，满足 $\mathbb{E}[\eta_k|\mathcal{H}_k] = \mathbb{E}[\eta_k] = 0, \mathbb{E}[\eta_k^2|\mathcal{H}_k] = \mathbb{E}[\eta_k^2] < \infty$.

[a] 欧拉-马斯刻若尼常数是一个数学常数，定义为调和级数与自然对数的差值

Robbins-Monro algorithm – Applying to mean estimation

定义 $f(w) \triangleq w - \mathbb{E}[X]$，我们需要求出 $f(w) = 0$ 的解 w^*，可以定义有噪观测为 $\tilde{f}(w, x) \triangleq w - x$，那么可以得到:

$$\tilde{f}(w, \eta) = (w - \mathbb{E}[X]) + (\mathbb{E}[X] - x) \triangleq f(w) + \eta \tag{5.7}$$

其中 $\eta \triangleq \mathbb{E}[X] - x$，$x$ 是对 X 的采样. 代入 RM 算法 (5.5) 中就得到:

$$w_{k+1} = w_k - \alpha_k \tilde{f}(w_k, \eta_k) = w_k - \alpha_k (w_k - x_k) \tag{5.8}$$

再套用 Theorem 5.1 即可得到相应的收敛性结论，并且此收敛性不要求 X 服从高斯分布或其他特定分布族，但仍需满足相应的矩条件和采样条件. 其收敛性分析也可以直接使用 Dvoretzkys convergence theorem，详细证明见附录C.

5.2 Dvoretzky's Convergence Theorem

Dvoretzkys convergence theorem

Dvoretzkys convergence theorem 是 SA 领域的经典结论，该定理可用于分析 RM 和许多强化学习算法的收敛性.

Theorem 5.2 (Dvoretzkys convergence theorem)**.** 考虑如下随机过程 *(*证明见附录*C)*:

$$w_{k+1} = (1-\alpha_k)w_k + \beta_k \eta_k, \tag{5.9}$$

其中 $\{\alpha_k\}_{k=1}^{\infty}, \{\beta_k\}_{k=1}^{\infty}, \{\eta_k\}_{k=1}^{\infty}$ 均为随机序列 *(stochastic sequences)*，$\forall\, k, \alpha_k \geqslant 0, \beta_k \geqslant 0$，那么 $w_k \xrightarrow{w.p.1} 0$，若:

1. $\sum_{k=1}^{\infty} \alpha_k = \infty, \sum_{k=1}^{\infty} \alpha_k^2 < \infty; \sum_{k=1}^{\infty} \beta_k^2 < \infty$ *uniformly w.p.1*
2. $\mathbb{E}[\eta_k | \mathcal{H}_k] = 0, \mathbb{E}[\eta_k^2 | \mathcal{H}_k] \leqslant C$ *w.p.1*，其中 $\mathcal{H}_k = \{w_k, w_{k-1}, \ldots, \eta_{k-1}, \ldots, \alpha_{k-1}, \ldots, \beta_{k-1}, \ldots\}$.

Note 5.3. 关于 *Dvoretzkys convergence theorem(DCT)* 有如下几点说明:

1. *DCT* 中 $\{\alpha_k\}, \{\beta_k\}$ 依赖于 $\mathcal{H}_k$，但在 *RM* 算法中 $\{\alpha_k\}$ 是确定性的，不依赖于 $\mathcal{H}_k$.
2. *DCT* 的推广版本可用于分析后续的 *Q-learning* 和 *TD learning* 算法.

Theorem 5.3 (Extension of Dvoretzky's theorem)**.** 设 $\mathcal{S}$ 为有限实数集. 对于随机过程

$$\Delta_{k+1}(s) = (1-\alpha_k(s))\Delta_k(s) + \beta_k(s)\eta_k(s), \tag{5.10}$$

若对于每个 $s \in \mathcal{S}$，满足以下条件，则有 $\Delta_k(s) \to 0$ 几乎必然成立:

(a) $\sum_k \alpha_k(s) = \infty$，$\sum_k \alpha_k^2(s) < \infty$，$\sum_k \beta_k^2(s) < \infty$，并且 $\mathbb{E}[\beta_k(s) \mid \mathcal{H}_k] \leqslant \mathbb{E}[\alpha_k(s) \mid \mathcal{H}_k]$ 几乎必然一致成立;

(b) $\|\mathbb{E}[\eta_k(s) \mid \mathcal{H}_k]\|_{\infty} \leqslant \gamma \|\Delta_k\|_{\infty}$，其中 $\gamma \in (0,1)$；$\mathrm{var}[\eta_k(s) \mid \mathcal{H}_k] \leqslant C(1 + \|\Delta_k(s)\|_{\infty}^2)$，其中 C 为常数.

$\mathcal{H}_k = \{\Delta_k, \Delta_{k-1}, \ldots, \eta_{k-1}, \ldots, \alpha_{k-1}, \ldots, \beta_{k-1}, \ldots\}$ 表示历史信息. 符号 $\|\cdot\|_{\infty}$ 表示最大范数*(maximum norm)*.

Note 5.4. *Mean estimation* 问题可以归约为 *RM* 算法从而直接使用 ***RM Theorem 5.1*** 进行收敛性分析，也可以使用 ***Dvoretzky's Theorem 5.2***分析，详见附录*C*.

5.3 Stochastic Gradient Decent

Stochastic gradient descent – Algorithm

随机梯度下降 (Stochastic gradient descent, SGD) 算法是机器学习中的常用优化算法，是梯度下降的随机化版本.

Problem statement. 考虑优化问题 $\min_w \ J(w) = \mathbb{E}[f(w, X)]$，其中 w 为优化的目标变量、X 为随机变量；w, X 可为标量也可以为向量，$f(\cdot)$ 为标量.

Gradient Decent (GD). 最基础的梯度下降法对解析式中的期望式求梯度，再使用梯度进行更新:

$$w_{k+1} = w_k - \alpha_k \nabla_w \mathbb{E}[f(w_k, X)] = w_k - \alpha_k \mathbb{E}[\nabla_w f(w_k, X)], \tag{5.11}$$

其中 α_k 为步长，控制下降的快慢. 实际使用中由于完整分布常未知，会使用所有样本点进行梯度估计并更新:

$$\mathbb{E}[\nabla_w f(w_k, X)] \approx \frac{1}{n}\sum_{i=1}^{N} \nabla_w f(w_k, x_i), \quad w_{k+1} = w_k - \alpha_k \frac{1}{N}\sum_{i=1}^{N} \nabla_w f(w_k, x_i) \tag{5.12}$$

Mini-batch Gradient Decent (BGD). 由于计算资源的限制和计算效率的要求，一次性计算全量样本的梯度并更新往往不可行，因此实际中会将完整数据集切分成小数据集 (大小为 B) 分批次进行 GD:

$$w_{k+1} = w_k - \alpha_k \frac{1}{B}\sum_{i=1}^{B} \nabla_w f(w_k, x_i) \tag{5.13}$$

Stochastic Gradient Decent (SGD). 为进一步降低计算量，随机梯度下降每次仅使用一个随机样本计算梯度:

$$w_{k+1} = w_k - \alpha_k \nabla_w f(w_k, x_k) \tag{5.14}$$

Stochastic gradient decent – Applying to mean estimation

Mean estimation 实际上是特殊的 SGD 算法，我们只需将其形式化为如下优化问题：

$$\min_{w} \quad J(w) = \mathbb{E}[f(w, X)] \triangleq \mathbb{E}\left[\frac{1}{2}\|w - X\|^2\right], \tag{5.15}$$

其中 $f(w, X) \triangleq \|w - X\|^2/2$，其梯度 $\nabla_w f(w, X) = w - X$. 通过求解 $\nabla_w J(w) = 0$ 可以得到最优解 $w^* = \mathbb{E}[X]$.

GD for Eq. (5.15): $w_{k+1} = w_k - \alpha_k \nabla_w J(w_k) = w_k - \alpha_k \mathbb{E}[\nabla_w f(w_k, X)] = w_k - \alpha_k \mathbb{E}[w_k - X]$.

SGD for Eq. (5.15): 使用单个样本代替期望，$w_{k+1} = w_k - \alpha_k \nabla_w f(w_k, x_k) = w_k - \alpha_k (w_k - x_k)$. 可以看到这与 mean estimation 的算法(5.4)一样，因此 mean estimation 是特殊的 SGD.

Stochastic gradient decent – Convergence

对比 GD 与 SGD，二者不同就是 SGD 将梯度从无噪的对期望求梯度 ($\nabla_w \mathbb{E}[f(w_k, X)]$) 变成了有噪的对单样本点求梯度 ($\nabla_w f(w_k, x_k)$)，因此 $\nabla_w f(w_k, x_k)$ 就可以看作 $\nabla_w \mathbb{E}[f(w_k, X)] = \mathbb{E}[\nabla_w f(w, X)]$ 的有噪估计：

$$\nabla_w f(w_k, x_k) = \mathbb{E}[\nabla_w f(w, X)] + \underbrace{\nabla_w f(w_k, x_k) - \mathbb{E}[\nabla_w f(w, X)]}_{\eta}. \tag{5.16}$$

Relative error analysis. 为衡量梯度计算发生改变带来的影响，定义如下梯度相对误差 (relative error):

$$\delta_k \triangleq \frac{\left|\nabla_w f\left(w_k, x_k\right) - \mathbb{E}\left[\nabla_w f\left(w_k, X\right)\right]\right|}{\left|\mathbb{E}\left[\nabla_w f\left(w_k, X\right)\right]\right|}. \tag{5.17}$$

为简便起见，假设 $w, \nabla_w f\left(w_k, x_k\right)$ 均为标量且 $f(\cdot)$ 是严格凸的，根据拉格朗日中值定理有：

$$\delta_k = \frac{\left|\nabla_w f\left(w_k, x_k\right) - \mathbb{E}\left[\nabla_w f\left(w_k, X\right)\right]\right|}{\left|\mathbb{E}\left[\nabla_w f\left(w_k, X\right)\right] - \mathbb{E}\left[\nabla_w f\left(w^*, X\right)\right]\right|} = \frac{\left|\nabla_w f\left(w_k, x_k\right) - \mathbb{E}\left[\nabla_w f\left(w_k, X\right)\right]\right|}{\left|\mathbb{E}\left[\nabla_w^2 f\left(\tilde{w}_k, X\right)\left(w_k - w^*\right)\right]\right|}, \tag{5.18}$$

其中 $\tilde{w}_k \in]w_k, w^*]$. 由于已经假设函数 $f(\cdot)$ 是严格凸的，即 $\exists\, c > 0,\ \forall w, X,\ \nabla_w^2 f \geqslant c \geqslant 0$，有：

$$\left|\mathbb{E}\left[\nabla_w^2 f\left(\tilde{w}_k, X\right)\left(w_k - w^*\right)\right]\right| = \left|\mathbb{E}\left[\nabla_w^2 f\left(\tilde{w}_k, X\right)\right]\right| \left|\left(w_k - w^*\right)\right| \geqslant c \left|w_k - w^*\right|. \tag{5.19}$$

代回式 (5.17) 中得：

$$\delta_k \leqslant \frac{\left|\overbrace{\nabla_w f(w_k, x_k)}^{\text{stochastic gradient}} - \overbrace{\mathbb{E}[\nabla_w f(w_k, X)]}^{\text{true gradient}}\right|}{\underbrace{c|w_k - w^*|}_{\text{distance to the optimal solution}}}. \tag{5.20}$$

从上式可以看出 ① 当 $w_k - w^*$ 较大，即离最优解较远时，SGD 会表现得更像 GD，梯度下降更新的更准确；② 当 $w_k - w^*$ 较小，即离最优解较近时，相对误差上界增大，可能会存在更多的随机性，但是此时已经接近了最优解.

Theorem 5.4 (Convergence of SGD). 对于 *SGD 算法(5.14)*，定义满足 $\nabla_w \mathbb{E}[f(w, X)] = 0$ 的根为 w^*，则 $w_k \xrightarrow{w.p.1} w^*$ 若满足以下条件：

1. $0 < c_1 \leqslant \nabla_w^2 f(w, X) \leqslant c_2$ *(严格凸性)*
2. $\sum_{k=1}^{\infty} a_k = \infty, \quad \sum_{k=1}^{\infty} a_k^2 < \infty$ *(与 RM 算法相同)*
3. $\{x_k\}_{k=1}^{\infty}$ *i.i.d. (常见要求)*

首先优化问题 $\min_w\ J(w) = \mathbb{E}[f(w, X)]$ 在严格凸的条件下可以转化为如下求根问题：

$$J(w) = \mathbb{E}[f(w, X)] \Rightarrow g(w) \triangleq \nabla_w J(w) = \mathbb{E}[\nabla_w f(w, X)] = 0. \tag{5.21}$$

令 $\tilde{g}(w, \eta) = \nabla_w f(w, x) = \underbrace{\mathbb{E}[\nabla_w f(w, X)]}_{g(w)} + \underbrace{\nabla_w f(w, x) - \mathbb{E}[\nabla_w f(w, X)]}_{\eta}$，那么解决 $g(w) = 0$ 的 RM 算法为：

$$w_{k+1} = w_k - a_k \tilde{g}(w_k, \eta_k) = w_k - a_k \nabla_w f(w_k, x_k). \tag{5.22}$$

此即 SGD，这样根据 Theorem 5.1 就可以得到上述结论. 证明详见附录C.

6 Lecture 6: Temporal-Difference Learning

Introduction

Lecture 5 中介绍了 Robbins-Monro (RM) Algorithm:

$$w_{k+1} = w_k - a_k \tilde{f}(w_k, \eta_k), \quad k = 1, 2, 3, \ldots, \tag{6.1}$$

其中 w_k 为对解的第 k 次估计. Temporal-Difference Learning (TD learning) 算法思想与 RM 算法相似，是强化学习中的一大类方法，包含例如 basic TD, Sarsa, n-step Sarsa 和 Q-learning 算法在内的多种强化学习算法.

6.1 Basic TD: TD Learning of State Values

TD Learning of state values – Basic TD

与 Lecture 4 中的 Monte Carlo Learning 相似，最简单的 TD learning 算法与 MC Basic 一样是应用于 Policy Iteration 中的 policy evaluation，被称为 Basic TD. 回顾 Policy Iteration 算法如下：

1. **Policy Evaluation:** $\boldsymbol{v}_{\pi_k} = \boldsymbol{r}_{\pi_k} + \gamma \boldsymbol{P}_{\pi_k} \boldsymbol{v}_{\pi_k}, \quad v_{\pi_k}(s) = \sum_a \pi_k(a|s) \left(\sum_{r,s'} p(r, s'|s, a) r + \gamma \sum_{s'} p(s'|s, a) v_{\pi_k}(s') \right)$
2. **Policy Improvement:** $\pi_{k+1}(s) = \arg\max_\pi \sum_a \pi(a|s) \underbrace{\left(\sum_{r,s'} p(r, s'|s, a) r + \gamma \sum_{s'} p(s'|s, a) v_{\pi_k}(s') \right)}_{q_{\pi_k}(s,a)}, s \in \mathcal{S}$

$$\Rightarrow \pi_{k+1}(a \mid s) = \begin{cases} 1, & a = a_k^*(s) \\ 0, & a \neq a_k^*(s) \end{cases}, \quad a_k^*(s) = \arg\max_a q_{\pi_k}(s, a).$$

MC Basic 是 model-free 算法，其做法是从 (s, a) 开始根据 policy π_k 进行 N 次模拟 (trajectory)，并对它们的 return $g^{(i)}(s, a) = r_{t+1}^{(i)} + \gamma r_{t+2}^{(i)} + \gamma^2 r_{t+3}^{(i)} + \cdots$ 取平均以逼近真实 action value $q_{\pi_k}(s, a)$:

$$q_{\pi_k}(s, a) = \mathbb{E}[G_t \mid S_t = s, A_t = a] \approx \frac{1}{N} \sum_{i=1}^{N} g^{(i)}(s, a). \tag{6.2}$$

与 MC Basic 一样，Basic TD 也是 model-free 算法，依赖于数据，即由给定的 policy π 生成的 state-reward 序列:

$$(s_0, r_1, s_1, \ldots, s_t, r_{t+1}, s_{t+1}, \ldots) \text{ or } \{(s_t, r_{t+1}, s_{t+1})\}_t. \tag{6.3}$$

Basic TD Algorithm. 但与 MC Basic 不同的是，Basic TD 不再等待所有采样结束后再进行估计，而是借鉴 RM 算法增量更新 (incremental update) 的思想在数据收集过程中不断迭代. 根据 state value $v(s)$ 的定义，有:

$$v_\pi(s_t) = \mathbb{E}\left[R_{t+1} + \gamma v_\pi(S_{t+1}) | S_t = s_t\right] \Rightarrow f(\boldsymbol{v}) = v(s_t) - (r_{t+1} + \gamma v(s_{t+1})) = 0. \tag{6.4}$$

其中 $f(\boldsymbol{v})$ 只使用了一条 trajectory，因此是一个有噪观测. 借鉴 RM 算法 (6.1)，将上式 $f(\boldsymbol{v}) = 0$ 的求解写成 RM 算法的增量更新形式就得到了 **Basic TD** 算法:

$$v_{t+1}(s_t) = v_t(s_t) - \alpha_t(s_t) \left[v_t(s_t) - [r_{t+1} + \gamma v_t(s_{t+1})] \right], \tag{6.5a}$$

$$v_{t+1}(s) = v_t(s), \quad \forall s \neq s_t \text{(未被访问的状态不更新，有时省略此式)}. \tag{6.5b}$$

其中式 (6.5a) 可以更详细地拆解为:

$$\underbrace{v_{t+1}(s_t)}_{\text{new estimate}} = \underbrace{v_t(s_t)}_{\text{current estimate}} - \alpha_t(s_t) \left[\overbrace{v_t(s_t) - \underbrace{[r_{t+1} + \gamma v_t(s_{t+1})]}_{\text{TD target } \bar{v}_t}}^{\text{TD error } \delta_t} \right]. \tag{6.6}$$

可以看出，新的估计 $v_{t+1}(s_t)$ 是当前估计 $v_t(s_t)$ 和 TD error [a]的组合.

[a]按照 [1] 中的写法，TD error $\delta_t \triangleq [r_{t+1} + \gamma v(s_{t+1})] - v(s_t) = \bar{v}_t - v(s_t)$.

Basic TD – TD Target and TD Error

TD Target. TD target 定义为:

$$\bar{v}_t \triangleq r_{t+1} + \gamma v(s_{t+1}). \tag{6.7}$$

称之为 target 是因为我们迭代的目标就是让 $v(s_t)$ 不断靠近 $\bar{v}_t$. 根据式(6.6)，可以得到

$$\begin{aligned} &v_{t+1}(s_t) - \bar{v}_t = v_t(s_t) - \bar{v}_t - \alpha_t(s_t)\left[v_t(s_t) - \bar{v}_t\right] = \left[1 - \alpha_t(s_t)\right]\left[v_t(s_t) - \bar{v}_t\right] \\ &\Longrightarrow |v_{t+1}(s_t) - \bar{v}_t| = |1 - \alpha_t(s_t)||v_t(s_t) - \bar{v}_t|. \end{aligned} \tag{6.8}$$

由于 discount rate $\gamma \in (0, 1)$，因此有:

$$0 < \alpha_t(s_t) < 1 \Rightarrow 0 < 1 - \alpha_t(s_t) < 1 \Rightarrow |v_{t+1}(s_t) - \bar{v}_t| \leqslant |v_t(s_t) - \bar{v}_t|. \tag{6.9}$$

因此 $v_t(s_t) \to \bar{v}_t$.

TD Error. TD Error 反映了估计值 v_t 与真实状态值 v_π 之间的差异，其定义为:

$$\delta_t \triangleq v(s_t) - [r_{t+1} + \gamma v(s_{t+1})] = v(s_t) - \bar{v}_t. \tag{6.10}$$

由于 $v_\pi(s_t) = \mathbb{E}\left[R_{t+1} + \gamma v_\pi(S_{t+1}) | S_t = s_t\right]$，因此在期望意义下 $\delta_t = 0$，即若 $v_t = v_\pi$，则 $\delta_t = 0$:

$$\mathbb{E}[\delta_{\pi,t} | S_t = s_t] = v_\pi(s_t) - \mathbb{E}\left[R_{t+1} + \gamma v_\pi(S_{t+1}) | S_t = s_t\right] = 0. \tag{6.11}$$

Note 6.1. 可以看到 *TD error* 的定义和算法更新涉及到 t 与 $t+1$ 两个时间步，因此将此算法称为 *"temporal difference"*(时序差分)，反映了两个时间步间的差异.

Basic TD – Idea

Bellman expectation equation. 回顾 element-wise form 的 Bellman Equation 如下:

$$v_\pi(s) = \sum_{a \in \mathcal{A}(s)} \pi(a|s) \left[\sum_{r,s'} p(r, s'|s, a) r + \gamma \sum_{s' \in \mathcal{S}} p(s'|s, a) v_\pi(s') \right], \quad \forall s \in \mathcal{S}. \tag{6.12}$$

Bellman expectation equation 与其形式等价，只是使用了更简洁的数学期望的形式. 根据 state value 的定义:

$$v_\pi(s) = \mathbb{E}\left[R + \gamma G \mid S = s\right], \quad s \in \mathcal{S}, \tag{6.13}$$

其中 G 为 discounted return. 由于 $\mathbb{E}[G|S = s] = \sum_a \pi(a|s) \sum_{s'} p(s'|s, a) v_\pi(s') = \mathbb{E}[v_\pi(S')|S = s]$，因此:

$$v_\pi(s) = \mathbb{E}\left[R + \gamma v_\pi(S') \mid S = s\right], \quad s \in \mathcal{S}. \tag{6.14}$$

RM algorithm for solving the Bellman expectation equation (6.14). 首先定义求解的目标:

$$g(v(s)) = v(s) - v_\pi(s) = v(s) - \mathbb{E}\left[R + \gamma v_\pi(S') \mid S = s\right] \Rightarrow g(v(s)) = 0. \tag{6.15}$$

进而根据 R 和 S' 的采样 r, s' 得到有噪观测:

$$\tilde{g}(v(s)) = \underbrace{\left(v(s) - \mathbb{E}\left[R + \gamma v_\pi(S') \mid s\right]\right)}_{g(v(s))} + \underbrace{\left(\mathbb{E}\left[R + \gamma v_\pi(S') \mid s\right] - \left[r + \gamma v_\pi(s')\right]\right)}_{\eta}. \tag{6.16}$$

因此求解问题 $g(v(s)) = 0$ 的 RM 算法为:

$$v_{t+1}(s) = v_t(s) - \alpha_t \tilde{g}(v_t(s)) = v_t(s) - \alpha_t \left(v_t(s) - \left[r_t + \gamma v_\pi(s'_t)\right]\right), \tag{6.17}$$

其中 $v_t(s)$ 为 $v_\pi(s)$ 在第 t 步的估计，r_t, s'_t 分别为 R, S' 在第 t 步的采样.

Note 6.2. ① 式 *(6.17)* 仅对一个 *state* s 进行估计，在 *TD learning* 中使用序列 $\{s_t, r_{t+1}, s_{t+1}\}$ 作为数据，因此在当前时间步 t 没有涉及到的 *state* 就保持不变 *(*如式*6.5b)*. ② 算法*(6.17)*中使用了我们估计的目标 v_π 进行迭代，在 *TD learning* 中由于 v_π 未知，因此需要使用 $v_t(s')$ 替换 $v_\pi(s'_t)$. 此时依然有收敛性，详见 ***Theorem ??***.

Note 6.3. ① *TD learning* 使用 *RM* 算法求解策略 π 的 *Bellman expectation equation*，但将估计目标 v_π 替换为当前状态下的估计 $v_t(s'_t)$. ② *TD learning* 的基本思想是根据新获得的信息纠正当前对状态值的估计，从而使 $v_t \to v_\pi$.

TD Learning of state values: Convergence analysis

Theorem 6.1 (Convergence of Basic TD learning)**.** 给定 *policy* π，使用 *Basic TD algorithm (6.5a)(6.5b)*时，若满足如下条件则 $\forall s \in \mathcal{S}$，$v_t(s) \xrightarrow[t\to\infty]{w.p.1.} v_\pi(s)$*:*

① $\sum_t \alpha_t(s) = \infty$ *(*每一个状态 s 都会被访问足够多/无穷次，因为未被访问时$\alpha_t(s) = 0$*)*

② $\sum_t \alpha_t^2(s) < \infty, \forall s \in \mathcal{S}$ *(*意味着$\alpha_t \to 0$，实际中设为随着 t 最后变得很小的数*)*

Note 6.4. 事实上当 α 为常数时，仍然可以证明算法在期望意义上收敛. ***Theorem 6.1*** 的证明详见附录**??**，其需要使用 *Lecture 5* 中 *Dvoretzky's theorem* 的推广版本.

6.2 Sarsa: TD Learning of Action Values

TD Learning of action values – Sarsa

Sarsa 算法的目标是估计给定 policy π 的 action value $q_\pi(s,a)$，因此与估计 state value 的 Basic TD 需要的数据 $\{(s_t, r_t, s_{t+1})\}_t$ 不同，其需要的数据是 $\{(s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1})\}_t$.

根据 action value 的定义，有:

$$q_\pi(s,a) = \sum_r p(r|s,a)r + \gamma \sum_{s'} p(s'|s,a)v_\pi(s') = \mathbb{E}\left[R + \gamma q_\pi(S', A')|s,a\right], \quad \forall s, a. \tag{6.18}$$

因此与 Basic TD 一样，定义求解目标为:

$$g(q(s)) = q(s) - q_\pi(s) = v(s) - \mathbb{E}\left[R + \gamma q_\pi(S', A')|s,a\right] \Rightarrow g(q(s)) = 0. \tag{6.19}$$

再应用 RM 算法并将未知的 $q_\pi(s', a')$ 替换成下一时间步的观测 $q_t(s_{t+1}, a_{t+1})$ 可得 **Sarsa** 算法为:

$$q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha_t(s_t, a_t)\left[q_t(s_t, a_t) - [r_{t+1} + \gamma q_t(s_{t+1}, a_{t+1})]\right], \tag{6.20a}$$

$$q_{t+1}(s,a) = q_t(s,a), \quad \forall (s,a) \neq (s_t, a_t), t = 0, 1, 2, \cdots \tag{6.20b}$$

其中 $q_t(s_t, a_t)$ 是对 $q_\pi(s_t, a_t)$ 的估计，α_t 为 learning rate.

TD Target. Sarsa 算法的 TD target 为:

$$\bar{q}_t \triangleq r_{t+1} + \gamma q_t(s_{t+1}, a_{t+1}). \tag{6.21}$$

TD Error. Sarsa 算法的 TD error 为估计值 q_t 与真实状态值 q_π 之间的差异，其定义为:

$$\delta_t \triangleq v(s_t) - [r_{t+1} + \gamma v(s_{t+1})] = v(s_t) - \bar{v}_t. \tag{6.22}$$

Note 6.5. ① *Sarsa* 使用的数据为 $\{(s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1})\}_t$，首字母的缩写即构成了 *s-a-r-s-a* 这个命名. ② 类似于 *TD learning*，*Sarsa* 在求解 *action value* 版本的 *Bellman equation (6.18)*，证明其为 *BE* 可详见附录**??**.

Theorem 6.2 (Convergence of Sarsa)**.** 给定 *policy* π，若满足如下条件则 $\forall s \in \mathcal{S}, a \in \mathcal{A}$，$q_t(s,a) \xrightarrow[t\to\infty]{w.p.1.} q_\pi(s,a)$*:*

① $\sum_t \alpha_t(s,a) = \infty$ *(*每对 (s,a) 都会被访问很多/无穷次，因为未被访问时$\alpha_t(s,a) = 0$*)*

② $\sum_t \alpha_t^2(s,a) < \infty, \forall s \in \mathcal{S}, a \in \mathcal{A}$ *(*意味着$\alpha_t \to 0$，实际中设置为随 t 逐渐变足够小*)*

Note 6.6. 收敛性的证明与 *TD learning* 中 ***Theorem ??***类似，因此暂略.

Sarsa – Algorithm

Sarsa 只能通过迭代得到给定 policy π 的 action value，其可用于 Policy Iteration 的 policy evaluation 部分计算 state value，后续仍需结合 policy improvement 才能得到 optimal policy. 事实上 Sarsa 常常指代结合后完整的算法，即将 Policy Iteration 中的 policy evaluation 使用 Sarsa 计算 state value，后续继续结合 policy improvement.

Algorithm 7 Optimal policy learning by Sarsa

1: **Initialization:** $\alpha_t(s,a) = \alpha > 0$ for all (s,a) and all t. $\epsilon \in (0,1)$. Initial $q_0(s,a)$ for all (s,a). Initial ϵ-greedy policy π_0 derived from q_0.
2: **Goal:** Learn an optimal policy that can lead the agent to the target state from an initial state s_0.
3: **for** each episode **do**
4: **if** the current s_t is not the target state **then**
5: Collect the experience $(s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1})$: In particular, take action a_t following $\pi_t(s_t)$, generate r_{t+1}, s_{t+1}, and then take action a_{t+1} following $\pi_t(s_{t+1})$.
6: Update q-value**(policy evaluation):**

$$q_{t+1}(s_t,a_t) = q_t(s_t,a_t) - \alpha_t(s_t,a_t)\left[q_t(s_t,a_t) - (r_{t+1} + \gamma q_t(s_{t+1},a_{t+1}))\right]$$

7: Update policy**(policy improvement):**

$$\pi_{t+1}(a \mid s_t) = \begin{cases} 1 - \epsilon + \frac{\epsilon}{|A|} & \text{if } a = \arg\max_a q_{t+1}(s_t,a) \\ \frac{\epsilon}{|A|} & \text{otherwise} \end{cases}$$

8: **end if**
9: **end for**

Note 6.7. 注意在做 *policy evaluation* 时只做了一次就进行了 *policy improvement* 并继续迭代，并没有准确进行估计，因此是基于 *generalized policy iteration* 的想法.

6.3 Expected Sarsa: TD Learning of Action Values

TD Learning of action values: Expected Sarsa

在 Sarsa 中 TD target (6.21) 包含下一次观测的 action value $q_t(s_{t+1}, a_{t+1})$，然而一次观测的方差过大，为了降低方差可以进行多次采样，将 $r_{t+1} + \gamma q_t(s_{t+1}, a_{t+1})$ 变为 $r_{t+1} + \gamma \mathbb{E}[q_t(S_{t+1}, A)]$，就得到了 **Expected Sarsa** 算法:

$$q_{t+1}(s_t,a_t) = q_t(s_t,a_t) - \alpha_t(s_t,a_t)\left[q_t(s_t,a_t) - (r_{t+1} + \gamma \mathbb{E}[q_t(s_{t+1},A)])\right], \tag{6.23a}$$

$$q_{t+1}(s,a) = q_t(s,a), \quad \forall (s,a) \neq (s_t,a_t). \tag{6.23b}$$

其中 $\mathbb{E}[q_t(s_{t+1}, A)] = \sum_a \pi_t(a|s_{t+1}) q_t(s_{t+1}, a) \triangleq v_t(s_{t+1})$.

Note 6.8. *Expected Sarsa* 需要更大的计算量去估计 $\mathbb{E}[q_t(s_{t+1}, A)]$，因此相较于 *Sarsa* 计算量增大，但是由于使用的随机变量的个数变少 *(*由 S, A 变为 S*)*，因此估计方差变小.

Note 6.9. 与 *Basic TD* 和 *Sarsa* 一样，*Expected Sarsa* 可以解释为使用 *RM* 算法求解如下方程:

$$q_\pi(s,a) = \mathbb{E}\left[R_{t+1} + \gamma \mathbb{E}_{A_{t+1}\sim\pi(S_{t+1})}[q_\pi(S_{t+1},A_{t+1})] | S_t = s, A_t = a\right], \quad \forall s,a. \tag{6.24}$$

由于

$$\mathbb{E}\left[q_\pi\left(S_{t+1},A_{t+1}\right) \mid S_{t+1}\right] = \sum_{A'} q_\pi\left(S_{t+1},A'\right)\pi\left(A' \mid S_{t+1}\right) = v_\pi\left(S_{t+1}\right) \tag{6.25}$$

因此式*(6.24)*是如下 *Bellman equation* 的另一种展开式:

$$q_\pi(s,a) = \mathbb{E}\left[R_{t+1} + \gamma v_\pi(S_{t+1}) | S_t = s, A_t = a\right] \tag{6.26}$$

6.4 n-Step Sarsa: TD Learning of Action Values

TD Learning of action values: n-step Sarsa

Sarsa (6.20a)(6.20b) 使用的样本只需要相邻时间步的 state-action-reward pair $\{(s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1})\}_t$，而 Monte Carlo 则使用完整的 trajectory $(s_t, a_t, r_{t+1}, \dots)$ 进行估计. n-step Sarsa 是 Sarsa 的推广，介于二者之间.

根据定义，action value $q_\pi(s,a) = \mathbb{E}[G_t | S_t = s, A_t = a]$，其中 G_t 为 discounted return，可以被写为多种形式:

$$\begin{aligned} \text{Sarsa} \longleftarrow \quad & G_t^{(1)} = R_{t+1} + \gamma q_\pi(S_{t+1}, A_{t+1}), \\ & G_t^{(2)} = R_{t+1} + \gamma R_{t+2} + \gamma^2 q_\pi(S_{t+2}, A_{t+2}), \\ & \cdots \\ n\text{-step Sarsa} \longleftarrow \quad & G_t^{(n)} = R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^n q_\pi(S_{t+n}, A_{t+n}), \\ \text{MC} \longleftarrow \quad & G_t^{(\infty)} = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots \end{aligned} \tag{6.27}$$

Note 6.10. 所有含有上标的 $G_t^{(1)}, \cdots, G_t^{(\infty)}$ 都是等价的，不同的仅是如何分解. $G_t^{(\infty)}$ 中 ∞ 指达到 *episode* 终点.

得到 G_t 不同的分解后，将其代入 action value 定义式中，可以得到不同算法的求解目标:

$$\begin{aligned} \text{Sarsa to solve:} \quad & q_\pi(s,a) = \mathbb{E}[G_t^{(1)} | s, a] = \mathbb{E}[R_{t+1} + \gamma q_\pi(S_{t+1}, A_{t+1}) | s, a] \\ n\text{-step Sarsa:} \quad & q_\pi(s,a) = \mathbb{E}[G_t^{(n)} | s, a] = \mathbb{E}[R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^n q_\pi(S_{t+n}, A_{t+n}) | s, a] \\ \text{MC learning:} \quad & q_\pi(s,a) = \mathbb{E}[G_t^{(\infty)} | s, a] = \mathbb{E}[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots | s, a] \end{aligned} \tag{6.28}$$

相应的随机近似算法如下:

$$\begin{aligned} \text{Sarsa:} \quad & q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha_t(s_t, a_t) \left[q_t(s_t, a_t) - [r_{t+1} + \gamma q_t(s_{t+1}, a_{t+1})] \right] \\ n\text{-step Sarsa:} \quad & q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha_t(s_t, a_t) \left[q_t(s_t, a_t) - [r_{t+1} + \cdots + \gamma^n q_t(s_{t+n}, a_{t+n})] \right] \\ \text{MC learning:} \quad & q_{t+1}(s_t, a_t) = r_{t+1} + \gamma r_{t+2} + \cdots + \gamma^n r_{t+n} + \cdots \end{aligned} \tag{6.29}$$

从统一的角度来看，n-step Sarsa 的 target 是:

$$G_{t:t+n} = \sum_{k=0}^{n-1} \gamma^k R_{t+k+1} + \gamma^n Q_t(S_{t+n}, A_{t+n}) = R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{n-1} R_{t+n} + \gamma^n Q_t(S_{t+n}, A_{t+n}). \tag{6.30}$$

更新为:

$$Q_{t+1}(S_t, A_t) = Q_t(S_t, A_t) + \alpha_t \left[G_{t:t+n} - Q_t(S_t, A_t) \right] = \underbrace{R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{n-1} R_{t+n}}_{\text{真实采样得到的奖励}} + \underbrace{\gamma^n Q_t(S_{t+n}, A_{t+n})}_{\text{bootstrap 估计}}. \tag{6.31}$$

因此 n 决定了采样多少步真实奖励，然后在什么位置停止采样并使用当前 Q 值进行 bootstrap. 当 $n = 1$ 时，n-step Sarsa 退化为 one-step Sarsa；当 n 延伸至 episode 终点时，其 target 退化为完整 Monte Carlo return.

Bias Analysis. 真实的 action value 定义为:

$$q_\pi(s,a) = \mathbb{E}_\pi \left[\sum_{k=0}^{n-1} \gamma^k R_{t+k+1} + \gamma^n q_\pi(S_{t+n}, A_{t+n}) \mid S_t = s, A_t = a \right]. \tag{6.32}$$

因此 TD target 与 真实 action value 的差异为:

$$\begin{aligned} & \mathbb{E}_\pi[G_{t:t+n} \mid S_t = s, A_t = a] - q_\pi(s,a) = \gamma^n \mathbb{E}_\pi \left[Q_t(S_{t+n}, A_{t+n}) - q_\pi(S_{t+n}, A_{t+n}) \mid S_t = s, A_t = a \right], \\ & \Rightarrow |\mathbb{E}[G_{t:t+n} \mid s, a] - q_\pi(s,a)| \leqslant \gamma^n \| Q_t - q_\pi \|_\infty. \end{aligned} \tag{6.33}$$

因此当 n 较小时，较早使用不准确的 Q_t bootstrap 通常会造成偏差较大；当 n 较大时，bootstrap 误差被 γ^n 衰减，偏差通常较小.

Variance Analysis. 当 n 较小时，target 只包含少量随机奖励，虽然 bootstrap 值可能不准确，但它通常比一长串随机奖励更加稳定；当 n 较大时，target 包含更多随机变量，因此方差通常上升.

n-step Sarsa: Comparison

下面比较这三种算法的不同之处：① **TD target:** Sarsa 与 n-step Sarsa 的 TD target是不同的：

$$\begin{aligned} \text{Sarsa:} \quad & \bar{v}_t^{(1)} \triangleq r_{t+1} + \gamma v(s_{t+1}) \\ n\text{-step Sarsa:} \quad & \bar{v}_t^{(2)} \triangleq r_{t+1} + \gamma r_{t+2} + \cdots + \gamma^n q_t(s_{t+n}, a_{t+n}) \end{aligned} \tag{6.34}$$

② **Data needed:** 三种算法都是 model free 的算法，都需要数据，但所需数据不同：

$$\begin{aligned} \text{Sarsa:} \quad & (s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1}) \\ n\text{-step Sarsa:} \quad & (s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1}, \cdots, r_{t+n}, s_{t+n}, a_{t+n}) \\ \text{TD learning:} \quad & (s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1}, \cdots, r_{t+n}, s_{t+n}, a_{t+n}, \cdots) \end{aligned} \tag{6.35}$$

③ **Online vs. Offline:** Sarsa 为 online 的算法，可以使用当前时间节点的数据立即进行算法迭代，而 MC learning 为 offline 的算法，需要等待到足够多的数据才可以估算 action value. n-step Sarsa 则既不是 online 也不是 offline 的算法，或将其看作是特殊的 online 算法，因为其需要等到 $t+n$ 时刻才可以更新 (s_t, a_t) 的 action value：

$$q_{t+n}(s_t, a_t) = q_{t+n-1}(s_t, a_t) - \alpha_{t+n-1}(s_t, a_t)\left[q_{t+n-1}(s_t, a_t) - [r_{t+1} + \gamma r_{t+2} + \cdots + \gamma^n q_{t+n-1}(s_{t+n}, a_{t+n})]\right] \tag{6.36}$$

6.5 Q-Learning: TD Learning of Optimal Action Values

TD Learning of optimal action values: Q-learning

Sarsa 及其变体是在估计 action value，在 Policy Iteration 算法中还需要结合 policy evaluation 才能够进行 policy searching (如Algorithm 7所示). 下面介绍非常经典且常用[a]的算法 Q-learning，其直接估计 optimal action value，无需交替进行 policy evaluation 和 policy improvement. 为实现这一点，只需将 TD target 修改为 optimal:

$$\begin{aligned} \text{Q-learning:} \quad & r_{t+1} + \gamma \max_{a \in \mathcal{A}} q_t(s_{t+1}, a) \\ \text{Sarsa:} \quad & r_{t+1} + \gamma q_t(s_{t+1}, a_{t+1}) \end{aligned} \tag{6.37}$$

因此 Q-learning 算法如下：

$$q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha_t(s_t, a_t)\left[q_t(s_t, a_t) - [r_{t+1} + \gamma \max_{a \in \mathcal{A}} q_t(s_{t+1}, a)]\right] \tag{6.38a}$$

$$q_{t+1}(s, a) = q_t(s, a), \quad \forall (s, a) \neq (s_t, a_t) \tag{6.38b}$$

Note 6.11. 与 *Sarsa* 在求解一个 *action value* 的 *Bellman equation (6.18)* 不同，*Q-learning* 是在求解一个带有期望的带有 *action value* 形式的 *Bellman optimality equation(BOE) (6.39)*. 关于下式 *(6.39)* 是 *BOE* 的证明详见附录**??**

$$q(s, a) = \mathbb{E}\left[R_{t+1} + \gamma \max_a q(S_{t+1}, a) | S_t = s, A_t = a\right], \quad \forall s, a. \tag{6.39}$$

[a] 现在常用的算法是 deep Q-learning，是将 Q-learning 与深度学习结合.

Behavior Policy and Target Policy

Definition 6.1 (Two policies in TD learning)**.** *TD learning* 中存在两种 *policy*，分别为 ***behavior policy*** 和 ***target policy***. 其中 *behavior policy* 用于生成 *experience samples*，是智能体与环境的交互策略，可以包含探索成分；*target policy* 定义我们希望估计的 *value* $q_\pi(s, a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a]$.

Note 6.12. ① *Behavior policy* 通常记作 b，*agent* 真正与环境交互时 $A_t \sim b(\cdot \mid S_t)$，然后环境产生 $R_{t+1}, S_{t+1} \sim p(\cdot, \cdot \mid S_t, A_t)$. 因此 b 决定: *(1)* 实际执行哪个动作 *(2)* 哪些状态动作对经常被访问 *(3)* 数据集或 *trajectory* 的概率分布 *(4)* 探索程度是否充分. 例如 *ϵ-greedy behavior policy* $b(a \mid s) = \begin{cases} 1 - \epsilon + \dfrac{\epsilon}{|\mathcal{A}|}, & a \in \arg\max_{a'} Q(s, a'), \\ \dfrac{\epsilon}{|\mathcal{A}|}, & \textit{otherwise} \end{cases}$ 偶尔执行非 *greedy* 动作以保证探索. ② *Target policy* 通常记作 π，用于定义我们希望估计的 *value*，其不一定真的在环境中执行，可以只存在于更新公式中.

Off Policy and On Policy

基于 target policy 和 behavior policy 是否一致可以将算法分为 on-policy 和 off-policy 两类.

Definition 6.2 (on-policy and off-policy)**.** 当一种学习方法的 *behavior policy* 与 *target policy* 一致时，称之为 ***on-policy***，即智能体在学习过程中是通过执行自己的 *target policy* 来收集数据和更新策略；当 *behavior policy* 与 *target policy* 不一致时，称之为 ***off-policy***，智能体可以通过 *behavior policy* 收集数据，而通过 *target policy* 来进行学习.

Note 6.13. ① 使用 *off-policy* 的好处是其可以使用别的 *(*例如探索性更强的*) policy* 生成的 *experience samples* 去搜索 *optimal policy*，可以加速学习进程. ② 区分算法是 *off-policy* 还是 *on-policy* 主要通过 *(1)* 明确求解目标 *(*例如是何种 *BE/BOE) (2)* 检查算法需要什么数据.

Sarsa is on-policy. Sarsa 在每个 iteration 中，第 ① 步需要通过求解给定 policy π 的 Bellman Equation 进行 policy evaluation. 这一步 Sarsa 使用由 policy π 产生的数据估计 action value，因此 π 为 behavior policy. 第 ② 步根据 π 的估计值进行 policy improvement π 也是 target policy.

Sarsa 需要的 **data** 为 $\{(s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1})\}_t$，其产生的方式为:

$$s_t \xrightarrow{\pi_b} a_t \xrightarrow{\text{model}} r_{t+1}, s_{t+1} \xrightarrow{\pi_b} a_{t+1}. \tag{6.40}$$

可以看到数据是通过 behavior policy π_b 产生的，因此使用这些数据估计的 target 是:

$$Y_t^{\text{Sarsa}} = R_{t+1} + \gamma Q_t(S_{t+1}, A_{t+1}), \quad A_{t+1} \sim \pi_b(\cdot \mid S_{t+1}). \tag{6.41}$$

因此 Sarsa 实际求解的是 behavior policy π_b 对应的 action value 版本 Bellman Equation:

$$Q(s,a) = \mathbb{E}\left[R_{t+1} + \gamma \sum_{a'} \pi_b(a' \mid S_{t+1}) Q(S_{t+1}, a') \mid s, a\right]. \tag{6.42}$$

而 Sarsa 的目的是估计 target policy π 的 action value，其满足:

$$q_\pi(s,a) = \mathbb{E}\left[R_{t+1} + \gamma \sum_{a'} \pi(a' \mid S_{t+1}) q_\pi(S_{t+1}, a') \mid s, a\right]. \tag{6.43}$$

要通过采样实现这个期望，应当令 $A_{t+1} \sim \pi(\cdot \mid S_{t+1})$，而实际中使用的是 π_b，因此需要 $\pi_b = \pi$.

MC learning is on-policy. MC learning 的目标是求 action value 的期望:

$$q_\pi(s,a) = \mathbb{E}\left[R_{t+1} + \gamma R_{t+2} + \ldots | S_t = s, A_t = a\right], \quad \forall s, a. \tag{6.44}$$

在实际算法中，根据 behavior policy π_b 使用多个采样来近似估算 $q(s,a)$:

$$q(s,a) \approx r_{t+1} + \gamma r_{t+2} + \ldots \tag{6.45}$$

为使得 使用的数据所估算的 $q(s,a)$ 与目标 $q_\pi(s,a)$ 对齐，需要 $\pi_b = \pi$，因此 MC learning 为 on-policy.

Q-learning is off-policy. Q-learning 实际上在求解一个 BOE (6.39):

$$q(s,a) = \mathbb{E}\left[R_{t+1} + \gamma \max_a q(S_{t+1}, a) | S_t = s, A_t = a\right], \quad \forall s, a. \tag{6.46}$$

其 target 为:

$$Y_t^Q = R_{t+1} + \gamma \max_{a'} Q_t(S_{t+1}, a'). \tag{6.47}$$

其中 $\max_{a'} Q_t(S_{t+1}, a')$ 假设未来按照 greedy policy 行动，因此 π 为 greedy policy:

$$\pi(a \mid s) = \mathbf{1}\left\{a \in \arg\max_{a'} Q_t(s, a')\right\}. \tag{6.48}$$

Q-learning 需要的数据为 $(s_t, a_t, r_{t+1}, s_{t+1})$，其中若 (s_t, a_t) 给定，那么 r_{t+1} 和 s_{t+1} 依赖于 model 而不依赖于 policy. 因此产生数据时，通常使用探索性的 behavior policy: $b_t = \epsilon$-greedy. 因此 $b_t = \epsilon$-greedy $\neq \pi_t =$ greedy.

Note 6.14. 有一些方法可以将 *on-policy* 算法转换为 *off-policy* 算法，例如重要性采样 *(importance sampling).*

Online and Offline

容易与 on-policy/off-policy 混淆的两个概念是 online/offline. 在线学习 (online) 是指 agent 在与环境交互过程中更新 value 和 policy. 离线学习 (offline) 是指 agent 使用预先收集的数据更新 value 和 policy 而无需与环境交互.

因此若一个算法是 on-policy 的，那么它只能以 online 的模式进行，不能使用别的 policy 预先产生的数据；若算法是 off-policy 的，那么它 online / offline 的模式都可以使用.

Q-learning – Algorithm

由于 Q-learning 是 off-policy 的，其 behavior policy 与 target policy 可以不同也可以相同，分别能够得到如下 online version 与 offline version 两种版本的算法:

Algorithm 8 Policy Searching by Q-learning (on-policy version)

1: **for** each episode **do**
2: **if** the current s_t is not the target state **then**
3: Collect the experience $(s_t, a_t, r_{t+1}, s_{t+1})$: In particular, take action a_t following $\pi_t(s_t)$, generate r_{t+1}, s_{t+1}.
4: **Update q-value:**

$$q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha_t(s_t, a_t)\left[q_t(s_t, a_t) - \left(r_{t+1} + \gamma \max_a q_t(s_{t+1}, a)\right)\right]$$

5: **Update policy:**

$$\pi_{t+1}(a \mid s_t) = \begin{cases} 1 - \frac{\epsilon}{|A|-1} & \text{if } a = \arg\max_a q_{t+1}(s_t, a), \\ \frac{\epsilon}{|A|} & \text{otherwise.} \end{cases}$$

6: **end if**
7: **end for**

Algorithm 9 Optimal Policy Search by Q-learning (off-policy version)

1: **for** each episode $\{s_0, a_0, r_1, s_1, a_1, r_2, \dots\}$ generated by π_b **do** ▷ π_b is behavior policy
2: **for** each step $t = 0, 1, 2, \dots$ of the episode **do**
3: **Update q-value:**

$$q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha_t(s_t, a_t)\left[q_t(s_t, a_t) - \left(r_{t+1} + \gamma \max_a q_t(s_{t+1}, a)\right)\right]$$

4: **Update target policy:**

$$\pi_{T,t+1}(a \mid s_t) = \begin{cases} 1 & \text{if } a = \arg\max_a q_{t+1}(s_t, a), \\ 0 & \text{otherwise.} \end{cases}$$

▷ π_T is target policy

5: **end for**
6: **end for**

Note 6.15. 在 *off-policy version* 中直接使用 *greedy* 策略进行 *target policy* 的更新而无需具备探索性是因为此时数据由 *behavior policy* 生成，只需要 *behavior policy* 保持探索性，而 *target policy* 直接选择最好的即可；相反在 *on-policy version* 中，由于需要使用 *target policy* 生成数据，因此需要保持一定的探索性，故使用 *ϵ-greedy* 的更新模式.

6.6 Comparison and Conclusion

Comparison: TD Learning and MC Learning

Table 6.1: Comparison between TD/Sarsa learning and Monte Carlo learning

TD/Sarsa learning[1]	Monte Carlo learning
Online / incremental 每获得一步transition sample，便可立即更新state/action-value estimate，无需等待episode 结束.	**Episode-wise / delayed** 由于target 是完整的discounted return，通常要等episode采样结束后才能更新.
Episodic and continuing tasks 可自然用于episodic task，也适用于没有terminal state 的continuing task.	**Primarily episodic tasks** 经典MC 方法依赖完整return，因此主要用于具有terminal state 的episodic task.
Bootstrapping Update target 使用已有的value estimate，例如Sarsa 的$R_{t+1}+\gamma Q(S_{t+1},A_{t+1})$.	**No bootstrapping** Update target 直接采用实际采样得到的完整return G_t，不依赖后继状态的value estimate.
Lower variance 一步TD target 涉及的随机样本较少；例如Sarsa 只需采样R_{t+1}、S_{t+1} 和A_{t+1}，通常具有较低方差.	**Higher variance** MC target $G_t=R_{t+1}+\gamma R_{t+2}+\gamma^2 R_{t+3}+\cdots$ 受后续整条trajectory 的随机性影响，通常具有较高方差.
Possible finite-sample bias 由于target 依赖当前的近似估计，有限样本下可能有偏；在适当条件下可收敛到真实value.	**Unbiased return target** 对由policy π 生成的完整episode，G_t 是$v_\pi(s_t)$（或$q_\pi(s_t,a_t)$）的无偏样本.

[1] Sarsa 是典型的on-policy TD control 方法，因此此处与TD learning 一并比较.

A Unified Point of View for TD Learning

本节中所介绍的所有算法实际上都可以表达为一个统一的形式:

$$q_{t+1}(s_t,a_t)=q_t(s_t,a_t)-\alpha_t(s_t,a_t)[q_t(s_t,a_t)-\bar{q}_t], \tag{6.49}$$

其中 $\bar{q}_t$ 为 TD target. 所有的 TD 算法的不同点在于 TD target 和数学上的求解目标:

Table 6.2: Update targets used by different reinforcement learning algorithms

Algorithm	Update target $\bar{q}_t$
Sarsa	$\bar{q}_t=R_{t+1}+\gamma q_t(S_{t+1},A_{t+1})$
n-step Sarsa	$\bar{q}_t=\sum_{k=0}^{n-1}\gamma^k R_{t+k+1}+\gamma^n q_t(S_{t+n},A_{t+n})$
Expected Sarsa	$\bar{q}_t=R_{t+1}+\gamma\sum_a \pi_t(a\mid S_{t+1})q_t(S_{t+1},a)$
Q-learning	$\bar{q}_t=R_{t+1}+\gamma\max_a q_t(S_{t+1},a)$
Monte Carlo[1]	$\bar{q}_t=G_t=\sum_{k=0}^{T-t-1}\gamma^k R_{t+k+1}$

[1] 当$\alpha_t(S_t,A_t)=1$时，有$q_{t+1}(S_t,A_t)=\bar{q}_t$.

Table 6.3: Bellman equations solved by different reinforcement learning algorithms

Algorithm	Equation Solved
Sarsa	**BE:** $q_\pi(s,a)=\mathbb{E}_\pi[R_{t+1}+\gamma q_\pi(S_{t+1},A_{t+1})\mid S_t=s,A_t=a]$
n-step Sarsa	**BE:** $q_\pi(s,a)=\mathbb{E}_\pi\left[\sum_{k=0}^{n-1}\gamma^k R_{t+k+1}+\gamma^n q_\pi(S_{t+n},A_{t+n})\mid S_t=s,A_t=a\right]$
Expected Sarsa	**BE:** $q_\pi(s,a)=\mathbb{E}_\pi\left[R_{t+1}+\gamma\sum_{a'}\pi(a'\mid S_{t+1})q_\pi(S_{t+1},a')\mid S_t=s,A_t=a\right]$
Q-learning	**BOE:** $q_*(s,a)=\mathbb{E}\left[R_{t+1}+\gamma\max_{a'}q_*(S_{t+1},a')\mid S_t=s,A_t=a\right]$
Monte Carlo	**BE:** $q_\pi(s,a)=\mathbb{E}_\pi[G_t\mid S_t=s,A_t=a]$

7 Lecture 7: Value Function Approximation

Introduction

Introduction

在前面的 Lecture 中一个隐含的默认假设是 state value 与 action value 都是离散的方程，即对于每个离散的 s_i 和 (s_i, a_j) 有相对应的 state value $v_\pi(s_i)$ 和 action value $q_\pi(s_i, a_j)$. 因此比较适合用表格 (table) 的方式进行表示:

s_1	s_2	s_3	s_4	$\cdots$	s_9
$v_\pi(s_1)$	$v_\pi(s_2)$	$v_\pi(s_3)$	$v_\pi(s_4)$	$\cdots$	$v_\pi(s_9)$

Table 7.1: Table of state values

State / Action	a_1	a_2	a_3	a_4	a_5
s_1	$q_\pi(s_1, a_1)$	$q_\pi(s_1, a_2)$	$q_\pi(s_1, a_3)$	$q_\pi(s_1, a_4)$	$q_\pi(s_1, a_5)$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
s_9	$q_\pi(s_9, a_1)$	$q_\pi(s_9, a_2)$	$q_\pi(s_9, a_3)$	$q_\pi(s_9, a_4)$	$q_\pi(s_9, a_5)$

Table 7.2: Table of action values

使用表格形式表示数据的优点在于易于理解和分析，但缺点在于难以处理大型或连续的 state / action space:

1. **Difficulty 1: Storage for Discrete Space.** 对于离散空间，空间较大时离散值较多，因此需要大量存储空间.
2. **Difficulty 2: Storage for Continuous Space.** 对于连续空间，处理时需要进行(网格)离散化. 当网格稀疏时不能很好地近似原空间，当网格稠密即节点过多时又面临与 Difficulty 1 一样的存储问题.
3. **Difficulty 3: Generalization.** 使用表格进行更新只能逐点进行，单点操作不能够影响周围未被访问到的值.

7.1 Function Approximation

Example

使用表格表示数据需要的存储量一般与数据量 N 成正比 ($O(N)$)，一种解决方式是使用这些数据通过函数近似 (**function approximation**) 拟合 action / state value function，最终只需要存储相应函数的少量参数即可.

Example 7.1. 考虑一维 *state* $s_1, \ldots, s_{|\mathcal{S}|} \in \mathbb{R}$，相应的 *state value* 为 $v_\pi(s_1), \ldots, v_\pi(s_{|\mathcal{S}|})$，其中 $\mathcal{S}$ 为 *state space*, $|\mathcal{S}|$ 为所有的状态个数, π 为给定的 *policy*. 假设 $|\mathcal{S}|$ 非常大，最简单的方式是使用直线 (如图 7.1) 拟合这些 *state-state value pair*，即 $(s, v_\pi(s))$，这样只需要存储该直线的少量参数. 拟合后的 *state value function* 可以写成如下形式:

$$\hat{v}(s, w) = as + b = \underbrace{[s, 1]}_{\phi^T(s)} \underbrace{\begin{bmatrix} a \\ b \end{bmatrix}}_{w} = \phi^T(s) w \tag{7.1}$$

其中 $w \in \mathbb{R}^m$ 为 *parameter vector* (参数向量)，$\phi(s)$ 为 s 的 *feature vector* (特征向量).

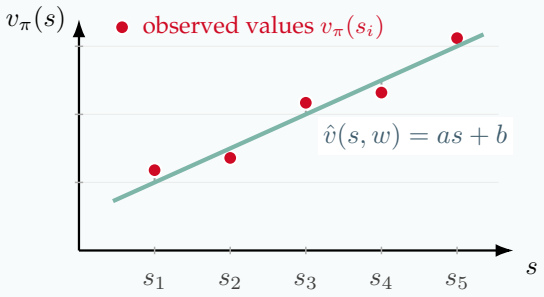


Figure 7.1: Linear approximation of a state-value function.

Curve Fitting

使用 function approximation 与 tabular method 的不同之处在于:

1. **Store:** tabular method 需要存储 $|\mathcal{S}|$ 个 state value，但 function approximation 只需要存储少量参数.
2. **Retrieve:** function approximation 中每当需要使用到 s 的 state value 时，需要重新计算 $\phi^T(s)w$.
3. **Accuracy:** 拟合/近似 (approximation) 会引入误差，因此我们牺牲了精度来提高存储效率.
4. **Update:** tabular method 逐点更新 state value 即可，而 function approximation 通过更新参数间接更新.
5. **Generalization ability:** function approximation 更新参数也会影响其他 state 的 state value. 因此只使用一个 state 的 experience sample 进行更新也可以帮助估计其他一些尚未访问 state 的 state value.

Note 7.1. 如果特征设计不好，由于 *function approximation* 有 *generalization ability*，其它状态可能错误地一起改变.

由于不同函数的表达能力不同，因此进行拟合前一个关键问题是选择什么样的拟合函数 $\hat{v}(s,w)$，主要有两种方式:

1. **Linear function:** $\hat{v}(s,w)=\phi(s)^Tw$，其中 $\phi(s)$ 为 feature vector，可以选取为多项式基底、Fourier 基底等. 更复杂的基底可以减小误差但相应地也增加了参数存储量，例如使用二阶曲线拟合:

$$\hat{v}(s,w)=as^2+bs+c=\underbrace{[s^2,s,1]}_{\phi^T(s)}\underbrace{\begin{bmatrix}a\\b\\c\end{bmatrix}}_{w}=\phi^T(s)w. \tag{7.2}$$

2. **Neural networks:** 特征向量 $\phi(\cdot)$ 的选择往往需要先验知识. 当没有先验知识时一个被广泛采用的解决方案是使用参数 θ 参数化的神经网络作为非线性函数逼近器输入神经网络的为 state，输出为非线性的 $\hat{v}(s,\theta)$.

Note 7.2. 在 *Linear function* $\hat{v}(s,w)=\phi(s)^Tw$ 中虽然 $\hat{v}(s,w)$ 对于 s 不一定是线性的，例如二阶曲线拟合 *(7.2)*. 但是对于参数 w 来说仍是线性的.

Tabular Method is One-hot Linear Function Approximation

虽然使用线性函数进行拟合的拟合能力不如选用非线性函数进行拟合，但线性情形下的理论分析较非线性情形时更简单、可以帮助理解非线性情形. 事实上 tabular method 实际上是线性函数逼近的一个特殊形式.

令 feature vector 为单位向量:

$$\phi(s)=e_s\in\mathbb{R}^{|S|}, \tag{7.3}$$

其中 e_s 是对应状态的 one-hot 向量，只有第 s 个元素为1，其余为0. 那么:

$$\hat{V}(s,w)=e_s^\top w=w(s). \tag{7.4}$$

其中 $w(s)$ 为 w 的第 s 个元素，就是表格中状态 s 对应的值. 因此:

Tabular Method = One-hot Linear Function Approximation.

7.2 Function Approximation based TD Learning

Introduction

现在，我们已经不再使用表格将所有的 state-state value pair 进行存储，而是基于 function approximation 方法使用参数化的 $\hat{v}(\cdot)$ 逼近 state value function $v_\pi(\cdot)$. 因此我们需要找到最优的参数向量 w 使 $\hat{v}$ 能够较好地近似 v_π.

这会产生两个重要问题:

① 如何度量 $\hat{v}(\cdot)$ 的好坏，或者说使用什么量作为标准去衡量 $\hat{v}(\cdot)$ 对 $v_\pi(\cdot)$ 的逼近程度.

② 如果有了该度量，那么应该朝着什么方向"努力"才能缩小 $\hat{v}(\cdot)$ 与 $v_\pi(\cdot)$ 的差异，以及如何设计相应的算法.

Objective Function

Objective Function. 度量 $\hat{v}(\cdot)$ 的好坏或 $\hat{v}(\cdot)$ 对 $v_\pi(\cdot)$ 的逼近程度一般使用目标函数 (objective function). 这里我们可以将目标函数定义为对于所有 state s，$\hat{v}(s)$ 与 $\hat{v}(x)$ 二者数值差异平方和的期望:

$$J(w) = \mathbb{E}[(v_\pi(S) - \hat{v}(S,w))^2]. \tag{7.5}$$

由于 $J(w)$ 变小意味着 $v_\pi(s)$ 与 $\hat{v}(s,w)$ 的差距变小，因此我们的目标是最小化 $J(w)$ 以找到 最优的 (optimal) w.

Note 7.3. 在深度学习中，目标函数也常被称为损失函数 *(loss function)*.

注意这里的 S 是一个随机变量，取期望时需要选择一定的分布进行计算. 分布的选择主要有如下两种方式:

① **Uniform distribution.** 即取到每个 state 的概率是一样的，都是 $1/|\mathcal{S}|$，此时目标函数实际可写为:

$$J(w) = \mathbb{E}[(v_\pi(S) - \hat{v}(S,w))^2] = \frac{1}{|\mathcal{S}|}\sum_{s\in\mathcal{S}}(v_\pi(s) - \hat{v}(s,w))^2. \tag{7.6}$$

平均分布假设所有的 state 同等重要，但实际上接近目标的状态应该是更重要的，需要分配更多的权重. 同时有一些 state 很少被访问，它们应该被分配更少的权重.

② **Stationary / steady-state / limiting distribution.** 描述了马尔可夫决策过程的长期行为 (long-run behavior)，即在给定 policy 下不断环境交互，达到稳定状态后每个 state 被访问概率的分布，将其记为 $\{d_\pi(s)\}_{s\in\mathcal{S}}$，其中 $d_\pi(s) \geqslant 0, \sum_{s\in\mathcal{S}} d_\pi(s) = 1$. 这样目标函数实际可写为:

$$J(w) = \mathbb{E}[(v_\pi(S) - \hat{v}(S,w))^2] = \sum_{s\in\mathcal{S}} d_\pi(s)(v_\pi(s) - \hat{v}(s,w))^2. \tag{7.7}$$

然而分布 $d_\pi(s)$ 的获取并不简单，一种方式是通过仿真实验近似:

$$d_\pi(s) \approx \frac{n_\pi(s)}{\sum_{s'\in\mathcal{S}} n_\pi(s')}. \tag{7.8}$$

实际上不需要进行仿真实验也可以根据 state 概率转移矩阵 P_π得到 $d_\pi(s)$ 的理论值，详细分析见附录 **??**.

Optimization: Gradient Descent

为最小化目标函数 $J(w)$，可以使用梯度下降法 (gradient descent, GD):

$$w_{k+1} = w_k - \alpha_k \nabla_w J(w_k), \tag{7.9}$$

其中梯度为:

$$\nabla_w J(w) = \nabla_w \mathbb{E}[(v_\pi(S) - \hat{v}(S,w))^2] = \mathbb{E}[\nabla_w (v_\pi(S) - \hat{v}(S,w))^2] = -2\mathbb{E}[(v_\pi(S) - \hat{v}(S,w))\nabla_w \hat{v}(S,w)] \tag{7.10}$$

由此得到相应的梯度下降 (GD) 和随机梯度下降 (SGD) 算法分别为:

$$\begin{aligned} &\text{GD:} \quad w_{k+1} = w_k + 2\alpha_k \mathbb{E}[(v_\pi(S) - \hat{v}(S,w_k))\nabla_w \hat{v}(S,w_k)], \\ &\text{SGD:} \quad w_{t+1} = w_t + 2\alpha_t \left(v_\pi(s_t) - \hat{v}(s_t,w_t)\right) \nabla_w \hat{v}(s_t,w_t). \end{aligned} \tag{7.11}$$

可以看到在进行梯度下降时梯度中已经包含了需要求解的目标 v_π，因此需要将其替换，常见的方法有两种:

1. **Monte Carlo:** 使用从 s_t 出发的一个 episode 的 discounted return g_t 代替 $v_\pi(s_t)$:

$$w_{t+1} = w_t + \alpha_t(g_t - \hat{v}(s_t,w_t))\nabla_w \hat{v}(s_t,w_t).$$

2. **TD learning:** 使用 $r_{t+1} + \gamma\hat{v}(s_{t+1},w_t)$ 代替 $v_\pi(s_t)$:

$$w_{t+1} = w_t + \alpha_t \left[r_{t+1} + \gamma\hat{v}(s_{t+1},w_t) - \hat{v}(s_t,w_t)\right] \nabla_w \hat{v}(s_t,w_t). \tag{7.12}$$

Note 7.4. 由于 α_t 可以自己设定，因此这里将原先 $2\alpha_t$ 的系数 2 都放入 α_t 中不写.

TD Learning with Function Approximation

当使用 $r_{t+1}+\gamma\hat{v}(s_{t+1},w_t)$ 代替 $v_\pi(s_t)$ 并使用线性函数 $\phi^T(s)w$ 逼近 $\hat{v}(s,w)$ 时，可以得到 TD-Linear 算法:

$$\text{TD linear: } w_{t+1}=w_t+\alpha_t\left[r_{t+1}+\gamma\phi^T(s_{t+1})w_t-\phi^T(s_t)w_t\right]\phi(s_t). \tag{7.13}$$

Algorithm 10 TD Learning with Function Approximation

1: **Initialization:** A function $\hat{v}(s,w)$ that is differentiable in w. Initial parameter w_0.
2: **Aim:** Approximate the true state values of a given policy π.
3: **for** each episode generated following the policy π **do**
4: **for** each step (s_t,r_{t+1},s_{t+1}) **do**
5: **In the general case:**

$$w_{t+1}=w_t+\alpha_t\left[r_{t+1}+\gamma\hat{v}(s_{t+1},w_t)-\hat{v}(s_t,w_t)\right]\nabla_w\hat{v}(s_t,w_t)$$

6: **In the linear case(TD linear):**

$$w_{t+1}=w_t+\alpha_t\left[r_{t+1}+\gamma\phi^T(s_{t+1})w_t-\phi^T(s_t)w_t\right]\phi(s_t)$$

7: **end for**
8: **end for**

7.3 Theoretical Analysis for TD-Linear

Convergence Analysis

由于非线性 approximation 的分析较为复杂，因此本部分仅考虑线性情形，即 TD-Linear (7.13).
在分析 TD learning 算法(7.13)之前，首先分析其确定性形式 (即 GD 算法):

$$w_{t+1}=w_t+\alpha_t\mathbb{E}\left[\left(r_{t+1}+\gamma\phi^T(s_{t+1})w_t-\phi^T(s_t)w_t\right)\phi(s_t)\right], \tag{7.14}$$

其中 s_t 的分布为 stationary distribution d_π. 分析确定性形式基于以下两点考虑:

1. 确定性算法的收敛性更容易 (尽管不是平凡的) 分析.
2. 确定性算法 (7.14) 的收敛性蕴含随机 TD 算法(7.13) 的收敛性.

为方便分析，首先定义如下两个矩阵:

$$\Phi=\begin{bmatrix}\vdots\\ \phi^T(s)\\ \vdots\end{bmatrix}\in\mathbb{R}^{n\times m},\quad D=\begin{bmatrix}\ddots & & \\ & d_\pi(s) & \\ & & \ddots\end{bmatrix}\in\mathbb{R}^{n\times n}, \tag{7.15}$$

其中 Φ 为特征向量矩阵，包含了 n 个 state 的特征向量 $\phi(s)\in\mathbb{R}^m$，D 为对角阵，元素 d_{ii} 为 state s_i 的 stationary distribution $d_\pi(s)$.

Lemma 7.1. 式*(7.14)*中的期望可以写为

$$\mathbb{E}\left[\left(r_{t+1}+\gamma\phi^T\left(s_{t+1}\right)w_t-\phi^T\left(s_t\right)w_t\right)\phi\left(s_t\right)\right]=b-Aw_t, \tag{7.16}$$

其中 $A\triangleq\Phi^TD\left(I-\gamma P_\pi\right)\Phi\in\mathbb{R}^{m\times m}$, $b\triangleq\Phi^TDr_\pi\in\mathbb{R}^m$.

Lemma 7.1 的证明可见附录**??**. 这样算法(7.14) 就可以写为:

$$w_{t+1}=w_t+\alpha_t(b-Aw_t). \tag{7.17}$$

Convergence target: 在分析收敛性之前我们首先需要找到收敛的目标，因此首先假设 $\exists\, w^*$，使得 $w_t\xrightarrow{t\to\infty}w^*$，因此 $w^*=w^*+\alpha_\infty(b-Aw^*)$，从而得到

$$w^*=A^{-1}b. \tag{7.18}$$

为保证式 (7.18)是有意义的，有几点说明：

1. A 可逆. 事实上 A 不仅可逆 (invertible)，而且是正定 (positive definite)的，即 $x^TAx > 0, \forall x \in \mathbb{R}^{\dim A}, x \neq \mathbf{0}$. 证明详见附录**??**.
2. $w^* = A^{-1}b$ 实际上是最小化投影贝尔曼误差 (projected Bellman error) 的最优解，详细说明见下一小节.
3. 在 tabular method 情形下，当 dim $w = n = |\mathcal{S}|$，$\phi(s) = [0, \dots, 1, \dots, 0]^T$时，有

$$w^* = A^{-1}b = v_\pi. \tag{7.19}$$

说明要学习的参数向量实际上是真实 state value，与表格 TD算法是TD-Linear算法的特例这一事实是一致的. 式 (7.19)的证明见附录**??**.

Convergence analysis: 在假设收敛的前提下得到收敛的目标 w^* 后，可以证明得到

$$w_t \xrightarrow{t\to\infty} w^* = A^{-1}b. \tag{7.20}$$

证明详见附录**??**.

Projected Bellman Error

事实上 TD-Linear 算法收敛的解 (7.19) 并不是 (7.5) 最小化的解，而是使投影贝尔曼误差 (**projected Bellman error**) 最小化的最优解，原因在于 ① 真实目标 v_π 未知 ② function approximation 带来的差异. 下面进行分析:

Objective Function 1: True value error. 目标函数构造的实际目标是使得 $\hat{v}$ 与真实的 v_π 尽可能接近:

$$J_E(w) = \mathbb{E}[(v_\pi(S) - \hat{v}(S, w))^2] = \|\hat{v}(w) - v_\pi\|_D^2, \tag{7.21}$$

其中 D 为 S 对应的分布，$\|x\|_D^2 = x^TDx = \|D^{1/2}x\|_2^2$.

Objective function 2: Bellman error. 由于 v_π 满足 Bellman equation $v_\pi = r_\pi + \gamma P_\pi v_\pi$，同时由于 v_π 未知，需要使用 $\hat{v}$ 替换. 所以期待 $\hat{v}(w)$ 尽可能满足方程 $\hat{v}(w) = r_\pi + \gamma P_\pi \hat{v}(w)$，代入式 (7.5) 中，得到:

$$J_{BE}(w) = \|\hat{v}(w) - (r_\pi + \gamma P_\pi \hat{v}(w))\|_D^2 \triangleq \|\hat{v}(w) - T_\pi(\hat{v}(w))\|_D^2 \tag{7.22}$$

其中 $T_\pi(x) \triangleq r_\pi + \gamma P_\pi x$，称为 Bellman operator. 最小化 Bellman error 是一个标准的最小二乘问题.

Objective function 3: Projected Bellman error. 由于使用的逼近函数可能出于函数结构原因无法完全满足 $\hat{v}(w) = r_\pi + \gamma P_\pi \hat{v}(w)$，因此可以使用投影矩阵 M 将 $T_\pi(\hat{v}(w))$ 投影到 $\hat{v}$ 的空间上使目标函数可以为 0:

$$J_{PBE}(w) = \|\hat{v}(w) - MT_\pi(\hat{v}(w))\|_D^2, \tag{7.23}$$

其中 M 为正交投影矩阵 (orthogonal projection matrix).

事实上approximation-based TD Learning 算法 (7.12) 的目的是最小化 projected Bellman error J_{PBE} 而非 J_E 和 J_{BE}. 此处仅考虑线性情形 $\hat{v}(w) = \Phi w$，Φ 的 range space 为所有可能的线性近似的集合. 此时正交投影矩阵 M 为:

$$M = \Phi(\Phi^TD\Phi)^{-1}\Phi^TD \in \mathbb{R}^{n\times n}. \tag{7.24}$$

由于 $\hat{v}(w)$ 和 $MT_\pi(\hat{v}(w))$ 均在 Φ 的 range space 中，因此能够找到 w 使得 $\min J_{PBE} = 0$. 同时有 (证明见附录**??**):

Theorem 7.1. 在线性情形 $\hat{v}(w^*) = \Phi w^*$ 下，估计值 $\hat{v}$ 与真实值 v_π 满足

$$\|\hat{v}(w^*) - v_\pi\|_D = \|\Phi w^* - v_\pi\|_D \leqslant \frac{1}{1-\gamma}\min_w \|\hat{v}(w) - v_\pi\|_D = \frac{1}{1-\gamma}\min_w \sqrt{J_E(w)}. \tag{7.25}$$

其中 γ 为 *discounted rate*.

Note 7.5. 虽然 *Theorem 7.1* 说明估计值与真实值差异的上界被 J_E 控制，但是实际这个 *upper bound* 很松，尤其是当 γ 很接近 *1* 时，因此主要具有理论意义.

7.4 Function Approximation based Sarsa

Function Approximation based Sarsa

Sarsa 与之前的算法唯一的不同就在于是在使用 $\hat{q}(s,a,w)$ 估计 action value $q(s,a)$ 而不是使用 $\hat{v}(s,w)$ 来估计 state value $v(s)$，因此非常自然地可以得到基于 function approximation 的 Sarsa 算法:

$$w_{t+1} = w_t + \alpha_t \left[r_{t+1} + \gamma \hat{q}(s_{t+1}, a_{t+1}, w_t) - \hat{q}(s_t, a_t, w_t) \right] \nabla_w \hat{q}(s_t, a_t, w_t). \tag{7.26}$$

式(7.26)实际上仍然在做 policy evaluation，将其与 policy improvement 结合就得到完整的 Sarsa 算法:

Algorithm 11 Function Approximation based Sarsa

1: **Aim:** Search a policy that can lead the agent to the target from an initial state-action pair (s_0, a_0).
2: **for** each episode **do**
3: **if** the current s_t is not the target state **then**
4: Take action a_t following $\pi_t(s_t)$, generate r_{t+1}, s_{t+1}, and then take action a_{t+1} following $\pi_t(s_{t+1})$
5: **Value update (parameter update):**

$$w_{t+1} = w_t + \alpha_t \left[r_{t+1} + \gamma \hat{q}(s_{t+1}, a_{t+1}, w_t) - \hat{q}(s_t, a_t, w_t) \right] \nabla_w \hat{q}(s_t, a_t, w_t)$$

6: **Policy update:**

$$\pi_{t+1}(a \mid s_t) = \begin{cases} 1 - \frac{\epsilon}{|A(s)|-1} & \text{if } a = \arg\max_a \hat{q}(s_t, a, w_{t+1}) \\ \frac{\epsilon}{|A(s)|} & \text{otherwise.} \end{cases}$$

7: **end if**
8: **end for**

7.5 Deep Q-Learning

Deep Q-Learning / Deep Q-Network

只需要将 Sarsa 中的 $\hat{q}(s_{t+1}, a_{t+1}, w_t)$ 替换为 $\max_{a \in \mathcal{A}(s_{t+1})} \hat{q}(s_{t+1}, a, w_t)$ 即可得到 approximation based Q-learning:

$$w_{t+1} = w_t + \alpha_t \left[r_{t+1} + \gamma \max_{a \in \mathcal{A}(s_{t+1})} \hat{q}(s_{t+1}, a, w_t) - \hat{q}(s_t, a_t, w_t) \right] \nabla_w \hat{q}(s_t, a_t, w_t) \tag{7.27}$$

Algorithm 12 Q-learning with Function Approximation (on-policy version)

1: **Initialization:** Initial parameter vector w_0. Initial policy π_0. Small $\epsilon > 0$.
2: **Aim:** Search a good policy that can lead the agent to the target from an initial state-action pair (s_0, a_0).
3: **for** each episode **do**
4: **if** the current s_t is not the target state **then**
5: Take action a_t following $\pi_t(s_t)$, and generate r_{t+1}, s_{t+1}.
6: **Value update (parameter update):**

$$w_{t+1} = w_t + \alpha_t \left[r_{t+1} + \gamma \max_{a \in A(s_{t+1})} \hat{q}(s_{t+1}, a, w_t) - \hat{q}(s_t, a_t, w_t) \right] \nabla_w \hat{q}(s_t, a_t, w_t)$$

7: **Policy update:** ▷ Since this is on-policy, so exploration needed

$$\pi_{t+1}(a \mid s_t) = \begin{cases} 1 - \frac{\epsilon}{|A(s_t)|-1} & \text{if } a = \arg\max_{a' \in A(s_t)} \hat{q}(s_t, a', w_{t+1}) \\ \frac{\epsilon}{|A(s_t)|} & \text{otherwise.} \end{cases}$$

8: **end if**
9: **end for**

Deep Q-Learning – Basic Idea

Deep Q-learning 也被称为Deep Q-network(DQN)，它是最早也是最成功的将深度学习引入强化学习的算法[a]. 其利用深度学习强大的函数逼近能力将深度学习作为非线性函数逼近器 (nonlinear function approximator) 拟合 q_π. 但有别于式 (7.27) 的是：由于神经网络强大的表达能力，我们不需要再底层地显式计算 $\hat{q}$.

Objective / Loss Function. DQN 目的是最小化如下目标函数/损失函数:

$$J(w) = \mathbb{E}\left[\left(R + \gamma \max_{a\in\mathcal{A}(S')} \hat{q}(S', a, w) - \hat{q}(S, A, w)\right)^2\right] \tag{7.28}$$

其中 (S, A, R, S') 为随机变量. 此 loss function 为 Bellman optimality error，因为根据 BOE 定义:

$$q(s, a) = \mathbb{E}\left[R_{t+1} + \gamma \max_{a\in\mathcal{A}(S_{t+1})} q(S_{t+1}, a)|S_t = s, A_t = a\right], \quad \forall s, a. \tag{7.29}$$

因此在期望意义下 $R + \gamma \max_{a\in\mathcal{A}(S')} \hat{q}(S', a, w) - \hat{q}(S, A, w)$ 应当为0.

Optimization. 最小化损失函数最直接的想法就是使用梯度下降 (GD). 在损失函数式(7.28)中参数 w 不仅存在于 $\hat{q}(S, A, w)$ 中，还存在于不好计算梯度的 $\gamma \max_{a\in\mathcal{A}(S')} \hat{q}(S', a, w)$ 中，因此此处将前半部分视作相对于 w 固定的整体，无需计算梯度，即定义:

$$y \triangleq R + \gamma \max_{a\in\mathcal{A}(S')} \hat{q}(S', a, w). \tag{7.30}$$

DQN Technique 1: Two Networks. Deep Q-learning 实际引入了两个不同的网络，其中一个网络即 $\hat{q}(s, a, w)$，用于逼近 q_π；第二个网络为了表示 $\hat{q}(s, a, w_T)$，将 $\hat{q}(s, a, w_T)$ 中的参数 w 固定方便优化时只针对第一个网络:

1. **main network:** $\hat{q}(s, a, w)$，用于逼近 q_π
2. **target network:** 用于表示 $\hat{q}(s, a, w_T)$，每隔一段时间就将 main network 的参数 w 赋值给 w_T，并持续一段时间内将 $\hat{q}(s, a, w_T)$ 固定、无需进行梯度下降，只针对 main network 进行优化.

这样损失函数就变为了:

$$J = \mathbb{E}\left[\left(R + \gamma \max_{a\in\mathcal{A}(S')} \hat{q}(S', a, w_T) - \hat{q}(S, A, w)\right)^2\right] \tag{7.31}$$

整个算法和训练的流程即:

$$\text{将 } w \text{ 赋值给 } w_T \to \text{固定 } w_T \text{ 只更新 } \hat{q}(S, A, w) \text{ 中的 } w \to \text{将 } w \text{ 赋值给 } w_T$$

DQN Technique 2: Experience Replay. 训练数据实际是智能体与环境交互产生的轨迹 (trajectory) 中得到的，例如一个 trajectory $S_t \to S_{t+1} \to S_{t+2}$ 可以得到有先后顺序的数据:

$$(S_0, A_0, R_1, S_1), (S_1, A_1, R_2, S_2), (S_2, A_2, R_3, S_3), \ldots \tag{7.32}$$

然而这些有先后顺序的数据具有强相关性，并且可能非常相似，因此这个 batch 计算出的梯度主要反映了某一段连续轨迹而不是整个经验分布，会导致梯度估计方差较大和学习不稳定. 因此实际中使用 experience replay，即数据打散后归入一个集合 $\mathcal{B} \triangleq \{(s, a, r, s')\}$，称之为 replay buffer，然后使用均匀分布进行随机抽取.

Note 7.6. *Tabular Q-learning* 不需要 *experience replay*，因为在表格形式中根本没有使用到分布的需要，是在不断求解所有 (s, a)的 *BOE*，而在 *DQN* 中，损失函数定义使用了期望，因此需要分布. 但也可以强制使用 *experience replay*，这样可以反复使用一次采样的经验，能够让数据 *(sample)* 使用地更高效.

[a] 虽然在 DQN 之前也有一些将深度学习结合强化学习的尝试，但是 DQN 是效果最好的，能够在一些游戏任务上达到人类玩家水准.

DQN Algorithm

Algorithm 13 Deep Q-learning (off-policy version)

Aim: Learn an optimal target network to approximate the optimal action values from the experience samples generated by a behavior policy π_b.
Store the experience samples generated by π_b in a replay buffer $\mathcal{B} = \{(s, a, r, s')\}$.
for each iteration **do**
    Uniformly draw a mini-batch of samples from $\mathcal{B}$.
    for each sample (s, a, r, s') **do**
        Calculate the target value as ▷ w_T is the parameter of the target network

$$y_T = r + \gamma \max_{a' \in A(s')} \hat{q}(s', a', w_T),$$

    end for
    Update the **main network** to minimize

$$(y_T - \hat{q}(s, a, w))^2$$

    using the mini-batch $\{(s, a, y_T)\}$.
    Set $w_T = w$ every C iterations.
end for

Note 7.7. *1.* 由于算法*13*是 *off-policy* 的，因此不需要进行 *policy update*

2. 为利用神经网络高效的"黑盒"性，因此不再使用之前基于梯度的 *policy update* 方式

3. DQN 原文中的算法是 *on-policy* 的，与此处略有不同

8 Policy Gradient

8.1 Policy Gradient

Basic Idea of Policy Gradient

在 Lecture 7 中我们使用 value function approximation 将 state value 和 action value 从表格 (table) 形式转变为函数 (function) 形式. 事实上 policy π 也是以表格 (如 Table 8.1 所示) 形式呈现，因此同样可以函数化.

State / Action	a_1	a_2	a_3	a_4	a_5
s_1	$\pi(s_1,a_1)$	$\pi(s_1,a_2)$	$\pi(s_1,a_3)$	$\pi(s_1,a_4)$	$\pi(s_1,a_5)$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
s_9	$\pi(s_9,a_1)$	$\pi(s_9,a_2)$	$\pi(s_9,a_3)$	$\pi(s_9,a_4)$	$\pi(s_9,a_5)$

Table 8.1: Action probabilities conditioned on states

记函数化后的 policy 为 $\pi(a|s;\theta)$，其中 $\theta\in\mathbb{R}^d$ 为参数向量，也可以记其为 $\pi_\theta(a,s)$ 和 $\pi_\theta(a|s)$. 常见的做法是使用参数为 θ 的神经网络函数化 $\pi(s,a)$，输入为 s，输出为 $\pi(a_1,s;\theta),\pi(a_2,s;\theta)\ldots$.

Value function approximation 的核心问题之一是将 state value 和 action value 尽量近似准确，以保证后续 policy evaluation 和 policy improvement 足够准确，因此需要最小化目标函数 $J(w)=\mathbb{E}[(v_\pi(S)-\hat{v}(S,w))^2]$ 确保近似效果. 然而当直接优化 policy 时，我们的目标是将当前 policy 不断优化至最优 (optimal) policy. 因此我们需要定义一个 metric / objective function $J(\theta)$ 度量 policy 的好坏，进而定义 optimal policy，并最大化 $J(\theta)$.

Example 8.1. 每次从 s_0 开始一直运行到回合结束，可以将目标函数定义为: $J(\theta)=\mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T-1}\gamma^t R_{t+1}\right]=v_{\pi_\theta}(s_0)$. 例如只有两个动作：动作 *1* 得 *10* 分，动作 *2* 得 *0* 分；*policy* 以概率 p_θ 选动作 *1*，那么

$$J(\theta)=10p_\theta+0(1-p_\theta)=10p_\theta. \tag{8.1}$$

此时最大化 J 就是提高好动作的选择概率. 但如果奖励或环境转移未知，我们通常无法直接写出这个具体的 J.

Note 8.1. ① 与 *Lecture 7* 中提到的一样，使用函数形式 *(function representation)* 相较于表格 *(tabular representation)* 的优势在于 *(1) Storage (2) Generalization.* ② *Policy* 的 *tabular* 与 *function representation* 主要有三点不同：

1. 最优策略 π^* 的定义不同: *(1)* 表格形式下 *policy* π^* 满足 $v_{\pi^*}(s)\geqslant v_\pi(s),\forall s$; *(2)* 函数形式下 *policy* π^* 最大化了一个预先定义的函数 *(*作为标量度量*, scalar metric).*
2. 获得某 *action* 的概率 $\pi(s,a)$ 的方式不同: *(1)* 表格形式下直接查表; *(2)* 函数形式下需要重新代入计算一遍.
3. 更新 $\pi(s,a)$ 方式不同: *(1)* 表格形式直接更改相应概率值; *(2)* 函数形式改变参数 θ，间接改变相应概率值.

8.2 Metrics to Define Optimal Policies

Metric 1: Average Value

度量 policy π 的好坏最简单和直观的想法就是对所有的 state value 取加权平均，这样的 metric 被称为 **average state value / average value**:

$$\bar{v}_\pi=\sum_{s\in\mathcal{S}}d(s)v_\pi(s)=\boldsymbol{d}^T\boldsymbol{v}_\pi. \tag{8.2}$$

其中 $\bar{v}_\pi$ 为所有 state value 的加权平均，权重 $d(s)\geqslant 0,\ \forall s,\sum_{s\in\mathcal{S}}d(s)=1$. $\boldsymbol{v}_\pi=[\ldots,v_\pi(s),\ldots]^T\in\mathbb{R}^{|\mathcal{S}|},\boldsymbol{d}=[\ldots,d(s),\ldots]^T\in\mathbb{R}^{|\mathcal{S}|}$. 权重 $\boldsymbol{d}$ 的选择主要分为与 policy 相关和不相关两种情况.

Case 1: Independent of the Policy. 最简单的情况是 d 的选择不依赖于 π，因此也不依赖于参数 θ，这样在优化过程中计算梯度时就不需要对 d 的部分进行计算. 此时将其记为 d_0，不过即使与 π 不相关，d_0 也有不同的选择：

1. 将所有的 state 平等对待，即 $d_0(s)=1/|\mathcal{S}|$;
2. 可能只关心某一个状态 s_0 (例如固定 s_0 处出发)，此时可以赋值 $d_0(s_0)=1,\quad d_0(s\neq s_0)=0$.

Case 2: Independent of the Policy. 第二种情况是 d 的选择依赖于 π. 一个常见的做法是 $d(s)=d_\pi(s)$，即 policy π 的 stationary distribution (满足 $d_\pi^TP_\pi=d_\pi^T$).

Metric 2: Average Reward

度量 π 的另一种方式是对 immediate reward 取加权平均，称之为 **average one-step reward / average reward**:

$$\bar{r}_\pi \triangleq \sum_{s\in\mathcal{S}} d_\pi(s) r_\pi(s) = \mathbb{E}[r_\pi(S)] \tag{8.3}$$

其中 $S \sim d_\pi$ 为 stationary distribution，$r_\pi(s)$ 为 state s 处的 immediate reward，定义为:

$$r_\pi(s) \triangleq \sum_{a\in\mathcal{A}} \pi(a|s) r(s,a), \quad r(s,a) = \mathbb{E}[R|s,a] = \sum_{r,s'} rp(r,s'|s,a). \tag{8.4}$$

在论文中计算 average reward 的常见形式是对于一个给定 policy π，已经生成了一个 trajectory，其中 reward 记为 $(R_{t+1}, R_{t+2}, \ldots)$，此时该 trajectory 的 single-step reward 为:

$$\lim_{n\to\infty} \frac{1}{n}\mathbb{E}\left[R_{t+1} + R_{t+2} + \cdots + R_{t+n} | S_t = s_0\right] = \lim_{n\to\infty} \frac{1}{n}\mathbb{E}\left[\sum_{k=1}^{n} R_{t+k} | S_t = s_0\right], \tag{8.5}$$

其中 s_0 为初始 state.

Note 8.2. 对于无穷多步的 *trajectory*，初始位置已不再有意义*:*

$$\lim_{n\to\infty} \frac{1}{n}\mathbb{E}\left[\sum_{k=1}^{n} R_{t+k} | S_t = s_0\right] = \lim_{n\to\infty} \frac{1}{n}\mathbb{E}\left[\sum_{k=1}^{n} R_{t+k}\right] \overset{\text{大数定律}}{=} \sum_s d_\pi(s) r_\pi(s) = \bar{r}_\pi. \tag{8.6}$$

Note 8.3. ① 由于上述的 π 都是被 θ 参数化的，因此定义的 *metric* 都是 θ 的函数. ② 优化上面介绍的两种 *metric* 的效果实际上是一样的，因为在 *discounted case* 下有*:*

$$\bar{r}_\pi = (1-\gamma)\bar{v}_\pi. \tag{8.7}$$

③ 关于 *average value* 有如下结论 *(*证明详见附录*):*

$$J(\theta) = \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t R_{t+1}\right] = \sum_{s\in\mathcal{S}} d(s)\mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t R_{t+1} | S_0 = s\right] = \sum_{s\in\mathcal{S}} d(s) v_\pi(s) = \bar{v}_\pi. \tag{8.8}$$

Small Conclusion

两种 metric 都度量了 policy 的好坏，即都定义了相应的目标函数 $J(\theta)$，我们的目标是 $\max J(\theta)$. 事实上这两种 metric 都有两种等价的定义:

Average value:

1. $J(\theta) = \sum_{s\in\mathcal{S}} d(s) v_\pi(s)$
2. $J(\theta) = \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t R_{t+1}\right]$

Average Reward:

1. $J(\theta) = \sum_{s\in\mathcal{S}} d_\pi(s) r_\pi(s)$
2. $J(\theta) = \lim_{n\to\infty} \frac{1}{n}\mathbb{E}\left[\sum_{k=1}^{n} R_{t+k} | S_t = s_0\right]$

8.3 Gradients of the Metrics

Introduction

Metric 梯度的计算事实上是 policy gradient methods 中最复杂的部分，因为在 metric 定义中有很多的情况，例如: ① 区分使用的 metric 是 $\bar{v}_\pi, \bar{r}_\pi, \bar{v}_\pi^0$; ② 区分是否是 discounted case. 因此此处仅做简要介绍.

Gradients of the Objectives

下面直接给出策略梯度的公式:

$$\nabla_\theta J(\theta) = \sum_{s\in\mathcal{S}} \eta(s) \sum_{a\in\mathcal{A}} \nabla_\theta \pi(a|s,\theta) q_\pi(s,a), \tag{8.9}$$

其中 $J(\theta)$ 可为 $\bar{v}_\pi, \bar{r}_\pi$ 和 $\bar{v}_\pi^0$; $=$ 可以为严格相等 $(=)$、约等 $(\approx)$ 与成比例 $(\propto)$; η 为分布/ state 的权重. 具体形式如下:

$$\nabla_\theta \bar{r}_\pi \simeq \sum_s d_\pi(s) \sum_a \nabla_\theta \pi(a|s,\theta) q_\pi(s,a), \tag{8.10a}$$

$$\nabla_\theta \bar{v}_\pi = \frac{1}{1-\gamma} \nabla_\theta \bar{r}_\pi \quad \text{(in discounted case)}, \tag{8.10b}$$

$$\nabla_\theta \bar{v}_\pi^0 = \sum_{s\in\mathcal{S}} \rho_\pi(s) \sum_{a\in\mathcal{A}} \nabla_\theta \pi(a|s,\theta) q_\pi(s,a). \tag{8.10c}$$

其中式 (8.10a) 中在 discounted case 下是约等于，在 undiscounted case 下是严格等于.

由于 $\nabla_\theta \ln \pi(a|s,\theta) = \frac{\nabla_\theta \pi(a|s,\theta)}{\pi(a|s,\theta)}$，可以得到如下非常有用的形式:

$$\begin{aligned} \nabla_\theta J(\theta) &= \sum_{s\in\mathcal{S}} \eta(s) \sum_{a\in\mathcal{A}} \nabla_\theta \pi(a|s,\theta) q_\pi(s,a) = \sum_s d(s) \sum_a \pi(a \mid s,\theta) \nabla_\theta \ln \pi(a \mid s,\theta) q_\pi(s,a) \\ &= \mathbb{E}_{S\sim d}\left[\sum_a \pi(a|S,\theta) \nabla_\theta \ln \pi(a|S,\theta) q_\pi(S,a) \right] = \mathbb{E}\left[\nabla_\theta \ln \pi(A|S,\theta) q_\pi(S,A) \right] \approx \nabla_\theta \ln \pi(a|s,\theta) q_\pi(s,a). \end{aligned} \tag{8.11}$$

将梯度 $\nabla_\theta J(\theta)$ 写成期望的形式再用单次采样 $\nabla_\theta \ln \pi(a|s,\theta) q_\pi(s,a)$ 近似是为说明可以使用 *SGD* 进行优化.

Note 8.4. 为保证 $\ln \pi(a|s,\theta)$ 合理，需要保证 $\pi(a|s,\theta) > 0$，因此不能采用 *greedy* 方式. 可以使用 *softmax function* $\pi(a|s,\theta) = \frac{e^{h(s,a,\theta)}}{\sum_{a'\in\mathcal{A}} e^{h(s,a',\theta)}}$，其中 $h(s,a',\theta)$ 为另外的函数，例如神经网络，直接将其输出层定为 *softmax* 即可.

8.4 REINFORCE Algorithm

Gradient-Ascent Algorithm

我们的目的是得到最优 policy，故需要最大化 $J(\theta)$，因此此时需要使用梯度上升算法:

$$\theta_{t+1} = \theta_t + \alpha \nabla_\theta J(\theta) = \theta_t + \alpha \mathbb{E}\left[\nabla_\theta \ln \pi(A|S,\theta_t) q_\pi(S,A) \right]. \tag{8.12}$$

在实际使用中一般使用如下随机版本 (stochastic gradient ascent method):

$$\theta_{t+1} = \theta_t + \alpha \nabla_\theta \ln \pi(a_t|s_t,\theta_t) q_\pi(s_t,a_t). \tag{8.13}$$

由于 q_π 实际未知，因此需要与之前算法中一样对其进行估计，得到如下梯度上升算法公式:

$$\theta_{t+1} = \theta_t + \alpha \nabla_\theta \ln \pi(a_t|s_t,\theta_t) q_t(s_t,a_t). \tag{8.14}$$

How to do sampling? 在随机算法中我们需要通过采样将期望进行替换，即:

$$\mathbb{E}_{S\sim d, A\sim\pi}\left[\nabla_\theta \ln \pi(A|S,\theta_t) q_\pi(S,A) \right] \longrightarrow \nabla_\theta \ln \pi(a|s,\theta_t) q_\pi(s,a) \tag{8.15}$$

① 对 $S \sim d$ 采样时，d 是 π 的 long run behavior，但实际中一般无法获得 long run behaviour，因为数据收集并不容易，因此收集太多数据并不实际.

② 采样 $A \sim \pi(A|S,\theta)$ 时，a_t 应当来自当前 policy $\pi(\theta_t)$ 下的 state s_t，因此 policy gradient 是 on-policy 方法[a].

[a] 也有 off-policy 的情况，但是需要额外技巧

REINFORCE Algorithm

How to do interpret REINFORCE? 由于 $\nabla_\theta \ln \pi(a_t|s_t,\theta_t) = \frac{\nabla_\theta \pi(a_t|s_t,\theta_t)}{\pi(a_t|s_t,\theta_t)}$，因此式(8.14)可以写为：

$$\theta_{t+1} = \theta_t + \alpha \nabla_\theta \ln \pi(a_t|s_t,\theta_t) q_t(s_t,a_t) = \theta_t + \alpha \underbrace{\left(\frac{q_t(s_t,a_t)}{\pi(a_t|s_t,\theta_t)} \right)}_{\beta_t} \nabla_\theta \pi(a_t|s_t,\theta_t) \triangleq \theta_t + \alpha\beta_t \nabla_\theta \pi(a_t|s_t,\theta_t). \tag{8.16}$$

此时 $\alpha\beta_t$ 作为新的步长. 可以看出实际上式 (8.16) 是在优化 π. 因为可以将 $-\pi(a_t|s_t,\theta_t)$ 看作是目标函数 $f(\theta_t)$. 根据一阶泰勒展开，当 $\theta_{t+1} - \theta_t$ 非常小时有:

$$\pi(a_t|s_t,\theta_{t+1}) \approx \pi(a_t|s_t,\theta_t) + (\nabla_\theta \pi(a_t|s_t,\theta_t))^T (\theta_{t+1} - \theta_t) = \pi(a_t|s_t,\theta_t) + \alpha\beta_t \|\nabla_\theta \pi(a_t|s_t,\theta_t)\|^2. \tag{8.17}$$

因此若 $\beta_t > 0$ 则 $\pi(a_t|s_t,\theta_{t+1}) > \pi(a_t|s_t,\theta_t)$；若 $\beta_t < 0$ 则 $\pi(a_t|s_t,\theta_{t+1}) < \pi(a_t|s_t,\theta_t)$.

Note 8.5. 事实上 $\beta_t = \frac{q_t(s_t,a_t)}{\pi(a_t|s_t,\theta_t)}$ 是在平衡探索 *(exploration)* 和利用 *(exploitation)*.

① ***Exploration:*** 可以看到 β_t 与 $\pi(a_t|s_t,\theta_t)$ 成反比，由于 $\pi(a_t|s_t,\theta_{t+1}) - \pi(a_t|s_t,\theta_t) \approx \alpha\beta_t \|\nabla_\theta \pi(a_t|s_t,\theta_t)\|^2$，因此 $\pi(a_t|s_t,\theta_t)$ 较小 $\to \beta_t$较大 $\to \pi(a_t|s_t,\theta_{t+1})$变大. 因此当前时间步 t 选择某个 *action* 的概率小，那么下一时间步 $t+1$ 选择该 *action* 的概率会变大，因此是一种探索.

② ***Exploitation:*** 可以看到 β_t 与 $q_t(s_t,a_t)$ 成正比，因此 q_t较大 $\to \beta_t$较大 $\to \pi(a_t|s_t)$变大. 因此对于 *action value* 更大的 *action* (q_t 更大)，当前的 *policy* 倾向于选择它 ($\pi_t(a_t|s_t)$) 大，因此是一种利用.

在式(8.14) 中，如果使用了 MC 的方法得到 $q_t(s_t,a_t)$，即采样得到一个 episode，计算相应的 return(g_t)，赋值给 q_t，那么就称其为 **REINFORCE** 算法. REINFORCE 算法是最早也是最简单的 policy gradient 的算法.

Algorithm 14 Policy Gradient by Monte Carlo (REINFORCE)

Initialization: A parameterized function $\pi(a \mid s,\theta)$, $\gamma \in (0,1)$, and $\alpha > 0$.
Aim: Search for an optimal policy maximizing $J(\theta)$.
for the kth iteration **do**
  Select s_0 and generate an episode following $\pi(\theta_k)$. Suppose the episode is $\{s_0, a_0, r_1, \ldots, s_{T-1}, a_{T-1}, r_T\}$.
  ▷ **offline**(because a complete episode is required)
  for $t = 0, 1, \ldots, T-1$ **do**
    Value update:

$$q_t(s_t,a_t) = \sum_{k=t+1}^{T} \gamma^{k-t-1} r_k$$

    Policy update:

$$\theta_{t+1} = \theta_t + \alpha \nabla_\theta \ln \pi(a_t \mid s_t,\theta_t) q_t(s_t,a_t)$$

  end for
  Set $\theta_k = \theta_T$
end for

9 Actor-Critic Methods

9.1 Q-Value based Actor-Critic (QAC) Algorithm

Q-Value based Actor-Critic (QAC): the Simplest Actor-Critic

首先回顾 Policy Gradient 的基本思想:

① 定义 scalar metric $J(\theta)$ 来衡量 policy π_θ 的好坏，其可以为 $\bar{v}_\pi = \sum_{s\in\mathcal{S}} d(s)v_\pi(s)$ 或 $\bar{r}_\pi \triangleq \sum_{s\in\mathcal{S}} d_\pi(s)r_\pi(s)$.

② 为获得 optimal policy 需要最大化 $J(\theta)$，即 $\theta^* = \arg\max_\theta J(\theta)$，需要使用梯度上升 (gradient ascent):

$$\theta_{t+1} = \theta_t + \alpha\nabla_\theta J(\theta_t) = \theta_t + \alpha\mathbb{E}_{s\sim d^\pi,\, a\sim\pi_\theta}\left[\nabla_\theta \log \pi_\theta(a \mid s, \theta_t) Q^\pi(s,a)\right]. \tag{9.1}$$

③ 实际中一般将理论上的期望转化为采样的 stochastic 版本:

$$\theta_{t+1} = \theta_t + \alpha\nabla_\theta \ln \pi(a_t|s_t,\theta_t) q_t(s_t,a_t). \tag{9.2}$$

在上述过程中实际存在两种角色: (1) **Actor:** 参数为 θ 的 policy π_θ，负责选择动作; (2) **Critic:** 负责评估 Actor 的表现，使用某种算法给出 $Q^\pi(s,a)$ 或 $q_t(s_t,a_t)$ 的估计值. 因此上述过程中 Actor 根据 Critic 提供的价值估计 q 按照式 (9.2) 更新 policy，Critic 根据某种算法更新 value function. 这就是 **Actor-Critic** 方法的基本思想.

在不同的算法中 Critic 的更新方式不同，即更新 action value q 的方式不同. 如果使用 MC 那么对应的算法就称为 REINFORCE 或者 Monte Carlo policy gradient. 如果使用 TD 的方法，那么对应的算法就称为 Actor-Critic. 在 TD method 中最简单的就是使用 Q-learning 来更新 $Q^\pi(s,a)$，称之为 **Q-Value based Actor-Critic (QAC)**.

Algorithm 15 Q-Value based Actor-Critic (QAC)

1: **Aim:** Search for an optimal policy by maximizing $J(\theta)$.
2: **for** each time step t in each episode **do**
3: Generate a_t following $\pi(a \mid s_t,\theta_t)$, observe r_{t+1}, s_{t+1}, and then generate a_{t+1} following $\pi(a \mid s_{t+1},\theta_t)$. ▷ Generate the experience sample $(s_t,a_t,r_{t+1},s_{t+1},a_{t+1})$ for SARSA
4: **Critic (value update):**

$$w_{t+1} = w_t + \alpha_w\left[r_{t+1} + \gamma q(s_{t+1},a_{t+1},w_t) - q(s_t,a_t,w_t)\right]\nabla_w q(s_t,a_t,w_t)$$

5: **Actor (policy update):**

$$\theta_{t+1} = \theta_t + \alpha_\theta \nabla_\theta \ln \pi(a_t \mid s_t,\theta_t) q(s_t,a_t,w_{t+1})$$

6: **end for**

Note 9.1. ① *QAC* 中 *Critic* 使用 *SARSA* 型 *TD* 更新+ *value function approximation* ② *Actor* 即 *policy update algorithm* ③ 由于动作由当前策略采样，因此 *QAC* 是 *on-policy* ④ 随机策略本身提供探索，因此不必额外使用 *ϵ-greedy* ⑤ 但随机策略不保证探索充分，实际常加入 *entropy regularization* $J_{actor} = \mathbb{E}[\log \pi_\theta(a_t \mid s_t)\hat{A}_t + \beta\mathcal{H}(\pi_\theta(\cdot \mid s_t))]$.

9.2 Advantage Actor-Critic (A2C)

Advantage Actor-Critic (A2C): Baseline

Advantage actor-critic (A2C) 是 QAC 的一个推广，因为名称有两个 "a" 所以简称 A2C. 其核心 idea 是引入一个 baseline / 基准线以减小估计的方差:

$$\nabla_\theta J(\theta) = \mathbb{E}_{S\sim\eta, A\sim\pi}\left[\nabla_\theta \ln \pi(A|S,\theta_t)(q_\pi(S,A) - b(S))\right] \tag{9.3}$$

其中 $b(S)$ 为 S 的标量函数，被称为 baseline，其必须具备两个性质:

① **Valid:** 引入 baseline 不会对梯度 $\nabla_\theta J(\theta)$ 的值造成影响.

② **Useful:** 引入 baseline 可以降低 variance，从而提高采样结果稳定性.

Baseline Property

Proposition 9.1 (Valid Baseline)**.** 引入 *baseline* 不会对 $\nabla_\theta J(\theta)$ 的值造成影响.

Proof.

$$\begin{aligned}\mathbb{E}_{S\sim\eta,A\sim\pi}\left[\nabla_\theta \ln\pi(A|S,\theta_t)b(S)\right] &= \sum_{s\in\mathcal{S}}\eta(s)\sum_{a\in\mathcal{A}}\pi(a|s,\theta_t)\nabla_\theta\ln\pi(a|s,\theta_t)b(s) = \sum_{s\in\mathcal{S}}\eta(s)\sum_{a\in\mathcal{A}}\nabla_\theta\pi(a|s,\theta_t)b(s)\\ = \sum_{s\in\mathcal{S}}\eta(s)b(s)\sum_{a\in\mathcal{A}}\nabla_\theta\pi(a|s,\theta_t) &= \sum_{s\in\mathcal{S}}\eta(s)b(s)\nabla_\theta\sum_{a\in\mathcal{A}}\pi(a|s,\theta_t) = \sum_{s\in\mathcal{S}}\eta(s)b(s)\nabla_\theta 1 = 0.\end{aligned} \tag{9.4}$$

因此

$$\mathbb{E}_{S\sim\eta,A\sim\pi}\left[\nabla_\theta\ln\pi(A|S,\theta_t)q_\pi(S,A)\right] = \mathbb{E}_{S\sim\eta,A\sim\pi}\left[\nabla_\theta\ln\pi(A|S,\theta_t)(q_\pi(S,A)-b(S))\right]. \tag{9.5}$$

□

Proposition 9.2 (Useful Baseline)**.** 引入 适当选择的 *baseline* 可以降低 *variance*，提高采样结果稳定性.

Proof. 下面证明 $b(S)$ 会对 $\mathrm{var}(X)$ 造成影响，为此我们考虑 $\mathrm{tr}[\mathrm{var}(X)] = \mathbb{E}[X^TX] - \bar{x}^T\bar{x}$，就有

$$\mathbb{E}[X^TX] = \mathbb{E}\left[(\nabla_\theta\ln\pi)^T(\nabla_\theta\ln\pi)(q(S,A)-b(S))^2\right] = \mathbb{E}\left[\|\nabla_\theta\ln\pi\|^2(q(S,A)-b(S))^2\right]. \tag{9.6}$$

□

因此可以看出，方差较小的算法会使得在采样时采样得到的 x 更接近 $\mathbb{E}[X]$，所以我们希望通过改变 baseline 来尽量降低方差，甚至如果存在，我们希望找到一个最优的 baseline 来使得方差最小化.

Optimal Baseline: 实际上最优的 baseline 为:

$$b^*(s) = \frac{\mathbb{E}_{A\sim\pi}[\|\nabla_\theta\ln\pi(A|s,\theta_t)\|^2 q(s,A)]}{\mathbb{E}_{A\sim\pi}[\|\nabla_\theta\ln\pi(A|s,\theta_t)\|^2]}. \tag{9.7}$$

但是由于其形式复杂，在算法中通常使用如下形式的 baseline:

$$b(s) = \mathbb{E}_{A\sim\pi}[q(s,A)] = v_\pi(s). \tag{9.8}$$

Advantage Actor-Critic (A2C) Algorithm

根据前面的分析，我们令 $b(s) = v_\pi(s)$，这样得到的 gradient-ascent algorithm 为

$$\theta_{t+1} = \theta_t + \alpha\mathbb{E}\left[\nabla_\theta\ln\pi(A|S,\theta_t)[q_\pi(S,A)-v_\pi(S)]\right] \triangleq \theta_t + \alpha\mathbb{E}\left[\nabla_\theta\ln\pi(A|S,\theta_t)\delta_\pi(S,A)\right] \tag{9.9}$$

其中 $\delta_\pi(S,A) \triangleq q_\pi(S,A) - v_\pi(S)$ 称为优势函数 (advantage function). 因为根据定义 $v_\pi(S) = \sum_a \pi(a|S)q_\pi$，若 $\delta_\pi > 0$，就说明此 action 比平均的 action 要好，是有优势的.

据此得到梯度上升算法为:

$$\begin{aligned}\theta_{t+1} &= \theta_t + \alpha\nabla_\theta\ln\pi(a_t|s_t,\theta_t)[q_t(s_t,a_t)-v_t(s_t)] = \theta_t + \alpha\nabla_\theta\ln\pi(a_t|s_t,\theta_t)\delta_t(s_t,a_t)\\ &= \theta_t + \alpha\frac{\nabla_\theta\pi(a_t|s_t,\theta_t)}{\pi(a_t|s_t,\theta_t)}\delta_t(s_t,a_t) = \theta_t + \alpha\underbrace{\left(\frac{\delta_t(s_t,a_t)}{\pi(a_t|s_t,\theta_t)}\right)}_{\text{step size}}\nabla_\theta\pi(a_t|s_t,\theta_t)\end{aligned} \tag{9.10}$$

同 Lecture 8 中 Note 8.5 一样，step size 此时是探索 (exploration) 和利用 (exploitation) 的平衡. 并且由于我们更关心相对好坏，即 value 的相对值，因此这一节的 δ_t 较上一节的 q_t 更合理.

由于 $\delta_\pi(S,A) \triangleq q_\pi(S,A) - v_\pi(S)$ 的计算同时涉及到 action 和 state value，由于在期望上有:

$$\mathbb{E}[q_\pi(S,A)-v_\pi(S)|S=s_t,A=a_t] = \mathbb{E}\left[R+\gamma v_\pi(S')-v_\pi(S)|S=s_t,A=a_t\right]. \tag{9.11}$$

因此实际中一个小的技巧是将 δ_t 进行替换:

$$\delta_t = q_t(s_t,a_t) - v_t(s_t) \to r_{t+1} + \gamma v_t(s_{t+1}) - v_t(s_t). \tag{9.12}$$

此时 δ_t 只需要一个神经网络近似 v_t 即可，而原先需要两个网络分别近似 q_t 和 v_t.

Advantage Actor-Critic (A2C) Algorithm

Algorithm 16 Advantage Actor-Critic (A2C) or TD Actor-Critic(on-policy)

1: **Aim:** Search for an optimal policy by maximizing $J(\theta)$.
2: **for** each time step t in each episode **do**
3: Generate a_t following $\pi(a \mid s_t, \theta_t)$ and then observe r_{t+1}, s_{t+1}.
4: **TD error (advantage function):**

$$\delta_t = r_{t+1} + \gamma v(s_{t+1}, w_t) - v(s_t, w_t)$$

5: **Critic (value update):**

$$w_{t+1} = w_t + \alpha_w \delta_t \nabla_w v(s_t, w_t)$$

6: **Actor (policy update):**

$$\theta_{t+1} = \theta_t + \alpha_\theta \delta_t \nabla_\theta \ln \pi(a_t \mid s_t, \theta_t)$$

7: **end for**

9.3 Off-policy Actor-Critic

Introduction

在计算 policy gradient $\nabla_\theta J(\theta) = \mathbb{E}_{S\sim\eta, A\sim\pi}[*]$ 时，期望的理论计算需要使用 $A \sim \pi$，即动作是由当前策略采样得到的，因此 behavior policy 和 target policy 相同，故 QAC 和 A2C 都是 on-policy 算法. 为将算法转换为 off-policy 的，一种常用方法是使用 importance sampling[a].

[a] Importance sampling 也可以在别的地方例如 MC、TD 使用

Importance Sampling

Motivating Example. 根据大数定律，为估计某随机变量 X 的期望 $\mathbb{E}(X)$ 可多次采样得到 samples $\{x_i\}$ 然后估算:

$$\bar{x} = \sum_{i=1}^{n} \frac{1}{n} x_i \to \mathbb{E}_{X\sim p_0}[X]. \tag{9.13}$$

其中 p_0 为 X 的分布. 在实际中我们希望依据一个已知但不同于 p_0 的分布 p_1 进行采样，但最终仍用于估算 $\mathbb{E}(X)$. 这对应了 off-policy 中使用与 target policy 不同的 behavior policy 产生数据轨迹，但最终估计的是 target policy.

Importance Sampling. 实际上可以通过简单的变量替换实现使用已知的 p_1 代替 p_0 进行采样:

$$\mathbb{E}_{X\sim p_0}[X] = \sum_x p_0(x) x = \sum_x p_1(x) \underbrace{\frac{p_0(x)}{p_1(x)}}_{f(x)} x = \mathbb{E}_{X\sim p_1}[f(X)]. \Rightarrow \bar{f} \triangleq \frac{1}{n}\sum_{i=1}^{n} f(x_i), \quad \text{where } x_i \sim p_1. \tag{9.14}$$

最后得到

$$\mathbb{E}_{X\sim p_0}[X] \approx \bar{f} = \frac{1}{n}\sum_{i=1}^{n} f(x_i) = \frac{1}{n}\sum_{i=1}^{n} \frac{p_0(x_i)}{p_1(x_i)} x_i. \tag{9.15}$$

其中 $\frac{p_0(x_i)}{p_1(x_i)}$ 被称为重要性权重 (importance weight).

Note 9.2. ① 一个直观的解释是若某样本点 x_i 在 p_0 下很容易被采到但是在 p_1 下很难被采到，那么就要很珍惜这个样本点，直观上来说这个样本点的权重应该比较大，这与 $\frac{p_0(x_i)}{p_1(x_i)}$ 此时很大刚好对应.

② *Importance sampling* 的应用逻辑为：*(1)* 若可以使用 p_0 进行采样那么直接使用 $\bar{x} = \sum_{i=1}^{n} \frac{1}{n} x_i \to \mathbb{E}_{X\sim p_0}[X]$ 即可 *(2)* 若只能根据 p_1 进行采样得到 $\{x_i\}$ 并已知这些离散点的 $p_0(x_i)$ [a] *(3)* 由于定义式 $\mathbb{E}_{X\sim p_0}[X] = \sum_x p_0(x)x$ 需要知道所有点的 p_0 因此无法直接利用 *(4)* 因此最终只能通过转化成 p_1 下有限个点来计算 p_0 的期望.

[a] 例如使用一个神经网络逼近 p_0，但是此时 p_0 都没有表达式.

Off-policy Policy Gradient Algorithm

考虑 off-policy 情形，即 behavior policy 不同于 target policy π. 设此时 behavior policy 为 β. 此时 metric 写为:

$$J(\theta) = \sum_{s\in\mathcal{S}} d_\beta(s) v_\pi(s) = \mathbb{E}_{S\sim d_\beta}[v_\pi(S)], \tag{9.16}$$

其中 d_β 为 policy β 的 stationary distribution.
直接给出 $J(\theta)$ 的梯度如下:

$$\nabla_\theta J(\theta) = \mathbb{E}_{S\sim\rho, A\sim\beta}\left[\frac{\pi(A|S,\theta)}{\beta(A|S)} \nabla_\theta \ln \pi(A|S,\theta) q_\pi(S,A)\right], \tag{9.17}$$

其中 $\gamma \in (0,1)$ 为 discount factor, ρ 为 state distribution.
加上 baseline $b(s)$，得到:

$$\nabla_\theta J(\theta) = \mathbb{E}_{S\sim\rho, A\sim\beta}\left[\frac{\pi(A|S,\theta)}{\beta(A|S)} \nabla_\theta \ln \pi(A|S,\theta) \left(q_\pi(S,A) - b(S)\right)\right]. \tag{9.18}$$

根据前面的结构，令 $b(s) = v_\pi(s)$，得到

$$\nabla_\theta J(\theta) = \mathbb{E}\left[\frac{\pi(A|S,\theta)}{\beta(A|S)} \nabla_\theta \ln \pi(A|S,\theta) \left(q_\pi(S,A) - v_\pi(S)\right)\right]. \tag{9.19}$$

对应的 stochastic 版本即:

$$\theta_{t+1} = \theta_t + \alpha_\theta \frac{\pi(a_t|s_t,\theta_t)}{\beta(a_t|s_t)} \nabla_\theta \ln \pi(a_t|s_t,\theta_t) \left(q_t(s_t,a_t) - v_t(s_t)\right). \tag{9.20}$$

与 on-policy 类似地有

$$q_t(s_t,a_t) - v_t(s_t) \approx r_{t+1} + \gamma v_t(s_{t+1}) - v_t(s_t) \triangleq \delta_t(s_t,a_t). \tag{9.21}$$

如此得到算法

$$\theta_{t+1} = \theta_t + \alpha_\theta \frac{\pi(a_t|s_t,\theta_t)}{\beta(a_t|s_t)} \nabla_\theta \ln \pi(a_t|s_t,\theta_t) \delta_t(s_t,a_t) = \theta_t + \alpha_\theta \left(\frac{\delta_t(s_t,a_t)}{\beta(a_t|s_t)}\right) \nabla_\theta \pi(a_t|s_t,\theta_t). \tag{9.22}$$

Algorithm 17 Off-policy Actor-Critic Based on Importance Sampling

1: **Initialization:** A given behavior policy $\beta(a \mid s)$. A target policy $\pi(a \mid s, \theta_0)$ where θ_0 is the initial parameter vector. A value function $v(s, w_0)$ where w_0 is the initial parameter vector.
2: **Aim:** Search for an optimal policy by maximizing $J(\theta)$.
3: **for** each time step t in each episode **do**
4: Generate a_t following $\beta(\cdot \mid s_t)$ and then observe r_{t+1}, s_{t+1}.
5: **TD error (advantage function):**

$$\delta_t = r_{t+1} + \gamma v(s_{t+1}, w_t) - v(s_t, w_t)$$

6: **Critic (value update):**

$$w_{t+1} = w_t + \alpha_w \frac{\pi(a_t \mid s_t, \theta_t)}{\beta(a_t \mid s_t)} \delta_t \nabla_w v(s_t, w_t)$$

7: **Actor (policy update):**

$$\theta_{t+1} = \theta_t + \alpha_\theta \frac{\pi(a_t \mid s_t, \theta_t)}{\beta(a_t \mid s_t)} \delta_t \nabla_\theta \ln \pi(a_t \mid s_t, \theta_t)$$

8: **end for**

9.4 Deterministic Actor-Critic (DPG)

Introduction: Continuous Action Space

前面介绍的所有的算法都有 $\pi(a|s,\theta)>0$，也就是说都带有 exploration 性，都是 stochastic 策略. 但是 $\pi(a|s,\theta)>0$ 隐含假设了 action space 是有限的，因为 $\sum_a \pi(a|s;\theta)=1$. 因此对于 continuous action space 还需要额外处理.

Deterministic Actor-Critic (DPG)

下面仅考虑 deterministic 的情形，此时重新定义一个 deterministic policy:

$$a=\mu(s,\theta)\doteq\mu(s). \tag{9.23}$$

这与 $\pi(a|s;\theta)$ 不同的是直接输出 action，而不是输出采取某些 action 对应的概率值.

Note 9.3. ① μ 是 *state space* 到 *action space* 的映射：$\mu:\mathcal{S}\to\mathcal{A}$. ② μ 可用参数为 θ 的神经网络替代，输入 s 输出 a.

与之前的推导过程类似，首先定义 discounted case 下的 scalar metric:

$$J(\theta)=\mathbb{E}[v_\mu(s)]=\sum_{s\in\mathcal{S}} d_0(s)v_\mu(s), \tag{9.24}$$

其中 $d_0(s)$ 为概率分布，满足 $\sum_{s\in\mathcal{S}} d_0(s)=1$，最简单的情形是与 μ 不相关 (例如只有某 s_0 的 $d_0(s_0)=1$，或 behavior policy 的 stationary distribution).
此处直接给出 discounted case 下的梯度表达式为

$$\nabla_\theta J(\theta)=\sum_{s\in\mathcal{S}}\rho_\mu(s)\nabla_\theta\mu(s)\left(\nabla_a q_\mu(s,a)\right)|_{a=\mu(s)}=\mathbb{E}_{S\sim\rho_\mu}\left[\nabla_\theta\mu(S)\left(\nabla_a q_\mu(S,a)\right)|_{a=\mu(S)}\right], \tag{9.25}$$

其中 $\gamma\in(0,1)$ 为 discount factor, ρ_μ 为 state distribution. 注意在此梯度的表达式中最后已经没有 a，即对 target policy 的更新已经不再和 behavior policy 相关，因此天然是 off-policy 的.
再代入梯度上升算法中，得到:

$$\theta_{t+1}=\theta_t+\alpha_\theta\mathbb{E}_{S\sim\rho_\mu}\left[\nabla_\theta\mu(S)\left(\nabla_a q_\mu(S,a)\right)|_{a=\mu(S)}\right]. \tag{9.26}$$

那么得到实际使用的 stochastic 版本为:

$$\theta_{t+1}=\theta_t+\alpha_\theta\nabla_\theta\mu(s_t)\left(\nabla_a q_\mu(s_t,a)\right)|_{a=\mu(s_t)}. \tag{9.27}$$

Algorithm 18 Deterministic Actor-Critic Algorithm

1: **Initialization:** A given behavior policy $\beta(a\mid s)$. A deterministic target policy $\mu(s,\theta_0)$ where θ_0 is the initial parameter vector. A value function $v(s,a,w_0)$ where w_0 is the initial parameter vector.
2: **Aim:** Search for an optimal policy by maximizing $J(\theta)$.
3: **for** each time step t in each episode **do**
4: Generate a_t following β and then observe r_{t+1},s_{t+1}.
5: **TD error:**

$$\delta_t=r_{t+1}+\gamma q(s_{t+1},\mu(s_{t+1},\theta_t),w_t)-q(s_t,a_t,w_t)$$

6: **Critic (value update):**

$$w_{t+1}=w_t+\alpha_w\delta_t\nabla_w q(s_t,a_t,w_t)$$

7: **Actor (policy update):**

$$\theta_{t+1}=\theta_t+\alpha_\theta\nabla_\theta\mu(s_t,\theta_t)\left(\nabla_a q(s_t,a,w_{t+1})\right)\Big|_{a=\mu(s_t)}$$

8: **end for**

Note 9.4. ① *DPG* 算法是 *off-policy* 的，*behavior policy* β 可以与 *target policy* μ 不同. ② β 也可以是 μ+*noise*，这样就有了探索性 (此时就是 *on-policy* 的实现方式). ③ 当使用线性函数逼近 $q(s,a,w)$ 时，即 $q(s,a,w)=\phi^T(s,a)w$，此时算法为 *DPG*. ④ 当使用神经网络逼近 $q(s,a,w)$ 时，被称为 *deep deterministic policy gradient (DDPG)*.

A Proof of Lecture 2

Proof of Eq. (2.19)

$$\boldsymbol{v}_k \xrightarrow{k\to\infty} \boldsymbol{v}_\pi = (\boldsymbol{I}-\gamma\boldsymbol{P}_\pi)^{-1}\boldsymbol{r}_\pi$$

Proof. 首先定义残差 $\boldsymbol{\delta}_k = \boldsymbol{v}_k - \boldsymbol{v}_\pi$，我们只需要证明 $\boldsymbol{\delta}_k \to \boldsymbol{0}$ 即可. 将 $\boldsymbol{v}_{k+1} = \boldsymbol{\delta}_{k+1} + \boldsymbol{v}_\pi$ 和 $\boldsymbol{v}_k = \boldsymbol{\delta}_k + \boldsymbol{v}_\pi$ 代入 $\boldsymbol{v}_{k+1} = \boldsymbol{r}_\pi + \gamma\boldsymbol{P}_\pi\boldsymbol{v}_k$ 中得到

$$\boldsymbol{\delta}_{k+1} = -\boldsymbol{v}_\pi + \boldsymbol{r}_\pi + \gamma\boldsymbol{P}_\pi\boldsymbol{\delta}_k + \gamma\boldsymbol{P}_\pi\boldsymbol{v}_\pi = \gamma\boldsymbol{P}_\pi\boldsymbol{\delta}_k - \boldsymbol{v}_\pi + (\boldsymbol{r}_\pi + \gamma\boldsymbol{P}_\pi\boldsymbol{v}_\pi) \xlongequal{\boldsymbol{v}_\pi = \boldsymbol{r}_\pi + \gamma\boldsymbol{P}_\pi\boldsymbol{v}_\pi} \gamma\boldsymbol{P}_\pi\boldsymbol{\delta}_k.$$

递归展开可得:

$$\boldsymbol{\delta}_{k+1} = \gamma\boldsymbol{P}_\pi\boldsymbol{\delta}_k = \gamma^2\boldsymbol{P}_\pi^2\boldsymbol{\delta}_{k-1} = \cdots = \gamma^{k+1}\boldsymbol{P}_\pi^{k+1}\boldsymbol{\delta}_0.$$

由于 $\boldsymbol{P}_\pi$ 是行随机矩阵，$0 \leqslant [\boldsymbol{P}_\pi]_{ij} = p_\pi(s_j|s_i) \leqslant 1$，即 $0 \leqslant [\boldsymbol{P}_\pi^k]_{ij} \leqslant 1$，因此 $\|\boldsymbol{P}_\pi\|_\infty \leqslant 1$，故:

$$\|\boldsymbol{P}_\pi\boldsymbol{x}\|_\infty \leqslant \|\boldsymbol{P}_\pi\|_\infty \cdot \|\boldsymbol{x}\|_\infty \leqslant \|\boldsymbol{x}\|_\infty, \forall \boldsymbol{x}.$$

因此有

$$\|\boldsymbol{\delta}_{k+1}\|_\infty = \left\|\gamma^{k+1}\boldsymbol{P}_\pi^{k+1}\boldsymbol{\delta}_0\right\|_\infty \leqslant \gamma^{k+1}\|\boldsymbol{\delta}_0\|_\infty \xrightarrow[0\leqslant\gamma<1]{k\to\infty} 0 \quad \Rightarrow \quad \boldsymbol{v}_k \to \boldsymbol{v}_\pi.$$

□

Reversibility of $\boldsymbol{I}-\gamma\boldsymbol{P}_\pi$

$$\boldsymbol{I}-\gamma\boldsymbol{P}_\pi \text{ 可逆}$$

Proof. 令 $\boldsymbol{A} \triangleq \boldsymbol{I}-\gamma\boldsymbol{P}_\pi$，根据 Gershgorin circle theorem A.1，矩阵 $\boldsymbol{A}$ 的所有特征值均位于至少一个 Gershgorin disc 中，其中第 i 个 Gershgorin disc 的圆心 a_{ii} 和半径 R_i 分别为

$$a_{ii} = [\boldsymbol{I}-\gamma\boldsymbol{P}_\pi]_{ii} = 1-\gamma p_\pi(s_i \mid s_i), \quad R_i = \sum_{j\neq i}|a_{ij}| = \sum_{j\neq i}|-\gamma p_\pi(s_j \mid s_i)| = \gamma\sum_{j\neq i} p_\pi(s_j \mid s_i) = \gamma\left(1-p_\pi(s_i \mid s_i)\right).$$

对于第 i 个 Gershgorin disc 中的任意一点 $z \in D(a_{ii}, R_i)$，其实部 $\mathrm{Re}(z)$ 满足

$$\mathrm{Re}(z) \geqslant a_{ii} - R_i = 1-\gamma p_\pi(s_i \mid s_i) - \gamma\left(1-p_\pi(s_i \mid s_i)\right) = 1-\gamma > 0, \quad 0 \leqslant \gamma < 1$$

因此，所有 Gershgorin discs 均严格位于复平面的右半平面，且不包含原点. 于是 0 不可能是 $\boldsymbol{I}-\gamma\boldsymbol{P}_\pi$ 的特征值，即 $\det(\boldsymbol{I}-\gamma\boldsymbol{P}_\pi) \neq 0$. 因此 $\boldsymbol{I}-\gamma\boldsymbol{P}_\pi$ 可逆.

□

Theorem A.1 (Gershgorin circle theorem)**.** 令 $\boldsymbol{A} = [a_{ij}] \in \mathbb{C}^{n\times n}$ 为复矩阵，R_i 为第 i 行除对角元外元素之和:

$$R_i \triangleq \sum_{j\neq i}|a_{ij}|$$

令 $D(a_{ii}, R_i) \triangleq \{z \in \mathbb{C} : |z-a_{ii}| \leq R_i\} \subseteq \mathbb{C}$ 表示中心为 a_{ii}，半径为 R_i 的闭圆盘*(disc)*，称之为 *Gershgorin disc.* 那么，$\boldsymbol{A}$ 的每个特征值均位于至少一个 *Gershgorin disc* 中，即

$$\sigma(\boldsymbol{A}) \subseteq \bigcup_{i=1}^{n} D(a_{ii}, R_i),$$

其中 $\sigma(\boldsymbol{A})$ 表示 $\boldsymbol{A}$ 的特征值集合.

B Proof of Lecture 3

Contraction Mapping Theorem

Definition B.1 (Fixed Point)**.** 对于方程 $f(x), f:\mathbb{R}^d\to\mathbb{R}^d$，点 x^* 被称为不动点 *(fixed point)*，若

$$f(x^*)=x^*. \tag{B.1}$$

Definition B.2 (Contraction Mapping)**.** 记 (X,d) 为一度量空间，其中 d 为 X 上定义的 *metric.* 映射 $f:X\to X$ 被称为压缩映射 *(contraction mapping)*，若 $\exists\,\gamma\in(0,1)$*:*

$$d(f(x_1)-f(x_2))\leqslant\gamma d(x_1-x_2). \tag{B.2}$$

Note B.1. 更简单和容易理解的版本为：函数 $f:\mathbb{R}^d\to\mathbb{R}^k$ 被称为压缩映射，若 $\exists\,\gamma\in(0,1)$*:*

$$\|f(x_1)-f(x_2)\|\leqslant\gamma\|x_1-x_2\|. \tag{B.3}$$

Theorem B.1 (Contraction Mapping Theorem / Banach Fixed-Point Theorem (Non-strict version))**.** 考虑完备度量空间 (X,d) 上的映射 $f:X\to X$，特别地直接考虑 *Banach* 空间 $X,\|\cdot\|$，对于方程 $f(x)=x$，若 f 为压缩映射，那么存在唯一不动点 x^* 作为解满足 $f(x^*)=x^*$. 且可以设计如下算法迭代逼近不动点 x^**:*

$$x_{k+1}=f(x_k). \tag{B.4}$$

满足 $x_k\xrightarrow{k\to\infty}x^*$，且以指数速率收敛.

Proof. 根据 X 的完备性可知任意 Cauchy 序列必收敛，因此为证明序列 $\{x_k=f(x_{k-1})\}_{k=1}^{\infty}$ 收敛需先证明其是 Cauchy 列. 即 $\forall\,\epsilon>0,\exists\,N>0,s.t.\forall\,m,n>N,\|x_m-x_n\|\leqslant\epsilon$. 然后再证明该收敛点为唯一不动点.

① 先证明序列 $\{x_k=f(x_{k-1})\}_{k=1}^{\infty}$ 是 Cauchy 列. 根据压缩性，有

$$\begin{aligned}&\|x_{n+1}-x_n\|=\|f(x_n)-f(x_{n-1})\|\leqslant\gamma\|x_n-x_{n-1}\|=\gamma\|f(x_{n-1})-f(x_{n-2})\|\\&\leqslant\gamma^2\|x_{n-1}-x_{n-2}\|=\gamma^2\|f(x_{n-2})-f(x_{n-3})\|\leqslant\cdots\leqslant\gamma^n\|x_1-x_0\|.\end{aligned} \tag{B.5}$$

$\forall\,\epsilon>0$，取 $N\geqslant\log\left(\frac{\epsilon}{\|x_1-x_0\|}(1-\gamma)-\gamma\right)$，那么 $\forall\,m,n>N$ 根据三角不等式有

$$\begin{aligned}&\|x_m-x_n\|\leqslant\|x_m-x_{m-1}\|+\cdots+\|x_{n+1}-x_n\|\leqslant\left(\gamma^{m-1}+\cdots+\gamma^n\right)\|x_1-x_0\|\\&=\frac{\gamma^n(1-\gamma^{m-n})}{1-\gamma}\|x_1-x_0\|\leqslant\frac{\gamma^n}{1-\gamma}\|x_1-x_0\|\leqslant\epsilon.\end{aligned} \tag{B.6}$$

因此 $\{x_k=f(x_{k-1})\}_{k=1}^{\infty}$ 是 Cauchy 列.

② 证明收敛的点是不动点. 设 Cauchy 列 $\{x_k=f(x_{k-1})\}_{k=1}^{\infty}$ 收敛至 x^*. 由于 $\|f(x_k)-x_k\|=\|x_{k+1}-x_k\|\leqslant\gamma^k\|x_1-x_0\|\xrightarrow{k\to\infty}0$，因此取极限 $k\to\infty$ 后有 $f(x^*)=x^*$. 因此 x^* 是不动点.

③ 证明不动点的唯一性. 使用反证法，若不动点不唯一，设 x_1^*,x_2^* 均为不动点，那么有如下矛盾：

$$\|x_2^*-x_1^*\|=\|f(x_2^*)-f(x_1^*)\|\leqslant\gamma\|x_2^*-x_1^*\|<\|x_2^*-x_1^*\|.$$

综上 ①-③ 所述，方程 $f(x)=x$ 存在唯一解. □

Contraction Property of Bellman Optimality Equation (Theorem 3.1)

Theorem B.2 (Contraction property of $f(v)$)**.** *BOE* 中的映射 T^* 是一个压缩映射，$\forall\, \boldsymbol{v}, \boldsymbol{u} \in \mathbb{R}^{|\mathcal{S}|}$，满足

$$\|T^*\boldsymbol{u} - T^*\boldsymbol{v}\|_\infty \leqslant \gamma\|\boldsymbol{u} - \boldsymbol{v}\|_\infty, \quad \|\boldsymbol{v}\|_\infty \triangleq \max_{s\in\mathcal{S}} |v(s)|,$$

其中 $\gamma \in (0,1)$ 为 *discount rate*，$\|\cdot\|_\infty$ 为 *maximum norm* (取绝对值最大者).

Proof. 根据定义，$T^*\boldsymbol{v} \triangleq \max_\pi (\boldsymbol{r}_\pi + \gamma \boldsymbol{P}_\pi \boldsymbol{v})$. 对于任意给定的两 state value $\boldsymbol{v}_1, \boldsymbol{v}_2 \in \mathbb{R}^{|\mathcal{S}|}$，取 $\pi_1^* \triangleq \arg\max_\pi(\boldsymbol{r}_\pi + \gamma \boldsymbol{P}_\pi \boldsymbol{v}_1)$, $\pi_2^* \triangleq \arg\max_\pi(\boldsymbol{r}_\pi + \gamma \boldsymbol{P}_\pi \boldsymbol{v}_2)$，那么就有

$$\begin{aligned} T^*\boldsymbol{v}_1 &= \max_\pi(\boldsymbol{r}_\pi + \gamma \boldsymbol{P}_\pi \boldsymbol{v}_1) = \boldsymbol{r}_{\pi_1^*} + \gamma \boldsymbol{P}_{\pi_1^*}\boldsymbol{v}_1 \geqslant \boldsymbol{r}_{\pi_2^*} + \gamma \boldsymbol{P}_{\pi_2^*}\boldsymbol{v}_1, \\ T^*\boldsymbol{v}_2 &= \max_\pi(\boldsymbol{r}_\pi + \gamma \boldsymbol{P}_\pi \boldsymbol{v}_2) = \boldsymbol{r}_{\pi_2^*} + \gamma \boldsymbol{P}_{\pi_2^*}\boldsymbol{v}_2 \geqslant \boldsymbol{r}_{\pi_1^*} + \gamma \boldsymbol{P}_{\pi_1^*}\boldsymbol{v}_1, \end{aligned} \tag{B.7}$$

其中 $\geqslant$ 为逐元素比较. 这样就可以得到

$$T^*\boldsymbol{v}_1 - T^*\boldsymbol{v}_2 = \boldsymbol{r}_{\pi_1^*} + \gamma \boldsymbol{P}_{\pi_1^*}\boldsymbol{v}_1 - (\boldsymbol{r}_{\pi_2^*} + \gamma \boldsymbol{P}_{\pi_2^*}\boldsymbol{v}_2) \leqslant \boldsymbol{r}_{\pi_1^*} + \gamma \boldsymbol{P}_{\pi_1^*}\boldsymbol{v}_1 - (\boldsymbol{r}_{\pi_1^*} + \gamma \boldsymbol{P}_{\pi_1^*}\boldsymbol{v}_2) = \gamma \boldsymbol{P}_{\pi_1^*}(\boldsymbol{v}_1 - \boldsymbol{v}_2). \tag{B.8}$$

同理可得

$$\gamma \boldsymbol{P}_{\pi_2^*}(\boldsymbol{v}_1 - \boldsymbol{v}_2) \leqslant T^*\boldsymbol{v}_1 - T^*\boldsymbol{v}_2 \leqslant \gamma \boldsymbol{P}_{\pi_1^*}(\boldsymbol{v}_1 - \boldsymbol{v}_2). \tag{B.9}$$

定义

$$\boldsymbol{z} \triangleq \max\left\{|\gamma \boldsymbol{P}_{\pi_2^*}(\boldsymbol{v}_1 - \boldsymbol{v}_2)|, |\gamma \boldsymbol{P}_{\pi_1^*}(\boldsymbol{v}_1 - \boldsymbol{v}_2)|\right\} \in \mathbb{R}^{|\mathcal{S}|}, z_i = \max\{\gamma|\boldsymbol{p}_i(\boldsymbol{v}_1 - \boldsymbol{v}_2)|, \gamma|\boldsymbol{q}_i(\boldsymbol{v}_1 - \boldsymbol{v}_2)|\} \tag{B.10}$$

其中 $\boldsymbol{p}_i, \boldsymbol{q}_i$ 分别为 $P_{\pi_1^*}, P_{\pi_2^*}$ 每一行元素，满足行和为 1，$\boldsymbol{p}_i \leqslant \mathbf{1}, \boldsymbol{q}_i \leqslant \mathbf{1}$，那么就有:

$$\|T^*\boldsymbol{v}_1 - T^*\boldsymbol{v}_2)\|_\infty \leqslant \|\boldsymbol{z}\|_\infty, \quad |\boldsymbol{p}_i(\boldsymbol{v}_1 - \boldsymbol{v}_2)| \leqslant \boldsymbol{p}_i|\boldsymbol{v}_1 - \boldsymbol{v}_2| \leqslant \|\boldsymbol{v}_1 - \boldsymbol{v}_2\|_\infty \Rightarrow z_i \leqslant \gamma\|\boldsymbol{v}_1 - \boldsymbol{v}_2\|_\infty \tag{B.11}$$

从而

$$\|T^*\boldsymbol{v}_1 - T^*\boldsymbol{v}_2\|_\infty \leqslant \|\boldsymbol{z}\|_\infty = \max_i |z_i| \leqslant \gamma\|\boldsymbol{v}_1 - \boldsymbol{v}_2\|_\infty. \tag{B.12}$$

因此 T^* 为压缩映射. □

Optimality

Theorem B.3. 通过 *Bellman Optimal Equation* 求解得到的 v^* 是 *optimal state value*，π^* 是 *optimal policy*，即

$$v^* = v_{\pi^*} \geqslant v_\pi, \forall\pi, \forall v. \tag{B.13}$$

Proof. 已知 $v_\pi = r_\pi + \gamma P_\pi v_\pi$，根据定义

$$\begin{aligned} &v^* = \max_\pi(r_\pi + \gamma P_\pi v^*) = r_{\pi^*} + \gamma P_{\pi^*} v^* \geqslant r_\pi + \gamma P_\pi v^* \Rightarrow v^* - v_\pi \geqslant (r_\pi + \gamma P_\pi v^*) - (r_\pi + \gamma P_\pi v_\pi) = \gamma P_\pi(v^* - v_\pi) \\ &\Rightarrow v^* - v_\pi \geqslant \lim_{n\to\infty} \gamma^n P_\pi^n (v^* - v_\pi) = 0 \end{aligned} \tag{B.14}$$

最后的不等式是由于 $\gamma < 1$，P_π 每个元素 $0 \leqslant p_i \leqslant 1$. □

Optimal Policy Invariance

Theorem B.4 (Optimal Policy Invariance)**.** 考虑一马尔可夫决策过程 *(MDP)*，其中$\boldsymbol{v}^* \in \mathbb{R}^{|S|}$ 是满足

$$\boldsymbol{v}^* = \max_{\pi\in\Pi}(\boldsymbol{r}_\pi + \gamma \boldsymbol{P}_\pi \boldsymbol{v}^*)$$

的 *optimal state value*. 如果对 *reward* $\boldsymbol{r} \in \mathbb{R}$ 进行仿射变换 $\alpha\boldsymbol{r} + \beta$，其中 $\alpha, \beta \in \mathbb{R}$ 且 $\alpha > 0$，则相应的 *optimal state value* $\boldsymbol{v}'$ 也将是 $\boldsymbol{v}^*$ 的仿射变换:

$$\boldsymbol{v}' = \alpha\boldsymbol{v}^* + \frac{\beta}{1-\gamma}\mathbf{1},$$

其中 *discount rate* $\gamma \in (0,1)$，$\mathbf{1} = [1, \ldots, 1]^T$. 因此从 $\boldsymbol{v}'$ 导出的 *optimal state value* 对于 *reward* 的仿射变换保持不变.

Proof. 根据 matrix form BOE，令 $r_\pi = [\dots, r_\pi(s_k), \dots]^T$，其中

$$r_\pi(s) = \sum_{a\in\mathcal{A}} \pi(a|s) \sum_{r\in\mathcal{R}} p(r|s,a)r, \quad s \in \mathcal{S}.$$

由于 $r' = \alpha r + \beta$，那么 $r'_\pi(s) = \alpha r_\pi(s) + \beta \Rightarrow r'_\pi = \alpha r_\pi + \beta\mathbf{1}$，其中 $\mathbf{1} = [1, \dots, 1]^T$. 这样得到

$$v' = \max_{\pi\in\Pi}(\alpha r_\pi + \beta\mathbf{1} + \gamma P_\pi v'). \tag{B.15}$$

不妨设 $v' = \alpha v^* + c\mathbf{1}$，带入式(B.15)中就得到

$$\alpha v^* + c\mathbf{1} = \max_{\pi\in\Pi}(\alpha r_\pi + \beta\mathbf{1} + \gamma P_\pi(\alpha v^* + c\mathbf{1})) \xlongequal{P_\pi\mathbf{1}=\mathbf{1}} \max_{\pi\in\Pi}(\alpha r_\pi + \beta\mathbf{1} + \alpha\gamma P_\pi v^* + c\gamma\mathbf{1}).$$

进而得到

$$\alpha v^* = \max_{\pi\in\Pi}(\alpha r_\pi + \alpha\gamma P_\pi v^*) + \beta\mathbf{1} + c\gamma\mathbf{1} - c\mathbf{1} \Rightarrow \beta\mathbf{1} + c\gamma\mathbf{1} - c\mathbf{1} = 0 \Rightarrow c = \frac{\beta}{(1-\gamma)}. \tag{B.16}$$

因此得到

$$v' = \alpha v^* + \frac{\beta}{1-\gamma}\mathbf{1}. \tag{B.17}$$

□

C Proof of Lecture 5

Last updated: October 15, 2026

References

[1] Richard S Sutton, Andrew G Barto, et al. *Reinforcement learning: An introduction*, volume 1. MIT press Cambridge, 1998.